PLOTTING FUNCTIONS ON THE DM42

From First Graphs to Parametric Curves, Vector Fields, Fractals and 3-D Plots

A pixels-first programming handbook

Full-resolution 400×240 plotting • complete programs • exact keystrokes

v0.51

Welcome to the plotting laboratory

The DM42 has no built-in command meaning “draw the graph of this equation.” It has something more revealing: a monochrome screen, commands that can light pixels, and a programmable calculator able to evaluate the equation repeatedly.

That is enough. A graph emerges when a program repeatedly performs four jobs:

1. choose a mathematical point;
2. convert that point into a screen position;
3. reject positions that should not be joined or cannot be seen;
4. light the appropriate pixel or connect it to the previous one.

The book begins with one pixel and $y = x$. The machinery then grows only when a plot gives us a reason to add it: axes, joining, clipping, domain checks, discontinuity tests, parametric stepping, contours, projection and iteration. By the time the Mandelbrot set appears, it is still the same idea applied with greater patience.

Central idea

The DM42 does not know what a graph is. It knows pixels. A plotting program converts mathematical coordinates into screen coordinates and lights the correct pixels.

What this book assumes

You should know how to enter numbers and perform ordinary RPN arithmetic. Short refreshers explain the stack decisions that matter. No prior graphics programming is assumed. Every unfamiliar plotting word is defined when it first becomes useful and again in the glossary.

This is a calculator-programming book. Native applications, firmware programming and low-level display drivers are outside its scope.

How to read the keys

Physical keys are boxed. A label such as [PIXEL] means the softkey immediately below that on-screen menu label. Read sequences from left to right.

[f] [PGM.FCN] [▼] [▼] [PIXEL]

This opens PGM.FCN, moves from page 1 to page 3 and selects `PIXEL`. In program mode that sequence inserts a `PIXEL` instruction.

XEQ is used when a named program or subroutine is actually executed or called. For example, while editing a program, XEQ M A P X ENTER inserts `XEQ "MAPX"`. Ordinary functions are entered from their physical key or documented menu route.

Program-entry conventions

Start a clean program area with:

f GTO · · f PRGM

The first pair enters `GTO..`; the second enters program mode. A final `END` already exists. Enter each instruction before it. Leave program mode with f PRGM.

For an alpha name, an entry such as M A P X means the physical keys carrying those blue alpha legends. The mixed-case virtual name `GrMod` requires DM uppercase for `G` and `M`, and DM lowercase for `r`, `o` and `d`; appendix A.3 gives the exact mode switches.

Contents

I Pixels first **1**

1 A 400 by 240 field of decisions **2**

1.1 The three usable graphics modes 2

1.2 Where is the origin? 2

1.3 The graphics commands we need 3

1.4 One pixel, completely 3

1.5 The same test in three modes 4

1.6 AGRAPH composition flags 4

2 The first plot: $y = x$ **6**

2.1 Version 1: one sample, one pixel 6

2.2 Version 2: join neighbours 8

2.3 Version 3: clipping and broken curves 8

3 From one program to a plotting toolkit **10**

3.1 The window contract 10

3.2 World to screen 10

3.3 Routine map 11

3.4 The `CPLOT` driver 11

3.5 Enter the core 11

3.6 A proper simple graph 12

II Functions become pictures **13**

4 Sixteen Cartesian laboratories **14**

4.1 Optional ticks and values 14

4.2 Linear function: slope and intercept 15

4.3 Quadratic: curvature and a vertex 16

4.4 Cubic: roots and turning points 16

4.5 Fifth-order polynomial: clipping is part of the picture . . . 17
4.6 Absolute value: a genuine corner . . . 18
4.7 Reciprocal: the line that must not be drawn . . . 19
4.8 Square root: a domain is a rule, not an error message . . . 20
4.9 Exponential: choose a window before it chooses one for you . . . 21
4.10 Natural logarithm: the inverse shape . . . 22
4.11 Sine and cosine: periodic comparison . . . 22
4.12 Tangent: repeated discontinuities . . . 23
4.13 Gaussian bell: a localised peak . . . 24
4.14 Logistic sigmoid: two horizontal limits . . . 24
4.15 Damped sine: two processes at once . . . 25
4.16 Fourier square-wave approximation . . . 26
4.17 Four scientific adapters . . . 27

5 Windows, sampling and performance **28**
5.1 Three questions before every plot . . . 28
5.2 Aliasing . . . 28
5.3 Clipping versus discontinuity . . . 29
5.4 Aspect ratio . . . 29
5.5 Resolution-independent routines . . . 29
5.6 Refresh control when waiting becomes noticeable . . . 29
5.7 A practical optimisation order . . . 30

6 Parametric and polar curves **31**
6.1 The parametric contract . . . 31
6.2 Circle: one angle chooses two coordinates . . . 31
6.3 Ellipse and rotated ellipse . . . 32
6.4 Lissajous figures . . . 34
6.5 Cycloid: a rolling circle leaves a trace . . . 34
6.6 Epicycloid and hypotrochoid . . . 35
6.7 The butterfly showcase . . . 36
6.8 Polar plotting is parametric plotting . . . 37
6.9 A branch curve without a contour algorithm . . . 39
6.10 Parametric quick table . . . 41

III Beyond a single curve **42**

7 Implicit curves, contours and fields **43**
7.1 Cassini ovals: geometry before algorithm . . . 43
7.2 From a threshold to Marching Squares . . . 44

7.3 A smooth implicit test . . . 44
7.4 Two-wave interference . . . 45
7.5 Equipotential curves . . . 45
7.6 An arrow routine turns numbers into a field . . . 46
7.7 Slope fields . . . 49
7.8 Phase space . . . 49
7.9 Streamlines . . . 50

8 Recursion, iteration and dynamical systems 52
8.1 Geometric fractals: replace, repeat, stop . . . 52
8.2 Iterated point fractals . . . 54
8.3 The logistic map as a laboratory . . . 55
8.4 Two planar attractors . . . 57
8.5 Continuous systems by small steps . . . 58

9 Three dimensions on a two-dimensional screen 60
9.1 Rotation and orthographic projection . . . 60
9.2 The 3-D routine contract . . . 60
9.3 A wireframe cube before surfaces . . . 61
9.4 3-D helix . . . 61
9.5 Plane . . . 61
9.6 Sphere . . . 62
9.7 Four height surfaces . . . 62
9.8 Radial surfaces . . . 63
9.9 Two-source waves and a potential well . . . 64
9.10 Torus and Möbius strip . . . 64

10 Complex-plane fractals 66
10.1 Mandelbrot: understand one pixel first . . . 66
10.2 Preview, symmetry and progressive work . . . 67
10.3 Julia sets: fix C, vary the starting point . . . 67
10.4 Burning Ship: a small formula change . . . 68
10.5 Newton fractal: classify convergence, not escape . . . 69
10.6 Complex-number operations on the DM42 . . . 70
10.7 The final lesson . . . 70

A Keystroke atlas 71
A.1 Program editing . . . 71
A.2 Program-function pages . . . 71
A.3 Exact mixed-case virtual names . . . 71
A.4 Frequently used functions . . . 72

B Complete program listings and entry tables **73**
B.1 First-pixel and first-line programs 73
B.2 Cartesian toolkit 74
B.3 Parametric toolkit 90
B.4 Threshold contour toolkit 94
B.5 3-D projection and wireframe toolkit 96
B.6 Mandelbrot renderer 101

C Bitmap labels with AGRAPH **105**

D Quick reference **106**
D.1 Modes and dimensions 106
D.2 Coordinate formulae 106
D.3 Common windows 106
D.4 Tick controls 106
D.5 Domain checklist 107
D.6 Discontinuity checklist 107
D.7 Troubleshooting 107

E Glossary **108**

F Complete plot catalogue **109**

Part I

Pixels first

CHAPTER 1

A 400 by 240 field of decisions

The display is monochrome: each graphics pixel is either off or on. There is no shade stored between those states. Apparent gray in a dense fractal is only a spatial pattern of black and white pixels.

1.1 The three usable graphics modes

The virtual variable `GrMod` selects the logical graphics surface. Store the mode number; the current program and the graphics commands then use the corresponding coordinate limits.

`GrMod`	Resolution	Valid `PIXEL` X	Valid `PIXEL` Y
0	131×16	1–131	1–16
1	reserved; behaves as mode 0	1–131	1–16
2	200×120	1–200	1–120
3	400×240	1–400	1–240

Store mode 3 directly:



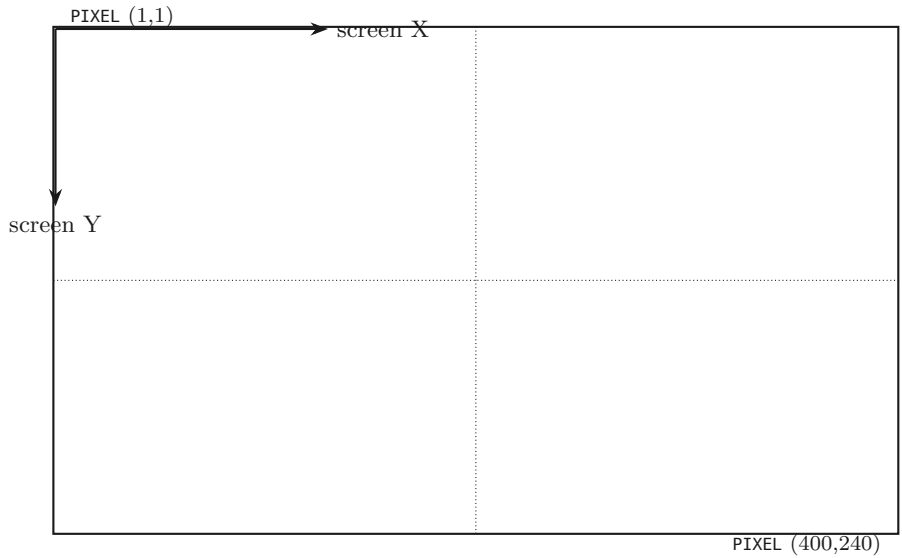
The alpha-mode switches assume the prompt begins in DM uppercase `[A]`. The displayed instruction is `STO "GrMod"`. Mode 3 is the default for the rest of this handbook.

Plotting trap

The drawing interface is *one-based*. The leftmost column is X=1, not X=0; the lowest valid row number is Y=1. Internally we will often count a clean zero-based column $u = 0, \ldots, 399$, then supply $u+1$ to `PIXEL`. This distinction prevents the classic invisible-column and off-by-one bugs.

1.2 Where is the origin?

The screen origin used by the drawing commands is at the upper left. X increases to the right; Y increases downward.



Mathematical Y usually increases upward, so every ordinary graph needs a Y reversal. It is not an arbitrary patch; it reconciles two different coordinate conventions.

1.3 The graphics commands we need

CLLCD Clears the graphics display. Enter it with f CLEAR ▼ CLLCD.

PIXEL Uses X from the X-register and Y from the Y-register. Enter it with f PGM.FCN ▼ ▼ PIXEL. Positive coordinates set one pixel. A negative X draws the entire vertical line at $|X|$; a negative Y draws the entire horizontal line at $|Y|$.

AGRAPH Draws the bit pattern in Alpha at the X,Y location. It is the third softkey on page 3 of PGM.FCN; PIXEL is the fourth. We use it later for compact bitmap labels.

WIDTH, HEIGHT Return the active display dimensions. They are extended catalog functions. The DM42-specific read-only virtual variables ResX and ResY return the same logical dimensions and are convenient in programs that already use virtual variables.

RefLCD Controls physical refresh. A value of 0 suspends periodic refresh of all zones; 1 enables the main graphics area. A negative mask forces the selected zone to refresh immediately without changing the stored setting. We postpone it until plots are slow enough to benefit.

1.4 One pixel, completely

The following program selects mode 3, clears the display and turns on the pixel at X=200, Y=120. Because PIXEL expects X in X and Y in Y, enter 120 first.

```
LBL "DOT"
3
STO "GrMod"
CLLCD
120
ENTER
200
PIXEL
RTN
```

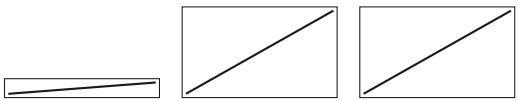
Exact DM42 keystrokes

Line	Keys
01	f PGM.FCN LBL D O T ENTER
02	3
03	STO ENTER; type GrMod as shown above, then ENTER
04	f CLEAR ▼ CLLCD
05	1 2 0
06	ENTER
07	2 0 0
08	f PGM.FCN ▼ ▼ PIXEL
09	f PGM.FCN RTN

Leave program mode with f PRGM. Run it with XEQ, select DOT from the program catalog and press its softkey. To call it by spelling the label, use XEQ D O T ENTER.

1.5 The same test in three modes

Mode 0 is a narrow compatibility canvas. In mode 2, each logical pixel covers a 2×2 block of physical pixels, producing a 200×120 grid. Mode 3 addresses the physical display one-for-one and exposes the full 400×240 field. A diagonal specified by the same *fractions* of width and height has progressively finer steps. The coordinate values themselves must respect the active mode.



Unless a preview is explicitly requested, every later framebuffer uses the full mode-3 resolution of 400 by 240 pixels.

1.6 AGRAPH composition flags

AGRAPH combines the source pattern with existing pixels according to flags 34 and 35.

F34	F35	Result at destination
0	0	destination OR source; normal additive drawing
0	1	source replaces the covered destination rectangle
1	0	destination AND NOT source; erase the source-shaped pixels
1	1	destination XOR source; toggle the source-shaped pixels

Clear both flags for ordinary labels. The program-entry routes are f FLAGS CF 3 4 and the same sequence ending in 3 5.

Try this

Change line 07 of `DOT` to 1, then 400. Keep line 05 at 120. These are the leftmost and rightmost valid columns. Try 0 only to observe that it is outside the plotting surface; restore a valid value afterward.

CHAPTER 2

The first plot: $y = x$

We choose the world window

$$x_{\min} = -5, \quad x_{\max} = 5, \quad y_{\min} = -5, \quad y_{\max} = 5.$$

The screen has 400 columns. Our zero-based internal counter therefore runs from $u = 0$ to $u = 399$. Each column chooses one mathematical X coordinate:

$$x = x_{\min} + \frac{u}{W-1}(x_{\max} - x_{\min}) = -5 + \frac{10u}{399}.$$

For $y = x$, the function evaluation is trivial. The row is the part that deserves care. Let $v = 0, \ldots, 239$ be a zero-based row. Because mathematical Y grows upward while screen rows grow downward,

$$v = (H-1) - \frac{y - y_{\min}}{y_{\max} - y_{\min}}(H-1) = \frac{5-y}{10}\,239.$$

Finally, `PIXEL` receives X equal to $u + 1$ and Y equal to $v + 1$. The $+1$ converts our convenient zero-based counter to the DM42 drawing command's one-based coordinates.

2.1 Version 1: one sample, one pixel

```
LBL "LINE1"
3
STO "GrMod"
CLLCD
0
STO 00
LBL 00
RCL 00
399
÷
10
×
5
-
STO 01
5
RCL- 01
10
÷
239
×
1
+
STO 02
RCL 02
RCL 00
1
+
PIXEL
```

```
39  RCL 00
40  1
41  +
42  STO 00
43  RCL 00
44  400
45  X>Y?
46  GTO 00
47  RTN
```

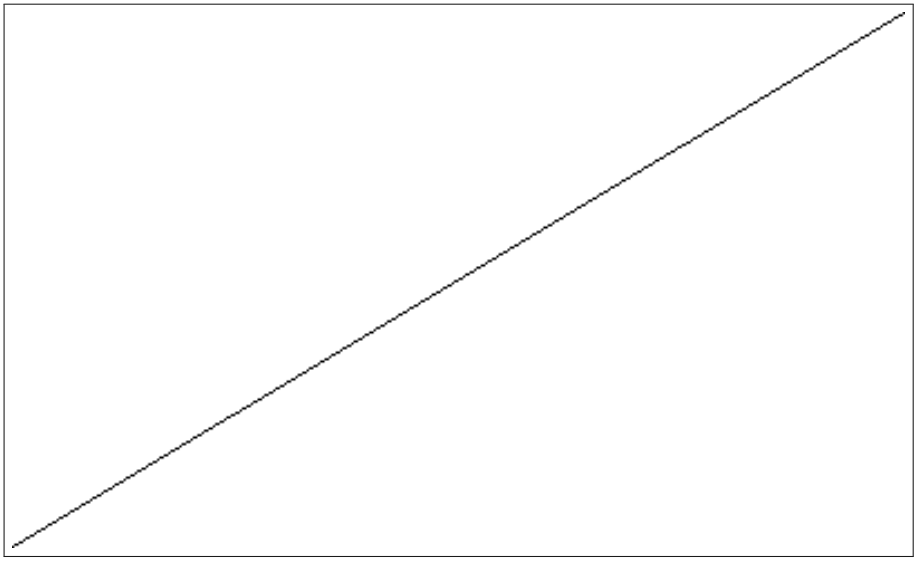
Registers 00, 01 and 02 hold u, mathematical x, and the one-based row. The final comparison works because, at that moment, Y contains the incremented counter and X contains 400: `X>Y?` is true while $400 > u$.

Exact DM42 keystrokes: complete LINE1 entry

Line	Exact keys
10	f PGM.FCN LBL L I N E f 1 ENTER
11	3
12	STO ENTER type `GrMod` using the case sequence in appendix A.3, ENTER
13	f CLEAR ▼ CLLCD
14	0
15	STO 0 0
16	f PGM.FCN LBL 0 0
17	RCL 0 0
18	3 9 9
19	÷
20	1 0
21	×
22	5
23	−
24	STO 0 1
25	5
26	RCL − 0 1
27	1 0
28	÷
29	2 3 9
30	×
31	1
32	+
33	STO 0 2
34	RCL 0 2
35	RCL 0 0
36	1
37	+
38	f PGM.FCN ▼ ▼ PIXEL
39	RCL 0 0
40	1
41	+
42	STO 0 0
43	RCL 0 0
44	4 0 0
45	f PGM.FCN ▼ X?Y X>Y?
46	f GTO 0 0
47	f PGM.FCN RTN

Leave program mode with f PRGM. Then run it with XEQ, select LINE1, and press its softkey.

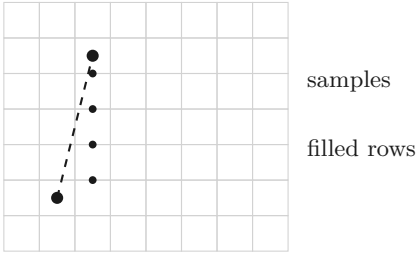
Expected DM42 plot



This is the display produced by `LINE1`: only the diagonal is drawn. The isolated samples are close enough that a shallow line appears continuous at normal size.

2.2 Version 2: join neighbours

A steep graph can change several rows while the loop advances by one column. One dot per column then leaves vertical gaps. The compact `VJOIN` routine stores the previous row, determines the sign and size of the row change, and draws every intermediate row at the new column. It is not a general line algorithm; it is precisely matched to a Cartesian plotter that advances one column at a time.



The consolidated program in appendix B.2 supplies `VJOIN` and the reusable `CPLOT`. Enter it after understanding `LINE1`; it solves a problem you have now seen.

2.3 Version 3: clipping and broken curves

A mathematical point may exist while lying outside the visible graph. The plotter checks whether $y_{\min} \le y \le y_{\max}$ before mapping it. If the point is outside, it clears

the “previous point is usable” register. That deliberate break matters: the next visible point must not be joined to a point on the far side of an invisible excursion.

Clipping alone does not detect an asymptote. Two individually valid samples can lie on opposite sides of a vertical asymptote. `CPLOT` therefore also breaks a segment when adjacent mapped rows differ by more than 80 pixels. This is a practical heuristic, not a theorem. A very steep legitimate curve may also be broken; a narrow false bridge can still survive if sampling is too coarse. chapter 5 develops the diagnosis.

Try this

In `LINE1`, change the factor 10 on line 20 to 20 without changing the window calculation. Predict where the line will leave the screen. Then run it. This deliberately inconsistent edit is a useful way to feel what clipping has to solve.

CHAPTER 3

From one program to a plotting toolkit

The first program mixed three jobs: choosing X, evaluating the function and mapping Y. Reuse becomes worthwhile now because the same coordinate conversion will serve dozens of equations.

3.1 The window contract

Before `CPLOT` runs, store the world window in four registers:

Register	Meaning	Register	Meaning
00	$x_{\min}$	01	$x_{\max}$
02	$y_{\min}$	03	$y_{\max}$

The plotter stores the current world X in R05 and expects `YFUN` to return (f(x)) in X. Flag 00 means "this result is valid." Ordinary functions set it; domain-restricted functions clear it before returning a harmless placeholder.

3.2 World to screen

With one-based DM42 coordinates, the reusable mappings are

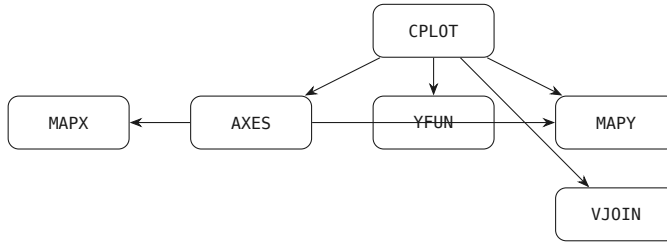
$$x_s = 1 + \frac{x - x_{\min}}{x_{\max} - x_{\min}}(399), \quad y_s = 1 + \frac{y_{\max} - y}{y_{\max} - y_{\min}}(239).$$

`MAPX` and `MAPY` implement exactly these expressions. They return real coordinates; `PIXEL` rounds them to the nearest integer pixel.

The inverse mappings are equally important when a screen location chooses the mathematical input:

$$x = x_{\min} + \frac{x_s - 1}{399}(x_{\max} - x_{\min}), \quad y = y_{\max} - \frac{y_s - 1}{239}(y_{\max} - y_{\min}).$$

3.3 Routine map



GSET Selects 400×240, radians and a clean display.

MAPX, MAPY Convert one world coordinate in X to a one-based screen coordinate in X.

AXES Draws the frame and the zero axes. Negative PIXEL coordinates make the four full-length lines economical. The optional tick layer introduced in chapter 4 is controlled by R10, R11 and R17.

VJOIN Fills the vertical run between two adjacent Cartesian samples.

CPLOT Samples 400 columns, calls YFUN, clips, breaks suspicious jumps and plots.

YFUN The replaceable mathematical function. It reads world X from R05 and returns Y.

3.4 The CPLOT driver

CPLOT is the main Cartesian plotting routine. It contains no particular equation; instead, it supplies each world X value to YFUN and turns the returned Y value into pixels. This separation is what lets one tested plotter draw every Cartesian example in the next chapter.

One pass through CPLOT has six stages:

Stage	What the program does
1	Clear the display, call AXES, and reset the column and previous-point state.
2	Convert the current screen column to world X and store it in R05.
3	Call YFUN; store its result in R06 and inspect flag 00.
4	Reject an invalid or vertically clipped result, breaking the previous-point chain.
5	Map a valid Y to a screen row; draw one pixel or call VJOIN when joining is safe.
6	Advance the column and repeat until all 400 columns have been sampled.

The global label CPLOT is part of the core block in appendix B.2, immediately after VJOIN and before the replaceable YFUN. The optional tick and numeric-label routines follow the core block; they support AXES but do not replace or redefine CPLOT.

3.5 Enter the core

The complete listing and every line's key route are in appendix B.2. After entering it, leave program mode with f PRGM, then set the $y = x$ window:

5 +/- STO 0 0 5 STO 0 1 5 +/- STO 0 2 5 STO 0 3

Run GSET, then CPLOT:

XEQ G S E T ENTER XEQ C P L O T ENTER

3.6 A proper simple graph

The default YFUN returns R05, so the result is $y = x$. AXES adds the frame and zero axes. At this stage leave R10 and R11 at zero, which keeps tick marks off. Chapter 4 turns them on deliberately and explains the three user settings. This separation is useful: coordinate mapping decides where a mathematical point belongs, while ticks and numbers merely annotate that map.

Plotting trap

Do not confuse a square screen region with equal mathematical scaling. The display ratio is $400 : 240 = 5 : 3$. A window $[-5, 5] \times [-5, 5]$ gives different pixels-per-unit horizontally and vertically; a geometric circle will look stretched. For equal scale, choose the Y span as $3/5$ of the X span, or reserve a centred square viewport.

Try this

Keep $x \in [-5, 5]$, but set $y \in [-3, 3]$. Plot the parametric circle later with this window. It should look circular because ten world units across correspond to six world units high, matching the physical 5:3 screen aspect.

Part II

Functions become pictures

CHAPTER 4

Sixteen Cartesian laboratories

Every example in this chapter uses `CPLOT`. Store the four stated window limits in R00–R03, make the displayed `YFUN` the active version, run `GSET`, then run `CPLOT`. From this chapter onward the figures also use a small number of adjustable axis ticks.

4.1 Optional ticks and values

The plotting toolkit reserves three user settings:

Register	Meaning
R10	Spacing between ticks on the horizontal axis; zero disables this tick family.
R11	Spacing between ticks on the vertical axis; zero disables this tick family.
R17	Value-label switch: zero draws marks only; any non-zero value also labels integer ticks from −9 through 9.

Read R10 = 2 as "put a tick every two X units." The tick spacing is measured in world units, not pixels. A family is drawn only when its zero axis is visible. Ticks outside the small integer label range remain useful marks but receive no number; non-integer spacing likewise produces marks without misleading rounded labels.

For the first graph below, enter:

2 STO 1 0 5 STO 1 1 1 STO 1 7

This means X ticks every 2 units, Y ticks every 5 units, with suitable integer values shown. To keep the axes but remove all ticks, store zero in R10 and R11. To retain ticks but remove their values, store zero in R17.

The following settings keep each example readable. They are starting points, not compulsory choices.

Examples	R10	R11	R17
Linear, quadratic, absolute value, reciprocal	2	5 or 2	1
Cubic and fifth-order polynomial	1	2 or 5	1
Square root, exponential, logarithm	5 or 2	1 or 5	1
Sine/cosine, tangent and Fourier sum	2	1 or 2	1
Gaussian, logistic and damped sine	2 or 5	0.5	1

After choosing the annotation settings, the standard first two lines of every adapter are:

```
LBL "YFUN"
SF 00
```

Enter them with f PGM.FCN LBL, spell YFUN, ENTER; then f FLAGS SF 0 0. The final RTN is f PGM.FCN RTN. All other key routes used below appear in the keystroke atlas, appendix A.

4.2 Linear function: slope and intercept

What we are plotting

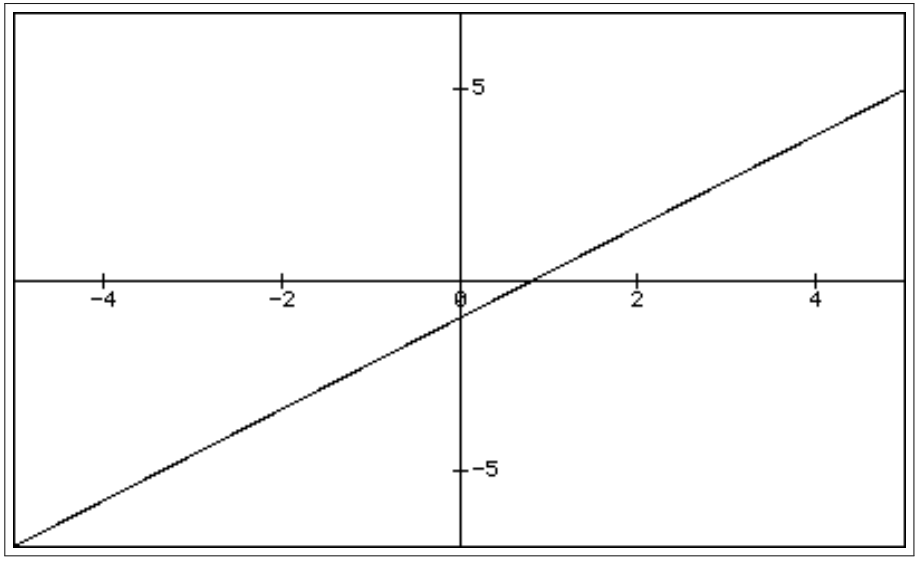
$$y = 1.2x - 1, \quad -5 \le x \le 5, \quad -7 \le y \le 7.$$

The coefficient 1.2 is the slope; -1 is the Y-intercept. We expect one straight line crossing the Y-axis at -1.

```
LBL "YFUN"
SF 00
RCL 05
1.2
×
1
-
RTN
```

Exact DM42 keystrokes

After the standard first lines: RCL 0 5 1 . 2 × 1 −, then f PGM.FCN RTN.



Try this

Change 1.2 to -2, then change the final subtraction to addition. Say how slope and intercept should change before running the graph.

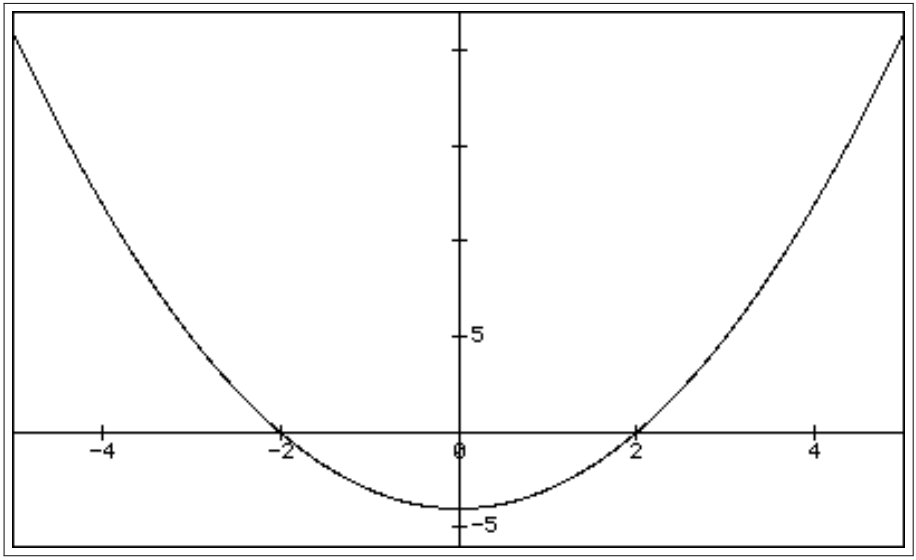
4.3 Quadratic: curvature and a vertex

$$y = x^2 - 4, \quad x \in [-5, 5], \quad y \in [-6, 22].$$

The minimum is at $(0, -4)$, and symmetry predicts roots at $x = \pm 2$.

```
LBL "YFUN"
SF 00
RCL 05↑
X2
4
-
RTN
```

Keys after `SF 00`: RCL 0 5 f x^2 4 − f PGM.FCN RTN.



Vertical window choice matters: a $[-5, 5] \times [-5, 5]$ window would hide most of the arms and make the parabola look much sharper.

4.4 Cubic: roots and turning points

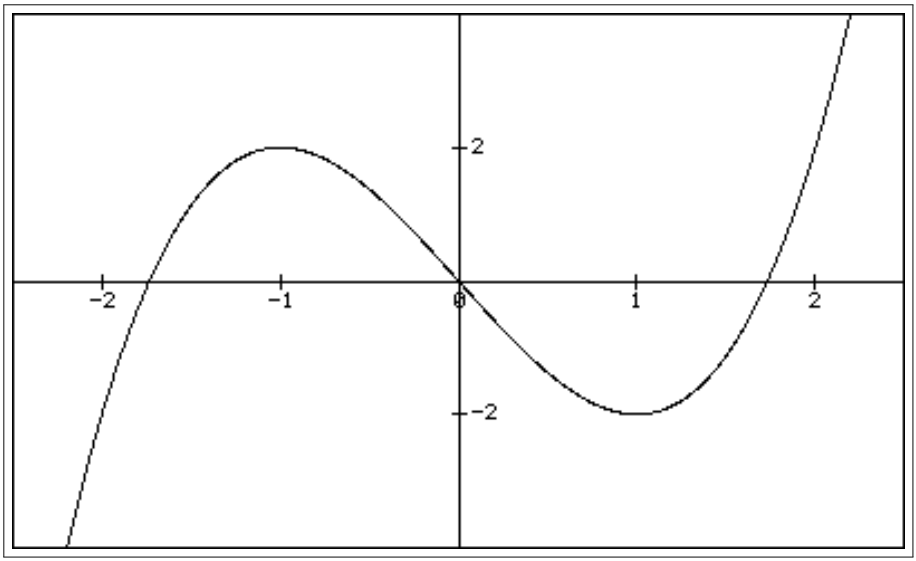
$$y = x^3 - 3x = x(x^2 - 3), \quad [-2.5, 2.5] \times [-4, 4].$$

The factored form predicts roots at 0 and $\pm\sqrt{3}$. The odd symmetry $f(-x) = -f(x)$ predicts a half-turn about the origin.

```
LBL "YFUN"
SF 00
RCL 05
ENTER↑
```

```
X2
×
RCL 05
3
×
-
RTN
```

The RPN phrase RCL 0 5 ENTER f x^2 × forms x^3. The next phrase forms $3x$; − subtracts it.



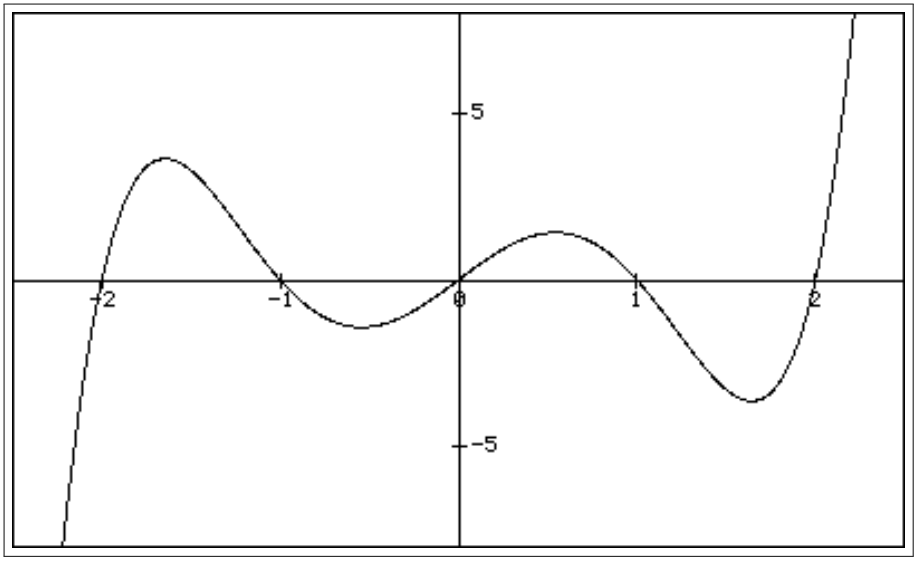
4.5 Fifth-order polynomial: clipping is part of the picture

$$y = x^5 - 5x^3 + 4x = x(x^2 - 1)(x^2 - 4).$$

The factored form predicts five roots: $-2, -1, 0, 1, 2$. The viewing window is $[-2.5, 2.5] \times [-8, 8]$. Rapid growth sends the outer arms off-screen; that is not missing mathematics but a chosen viewport.

```
LBL "YFUN"
SF 00
RCL 05
ENTER↑
X2↑
X2
×
STO 16
RCL 05
ENTER↑
X2
×
5
×
+/-
RCL+ 16
RCL 05
4
×
+
RTN
```

R16 temporarily holds x^5. Each instruction uses a direct arithmetic key except `X↑2`, entered with f x^2; named program control is not involved.



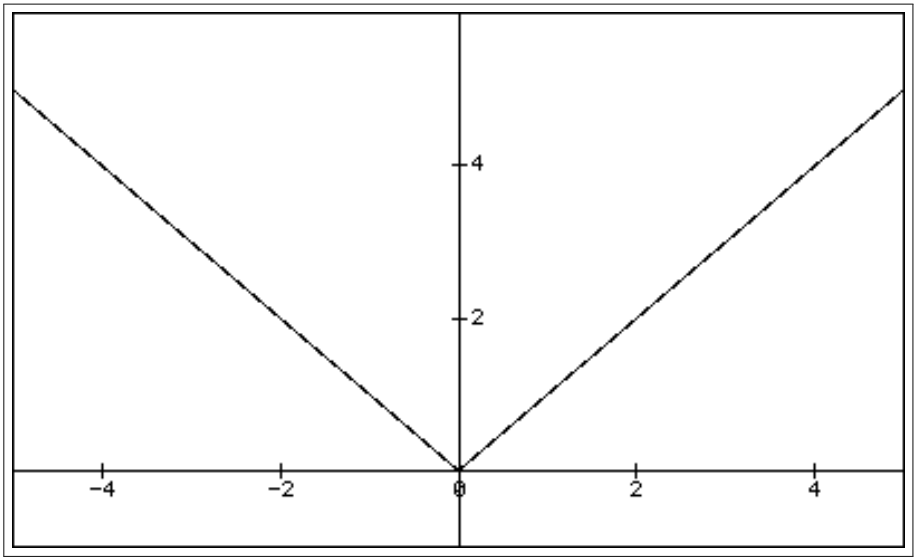
4.6 Absolute value: a genuine corner

$$y = |x|, \quad [-5, 5] \times [-1, 6].$$

The bars mean "distance from zero," so negative and positive X values both produce non-negative Y. The cusp at the origin is not a sampling defect.

```
LBL "YFUN"
SF 00
RCL 05
ABS
RTN
```

Enter `ABS` with f CONVERT ▼ ABS.



4.7 Reciprocal: the line that must not be drawn

$$y = \frac{1}{x}, \quad [-5, 5] \times [-5, 5].$$

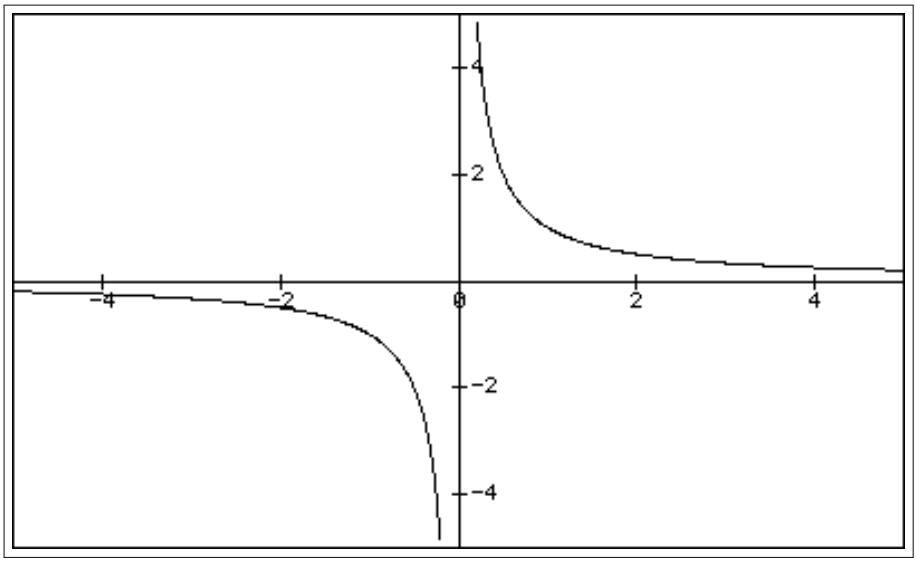
The function is undefined at $x = 0$. On the left, Y is negative and falls without bound as X approaches zero; on the right it is positive and rises without bound. The two branches never meet.

A naïve point-joining plotter samples just left and right of zero, sees two valid pixels, and connects them with a nearly vertical stroke. That stroke falsely claims the graph passes through the asymptote. `CPLOT` breaks the previous-point chain when a point clips outside the Y window and also when adjacent rows jump by more than 80 pixels.

```
LBL "YFUN"
SF 00
RCL 05
X=0?
GTO 90
1/X
RTN
LBL 90
CF 00
0
RTN
```

Exact DM42 keystrokes

`X=0?`: f PGM.FCN ▼ X?0 X=0?. `GTO 90`: f GTO 9 0. `1/X`: the physical $1/x$. `CF 00`: f FLAGS CF 0 0.



Try this

Temporarily remove the 80-pixel jump test from `CPLOT`. If your sampling does not land exactly on zero, note the false bridge. Restore the test.

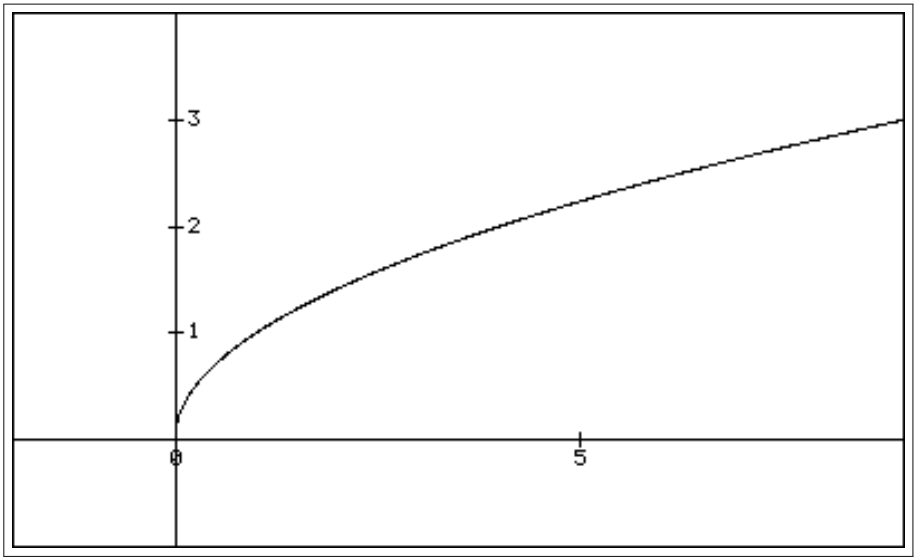
4.8 Square root: a domain is a rule, not an error message

$$y = \sqrt{x}, \quad [-2, 9] \times [-1, 4].$$

Over the real numbers, the formula is valid only for $x \geq 0$. Rather than asking the calculator to evaluate an invalid expression and then recovering from an error, the adapter checks the domain first.

```
LBL "YFUN"
SF 00
RCL 05
X<0?
GTO 90
SQRT
RTN
LBL 90
CF 00
0
RTN
```

Use the physical $\boxed{\sqrt{x}}$ for `SQRT`. The plot begins exactly at the origin and has no invented left-hand branch.



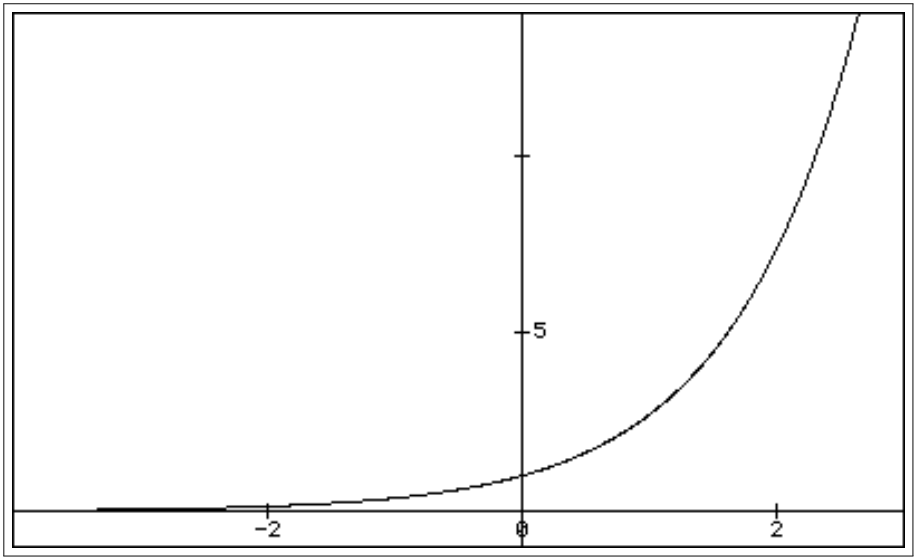
4.9 Exponential: choose a window before it chooses one for you

$$y = e^x, \quad [-4, 3] \times [-1, 14].$$

The curve is always positive, passes through $(0, 1)$, approaches zero to the left and grows rapidly to the right.

```
LBL "YFUN"
SF 00
RCL 05↑
EX
RTN
```

Enter E↑X with f E↑X.



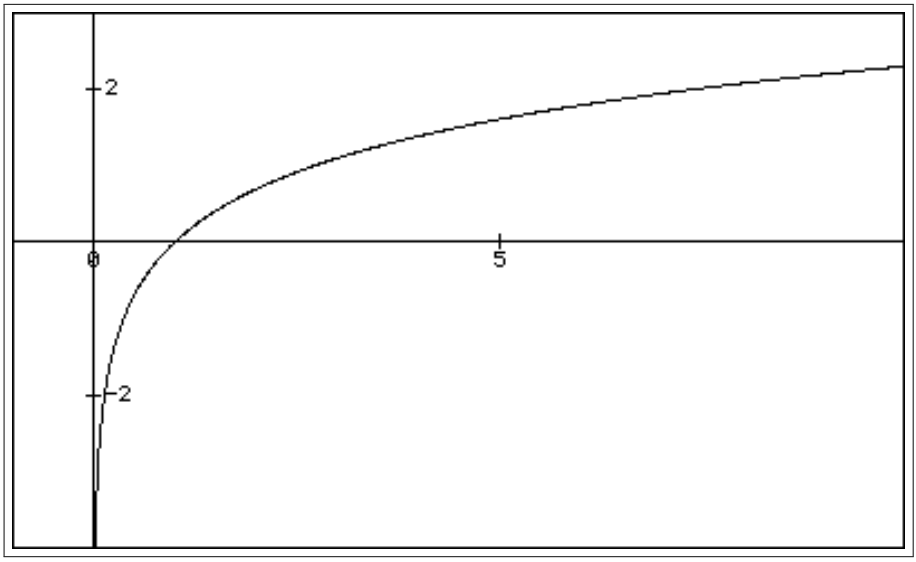
4.10 Natural logarithm: the inverse shape

$$y = \ln x, \quad [-1, 10] \times [-4, 3], \quad x > 0.$$

Read it “the natural logarithm of X.” It crosses $(1, 0)$, grows slowly, and approaches a vertical asymptote at $x = 0^+$.

```
LBL "YFUN"
SF 00
RCL 05
X>0?
GTO 91
CF 00
0
RTN
LBL 91
LN
RTN
```

The conditional rule executes the line immediately following it when true and skips that line when false. Thus a positive X jumps to the logarithm; zero or negative X clears validity.

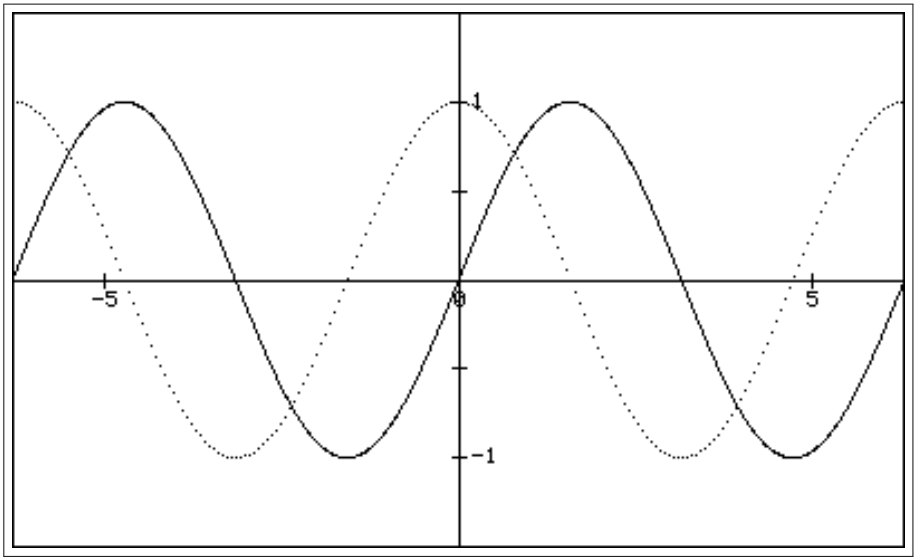


4.11 Sine and cosine: periodic comparison

Use radians and the window $[-2\pi, 2\pi] \times [-1.5, 1.5]$.

$$y = \sin x, \quad y = \cos x.$$

Both have period 2π and amplitude 1. Cosine is sine shifted left by $\pi/2$. The sine adapter is only `RCL 05`, `SIN`; the cosine adapter substitutes `COS`. Plot one as a solid connected curve. Plot the second using every fourth sample or a dotted marker pattern so the monochrome display distinguishes them.



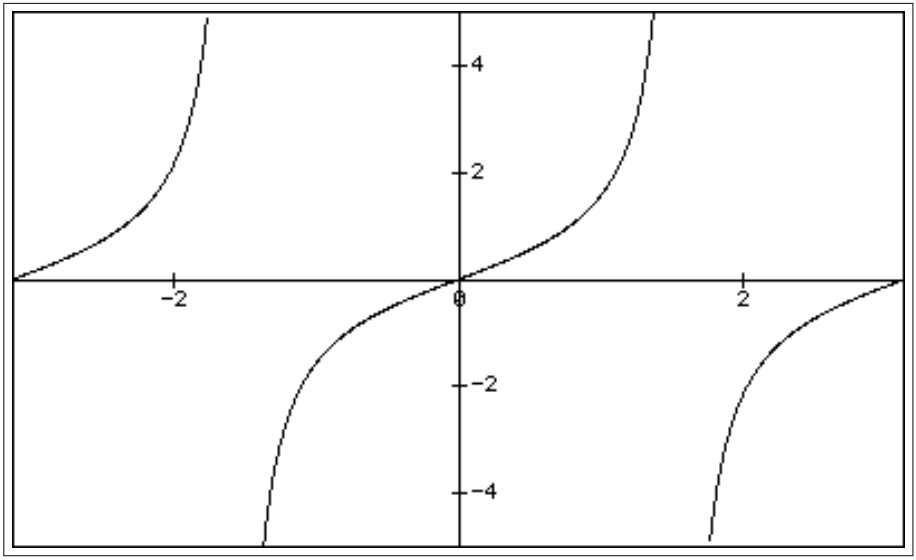
Try this

Change the window to $[-4\pi, 4\pi]$ without changing 400 samples. You have doubled the frequency measured in pixels. Watch the curves become less smooth.

4.12 Tangent: repeated discontinuities

$$y = \tan x, \quad [-\pi, \pi] \times [-5, 5].$$

Vertical asymptotes occur at $x = \pm\pi/2$. The adapter is `RCL 05`, `TAN`; the plotter's clipping and jump break must work repeatedly, not merely at zero.



Plotting trap

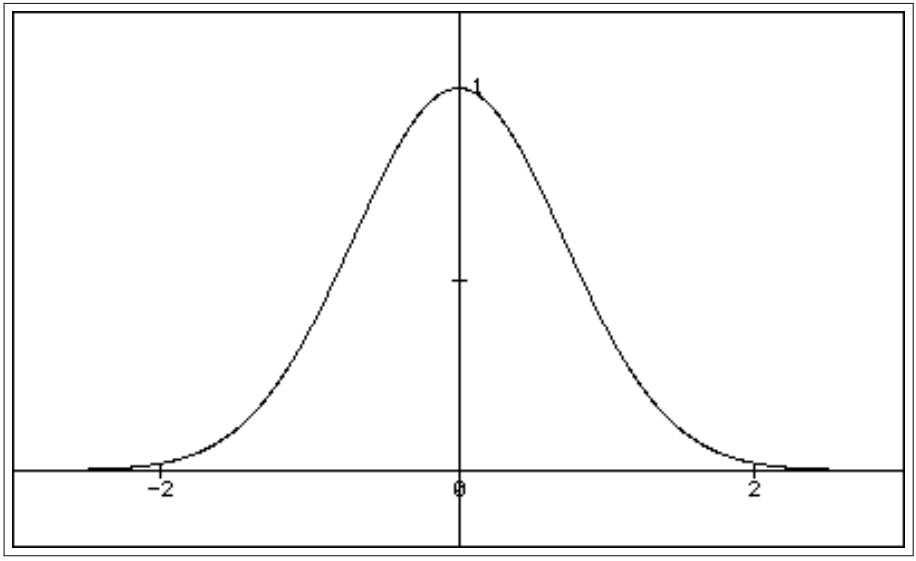
A jump threshold is a heuristic. Zooming or changing the Y range changes a mathematical slope's pixel height. Inspect any plot with asymptotes; do not equate "the program drew it" with "the segment is mathematically valid."

4.13 Gaussian bell: a localised peak

$$y = e^{-x^2}, \quad [-3, 3] \times [-0.2, 1.2].$$

It is even, peaks at 1 when $x = 0$, and has tails close to zero.

```
RCL 05↑
X2
+/-↑
EX
```

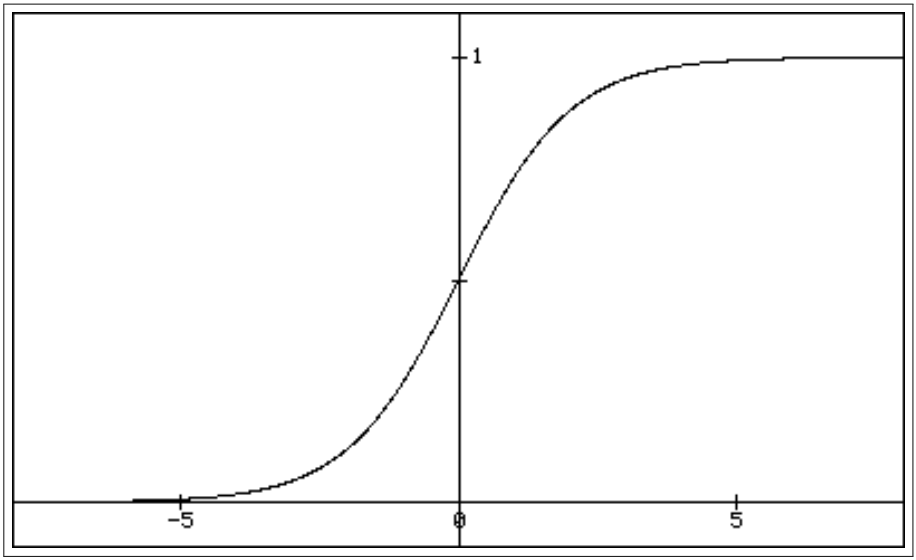


4.14 Logistic sigmoid: two horizontal limits

$$y = \frac{1}{1 + e^{-x}}, \quad [-8, 8] \times [-0.1, 1.1].$$

The curve approaches 0 on the left, 1 on the right and passes through $(0, 1/2)$.

```
RCL 05
+/-↑
EX
1
+
1/X
```

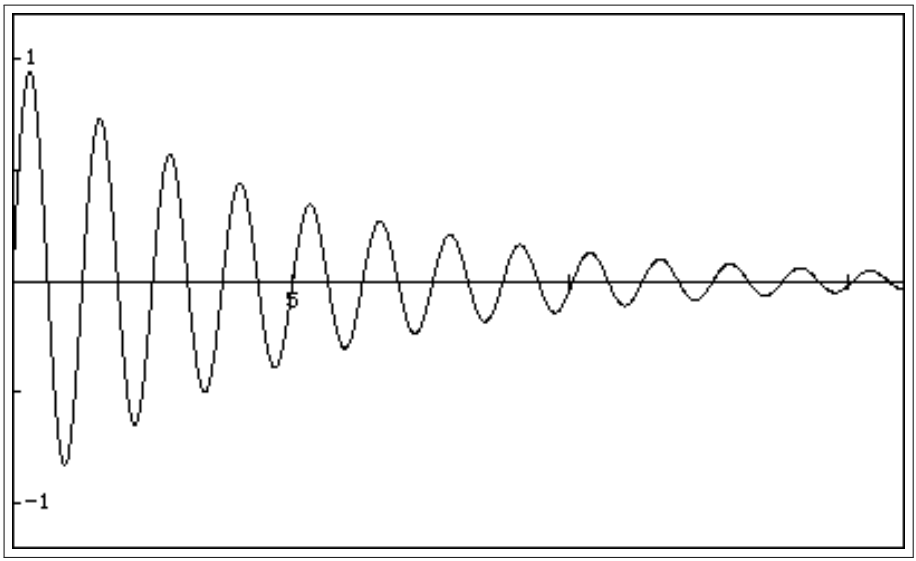


4.15 Damped sine: two processes at once

$$y = e^{-0.2x}\sin(5x), \quad [0, 16] \times [-1.2, 1.2].$$

The sine supplies oscillation; the exponential supplies a shrinking envelope.

```
RCL 05
STO 16
.2
×
+/-↑
EX
RCL 16
5
×
SIN
×
```



If the oscillations look angular, increase samples in a generic plotter or narrow the X window. The Cartesian core intentionally uses one sample per column so sampling remains visible.

4.16 Fourier square-wave approximation

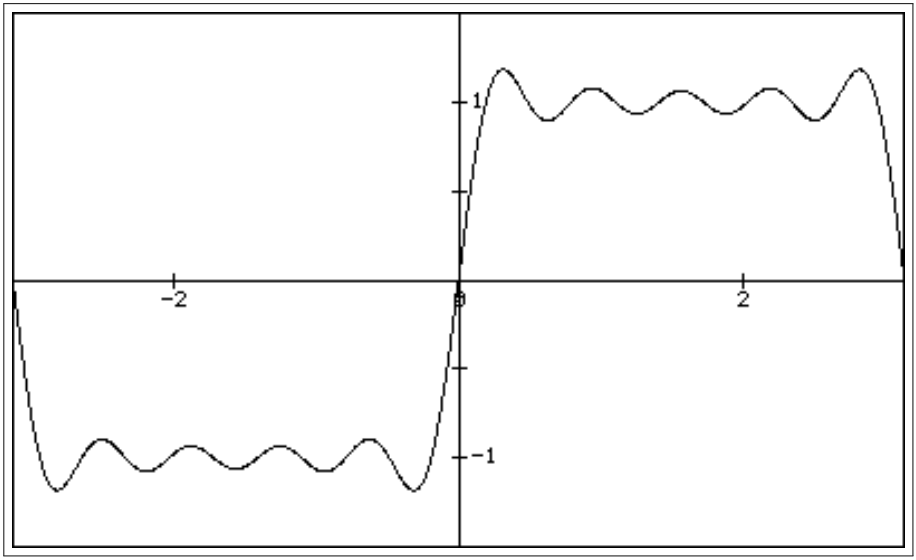
$$S_5(x) = \frac{4}{\pi}\left(\sin x + \frac{\sin 3x}{3} + \frac{\sin 5x}{5} + \frac{\sin 7x}{7} + \frac{\sin 9x}{9}\right).$$

The result is nearly flat away from a jump but overshoots near the jump. Adding harmonics narrows the disturbed region without removing the peak overshoot: the Gibbs phenomenon.

```
RCL 05
SIN
RCL 05
3
×
SIN
3
÷
+
RCL 05
5
×
SIN
5
÷
+
RCL 05
7
×
SIN
7
÷
+
RCL 05
9
×
SIN
9
÷
+
```

```
4
×
PI
÷
```

Enter `PI` with f π. Use $[-\pi, \pi] \times [-1.5, 1.5]$.



Try this

Stop after the 1st, 2nd, 3rd and 5th odd harmonics. The transition sharpens, the flat portions settle, and the overshoot persists.

4.17 Four scientific adapters

These use the same `CPLOT`; only `YFUN`, constants and the window change.

Model	Function	Good first window
Projectile	$y = x\tan\theta - \dfrac{gx^2}{2v_0^2\cos^2\theta}$, with $v_0 = 20$, $\theta = 45°$, $g = 9.81$	$x : 0..42$, $y : -2..23$
RC discharge	$V(t) = 5e^{-t/2}$	$t : 0..10$, $V : -.5..5.5$
Resonance	$A(f) = \left[(1-f^2)^2 + (0.2f)^2\right]^{-1/2}$	$f : 0..2$, $A : 0..6$
Normal density	$y = e^{-x^2/2}/\sqrt{2\pi}$	$x : -4..4$, $y : -.05...45$

For the projectile, put the calculator in radians and store $45\pi/180$ as the angle constant before plotting. For the resonance formula, form the complete positive denominator before `SQRT` and reciprocal. Each window was chosen to include the physically interesting peak or flight path rather than to make automatic scaling do the thinking.

CHAPTER 5

Windows, sampling and performance

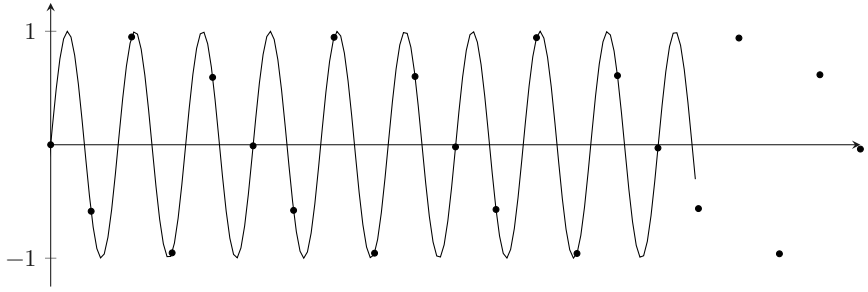
The calculator draws what it samples, not an infinitely detailed mathematical object. The difference becomes visible when the function changes significantly between neighbouring sample points.

5.1 Three questions before every plot

1. **Where are the features?** Roots, turning points, asymptotes and narrow peaks determine the useful window.
2. **How quickly can the function change?** A high-frequency or fractal plot may need more samples than a line.
3. **Does geometry require equal scale?** Circles, vector directions and angles are misleading when world units have different physical pixel sizes.

5.2 Aliasing

Suppose $y = \sin(60x)$ is sampled at only a few dozen evenly spaced X values. Those values may accidentally resemble a slower sine wave, or every sample may land close to zero. The display is then a faithful graph of the samples and a false impression of the continuous function.



The line is the ideal reference curve; the dots are the sampled values. A plotter joining only the dots would invent the missing path.

5.3 Clipping versus discontinuity

Clipping answers “is this point visible?” A discontinuity test asks “is it reasonable to connect this point to the last one?” These questions are independent. Reciprocal and tangent need both. Domain checks answer a third question: “does this real value exist at all?”

Case	Domain check	Clipping	Break test
$\sqrt{x}$	reject $x < 0$	reject outside window	usually not needed
$\ln x$	reject $x \le 0$	reject outside window	useful near asymptote
$1/x$	reject $x = 0$	reject outside window	essential
$\tan x$	undefined at odd multiples of $\pi/2$	reject outside window	essential and repeated

5.4 Aspect ratio

If the world spans $X_R = x_{\max} - x_{\min}$ and $Y_R = y_{\max} - y_{\min}$, the physical scales are $399/X_R$ pixels per X unit and $239/Y_R$ pixels per Y unit. Equal scale requires

$$\frac{399}{X_R} = \frac{239}{Y_R}, \quad Y_R = \frac{239}{399} X_R \approx 0.599 X_R.$$

For $x \in [-5, 5]$, a nearly equal-scale Y range is about $[-3, 3]$.

5.5 Resolution-independent routines

Once the fixed full-resolution formulas are familiar, replace 399 by `WIDTH`-1 and 239 by `HEIGHT`-1. `WIDTH` and `HEIGHT` return the active logical dimensions. Alternatively, `RCL "ResX"` and `RCL "ResY"` retrieve the DM42 virtual variables. The latter names are case-sensitive and read-only.

Do not mix the two conventions: the returned width is 400, whereas the mapping denominator is width minus one. Valid `PIXEL` coordinates remain 1 through width.

5.6 Refresh control when waiting becomes noticeable

For a slow fractal, periodic physical repainting competes with computation. Store 0 in `RefLCD` before drawing. At the end, store (-1) to force the main area to refresh immediately, then store 1 to restore normal main-area refresh.

```
0
STO "RefLCD"
... drawing loop ...
-1
STO "RefLCD"
1
STO "RefLCD"
```

This changes refresh timing, not the graphics buffer or the mathematics. Do not leave refresh disabled after an interrupted experiment; store 1 in `RefLCD` manually if necessary.

5.7 A practical optimisation order

1. Choose a smaller preview mode or fewer samples.
2. Remove repeated calculations from the inner loop.
3. Clip before performing expensive work that cannot affect the screen.
4. Use symmetry where the mathematics guarantees it.
5. Suspend periodic refresh for long plots.
6. Render progressively: coarse preview, then full result.

Optimisation begins after the plot is correct. A fast false asymptote is still false.

Plotting trap

If a graph is blank, do not immediately rewrite the program. Check the graphics mode, window, angle mode, domain, validity flag and whether the plot lies entirely outside the chosen range. These six checks solve most blank-screen cases.

CHAPTER 6

Parametric and polar curves

PPLOT now calls the same AXES routine used by CPLOT. R10, R11 and R17 therefore retain the meanings introduced in chapter 4. Set both spacing registers to zero for an unannotated curve, which is often the cleanest choice for dense rosettes and spirals.

In a Cartesian plot, the screen column chooses X and the function supplies one Y. A parametric curve replaces that assumption with

$$x = x(t), \quad y = y(t).$$

The program chooses t, evaluates both functions, maps both coordinates and connects the result to the previous point. Loops, sideways turns and self-intersections now become natural.

6.1 The parametric contract

The PPLOT toolkit uses these registers:

R20	$t_{\min}$	R21	$t_{\max}$
R22	number of parameter steps	R24	current t
R25,R26	world X,Y	R27,R28	screen X,Y
R29,R30	previous screen X,Y	R31	previous-point valid flag

XFUN and YFUNP read t from R24. SEG uses a digital differential analyser: it takes the larger of $|\Delta x_s|$ and $|\Delta y_s|$ as a step count, then advances both coordinates by fractional increments. This is a genuine general segment routine, unlike Cartesian VJOIN.

The complete listing and key audit are in appendix B.3. Run GSET, store the window, parameter range and step count, then run PPLOT.

6.2 Circle: one angle chooses two coordinates

$$x = \cos t, \quad y = \sin t, \quad 0 \le t \le 2\pi.$$

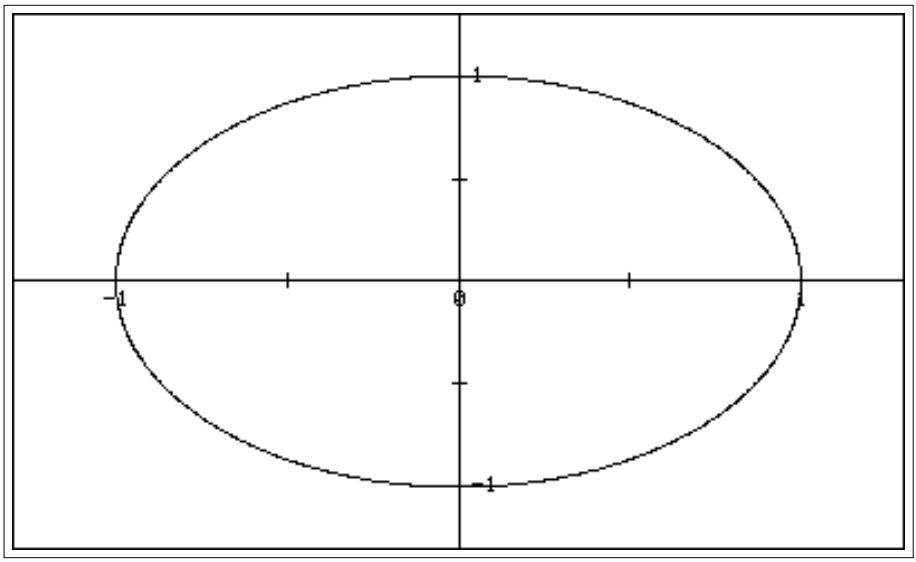
Cosine is the horizontal coordinate of a point on the unit circle; sine is the vertical coordinate. The curve should close after one revolution. Use the equal-scale window $[-1.7, 1.7] \times [-1.02, 1.02]$, or accept the deliberately wider-looking circle in $[-1.3, 1.3]^2$.

```
LBL "XFUN"
RCL 24
COS
RTN
```

```
LBL "YFUNP"
RCL 24
SIN
RTN
```

Exact entries for the new lines are [RCL][2][4][COS] and [RCL][2][4][SIN]. Set the parameter registers:

[0][STO][2][0] [2][f][π][×][STO][2][1] [8][0][0][STO][2][2]

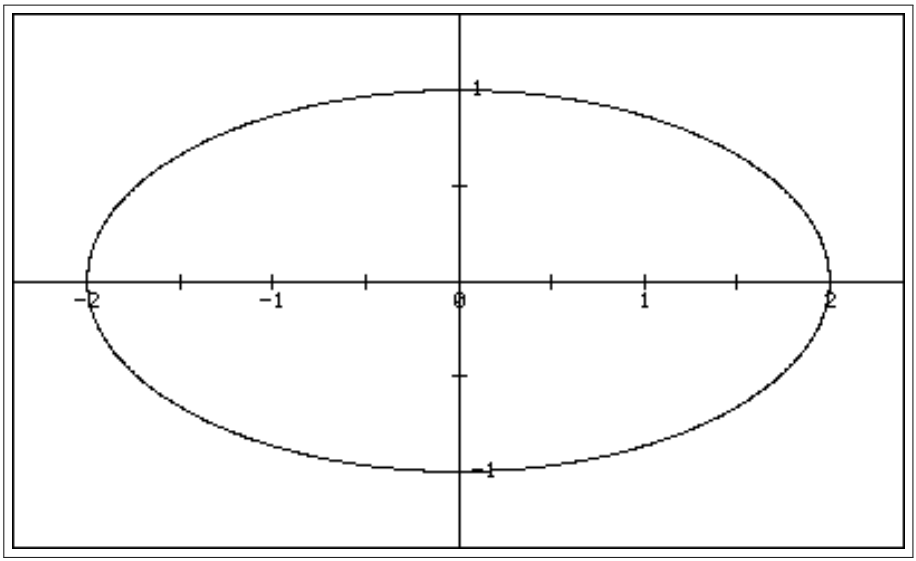


6.3 Ellipse and rotated ellipse

For an axis-aligned ellipse,

$$x = a\cos t, \quad y = b\sin t.$$

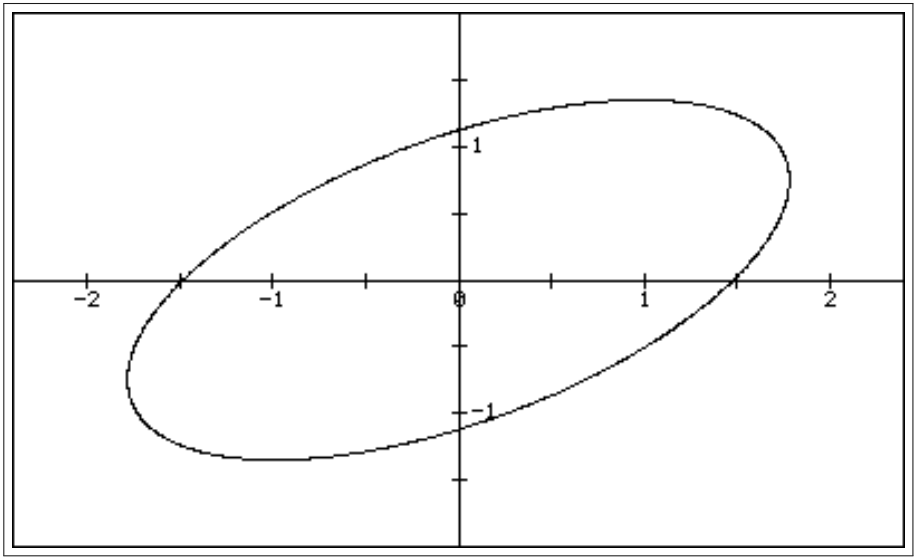
With $a = 2, b = 1$, append [2][×] to the X adapter and leave the Y adapter unchanged. Use $[-2.4, 2.4] \times [-1.44, 1.44]$ for equal scale.



Rotation by ϕ mixes both original coordinates:

$$x = a\cos t\cos\phi - b\sin t\sin\phi,$$
$$y = a\cos t\sin\phi + b\sin t\cos\phi.$$

Store a, b, ϕ in R50–R52. Each adapter recalls R24 twice and the constants it needs. With $a = 2, b = 1, \phi = 0.55$, use $[-2.4, 2.4] \times [-1.8, 1.8]$.

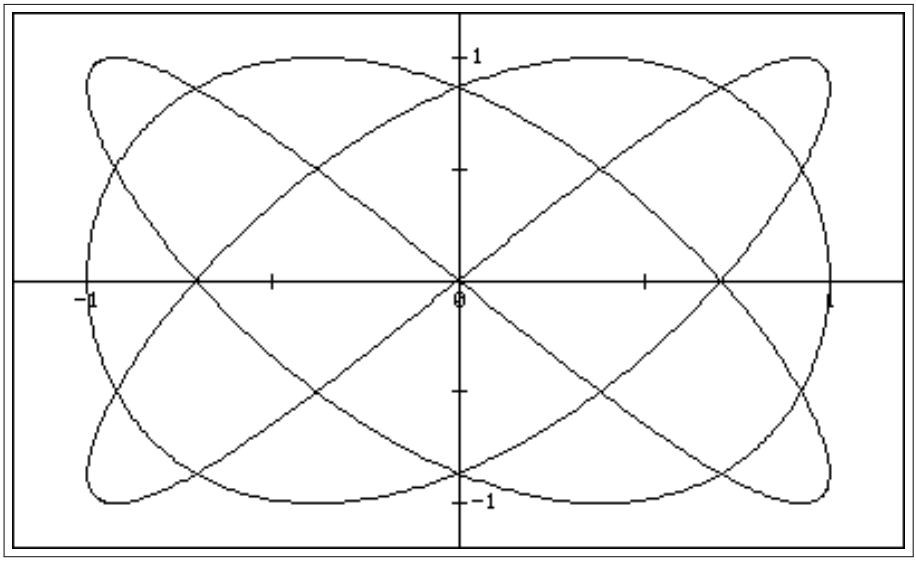


This is an ordinary ellipse followed by a coordinate rotation. Rotation changes orientation, not the curve's semi-axis lengths.

6.4 Lissajous figures

$$x = \sin(at), \quad y = \sin(bt + p).$$

Use $a = 3, b = 4, p = \pi/3$, $t \in [0, 2\pi]$, and $[-1.2, 1.2]^2$. Frequency ratio controls the lobe count; phase shifts how the lobes meet.



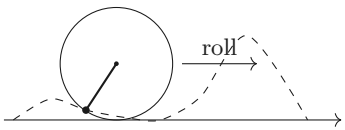
Try this

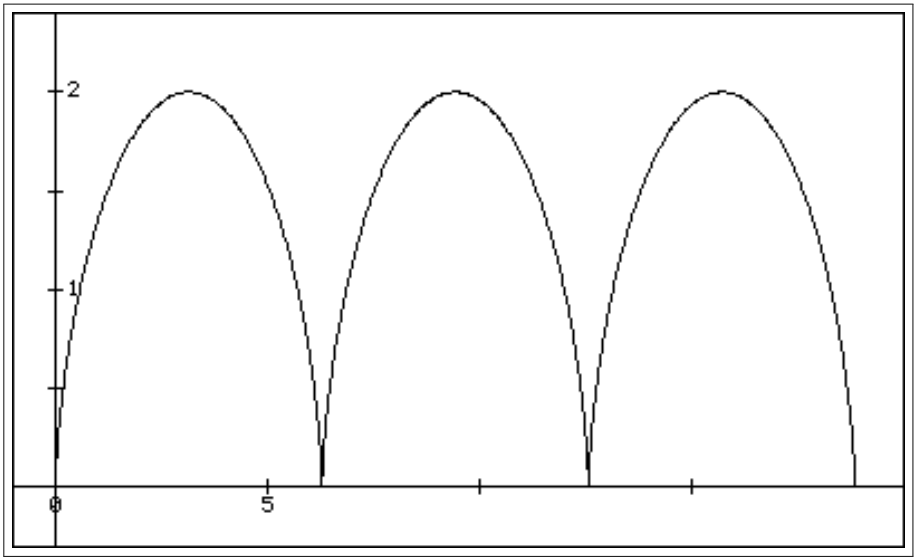
Try ratios $1 : 1$, $2 : 3$, $3 : 4$, then vary p from 0 to $\pi/2$. Predict whether the origin will be crossed before plotting.

6.5 Cycloid: a rolling circle leaves a trace

$$x = t - \sin t, \quad y = 1 - \cos t, \quad 0 \le t \le 6\pi.$$

As a unit circle rolls, t is both its rotation angle and the distance travelled by its centre. Subtracting $\sin t$ and $\cos t$ accounts for the marked point's offset from that centre.



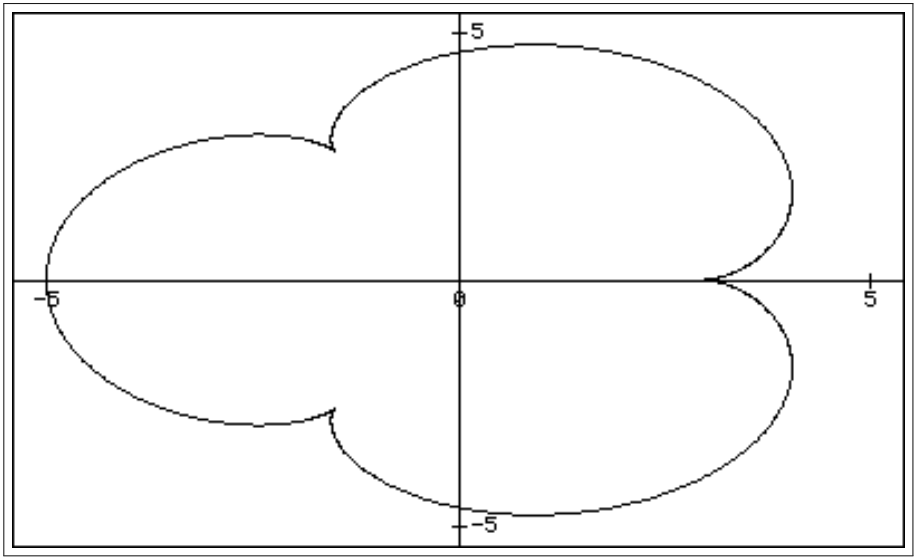


6.6 Epicycloid and hypotrochoid

An epicycloid is traced by a point on a circle of radius r rolling outside a circle of radius R:

$$x = (R+r)\cos t - r\cos\frac{R+r}{r}t, \quad y = (R+r)\sin t - r\sin\frac{R+r}{r}t.$$

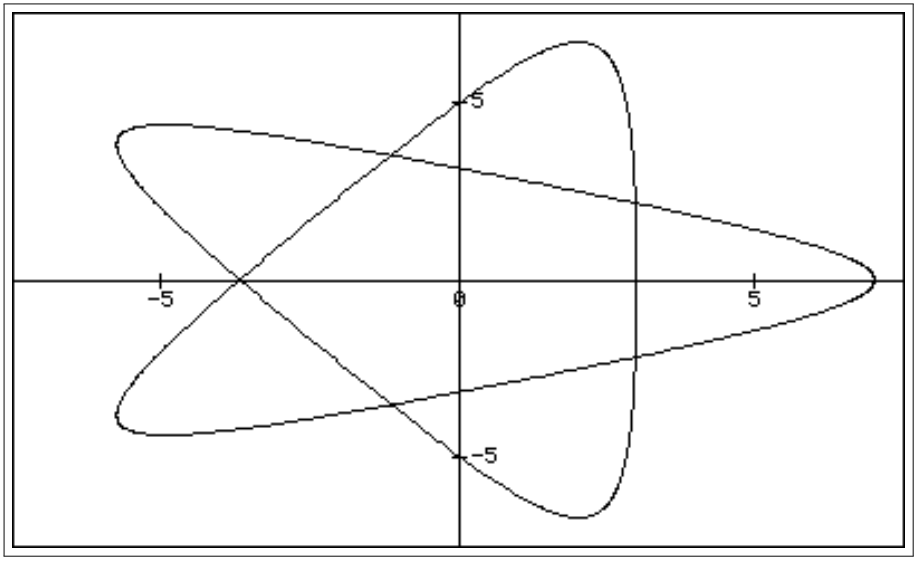
Use $R = 3, r = 1$, $t \in [0, 2\pi]$, window $[-5.4, 5.4]^2$.



For a hypotrochoid, a point at distance d from the centre of the rolling circle traces

$$x = (R-r)\cos t + d\cos\frac{R-r}{r}t, \quad y = (R-r)\sin t - d\sin\frac{R-r}{r}t.$$

The displayed example uses $R = 5, r = 3, d = 5$, $0 \le t \le 6\pi$, window $[-7.5, 7.5]^2$.

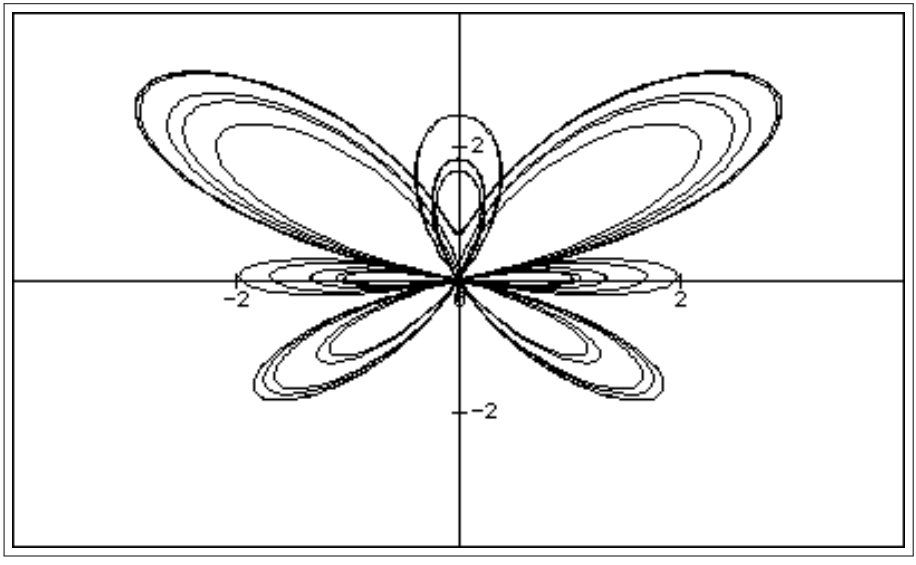


6.7 The butterfly showcase

Let

$$q(t) = e^{\cos t} - 2\cos 4t - \sin^5(t/12), \quad x = \sin t\, q(t), \quad y = \cos t\, q(t).$$

Use $0 \le t \le 12\pi$, 2400 steps, and $[-4, 4]^2$. The seemingly forbidding equation is only a radial modulation $q(t)$ multiplied by sine and cosine. The plotting machinery does not become more mysterious because the radius does.



6.8 Polar plotting is parametric plotting

A polar equation $r = f(\theta)$ becomes

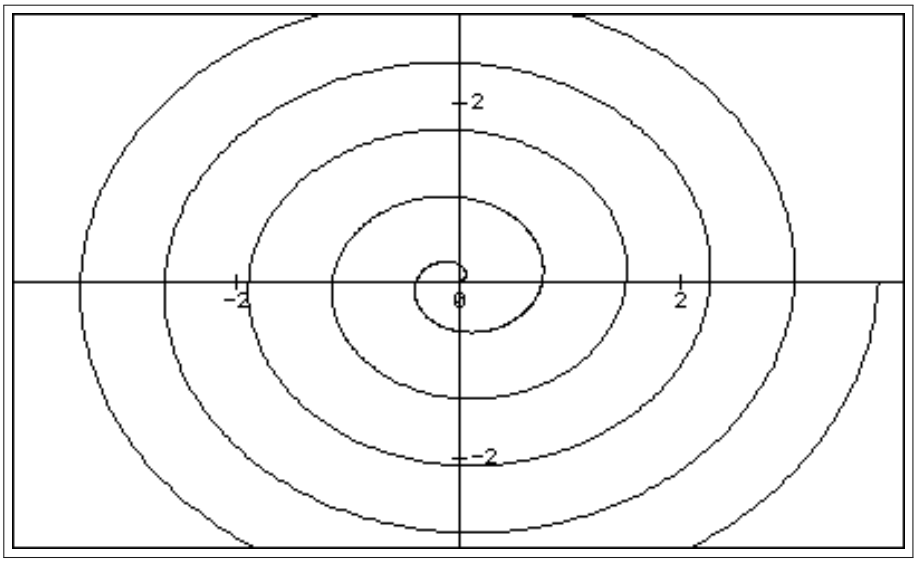
$$x = r\cos\theta, \quad y = r\sin\theta.$$

Thus `PPLOT` remains unchanged. A replaceable `RFUN` returns r from angle R24; `XFUN` multiplies that radius by $\cos\theta$, and `YFUNP` by $\sin\theta$. The radius routine is evaluated twice per sample in the simple design. A faster version stores it once.

6.8.1 Archimedean spiral

$$r = 0.12\theta, \quad 0 \le \theta \le 10\pi, \quad [-4, 4] \times [-3, 3].$$

Equal increases in angle add equal radial distance, so successive turns are evenly spaced.



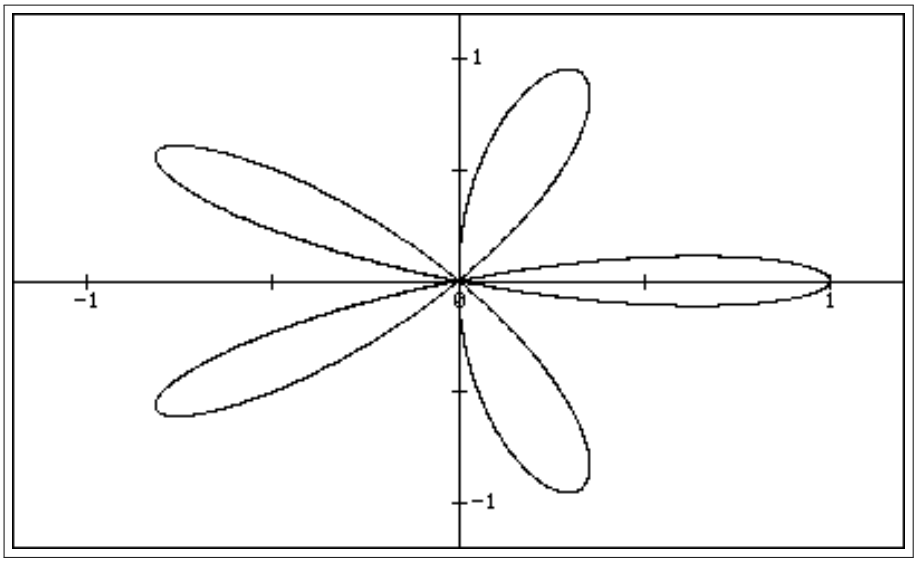
6.8.2 Polar rose

$$r = \cos(5\theta), \quad 0 \le \theta \le 2\pi.$$

Negative radius reverses direction by π; it is not an invalid value. Odd n gives n petals over a full revolution, whereas even n gives $2n$.

```
LBL "RFUN"
RCL 24
5
×
COS
RTN
```

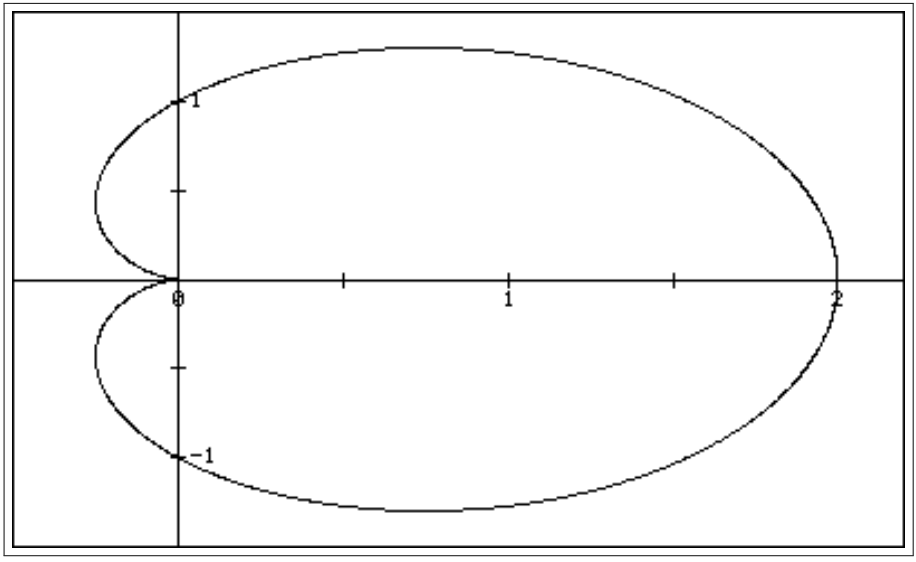
New-line keys: RCL 2 4 5 × COS.



6.8.3 Cardioid

$$r = 1 + \cos\theta, \quad 0 \le \theta \le 2\pi.$$

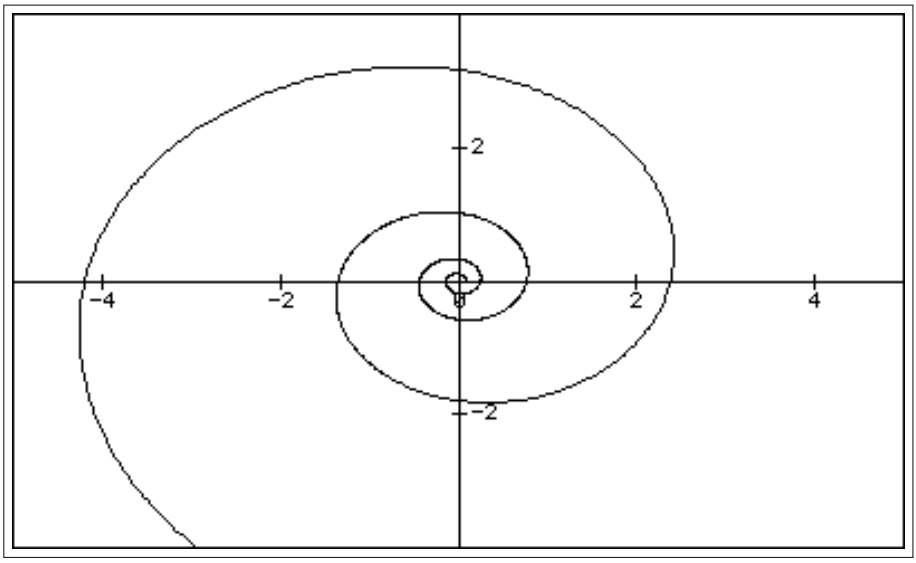
The radius is zero at $\theta = \pi$, producing the cusp. Use $[-0.5, 2.2] \times [-1.5, 1.5]$.



6.8.4 Logarithmic spiral

$$r = 0.08e^{0.18\theta}, \quad 0 \le \theta \le 8\pi.$$

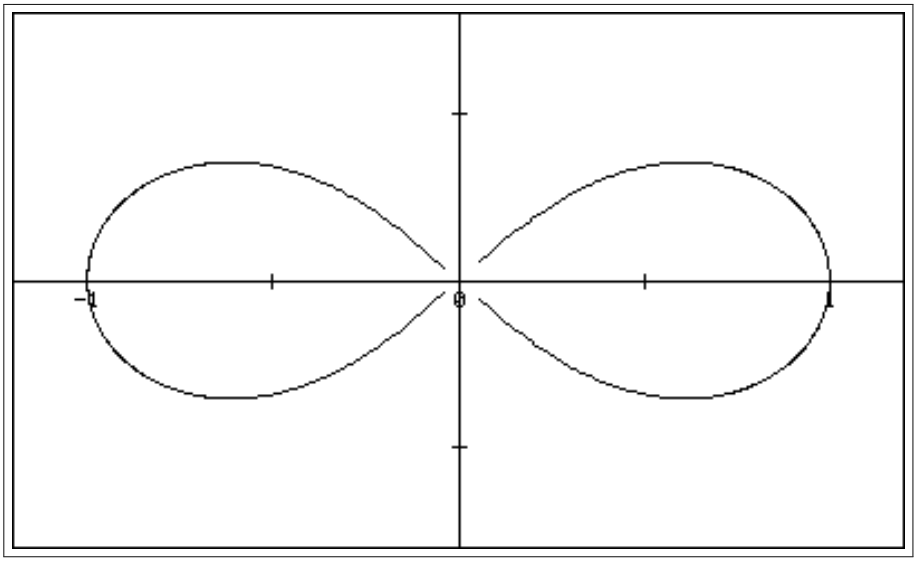
Each equal angular turn multiplies the radius by the same factor. Use $[-5, 5] \times [-4, 4]$.



6.8.5 Lemniscate

$$r^2 = \cos 2\theta.$$

The real radius exists only when $\cos 2\theta \geq 0$. `RFUN` must clear its validity flag in the excluded intervals instead of taking a real square root of a negative number.



6.9 A branch curve without a contour algorithm

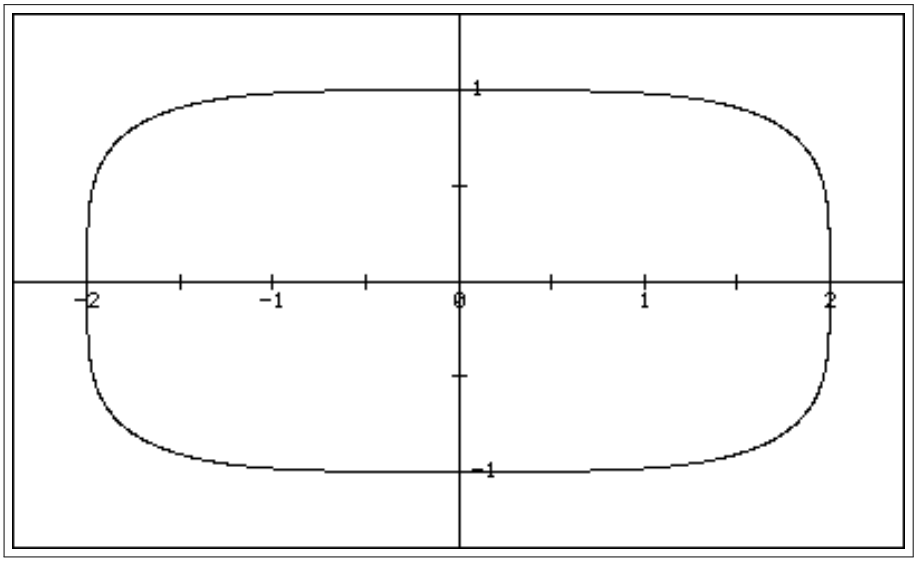
The superellipse

$$\left|\frac{x}{a}\right|^n + \left|\frac{y}{b}\right|^n = 1$$

has the parameterisation

$$x = a\ \mathrm{sgn}(\cos t)|\cos t|^{2/n}, \quad y = b\ \mathrm{sgn}(\sin t)|\sin t|^{2/n}.$$

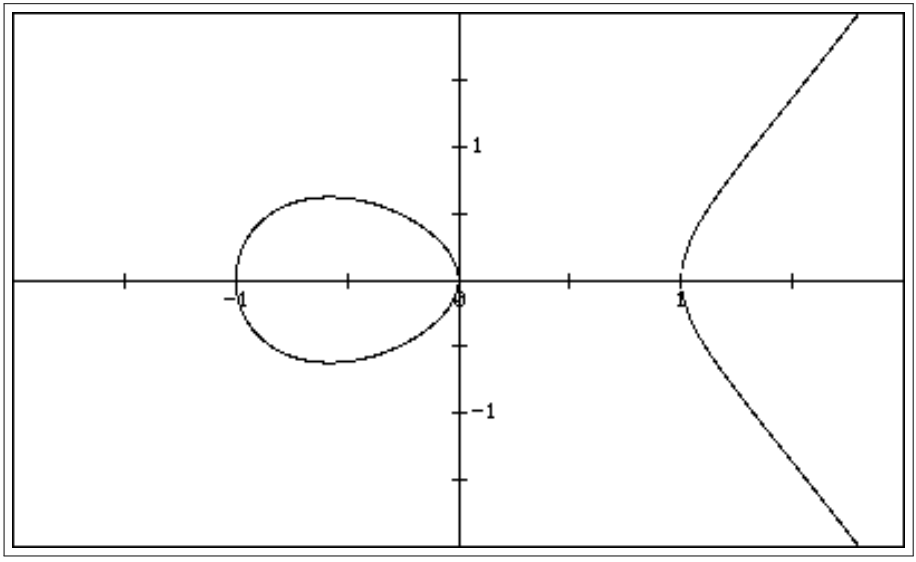
With $a = 2, b = 1, n = 4$, it looks like a rectangle whose corners have been rounded continuously.



The elliptic algebraic curve $y^2 = x^3 - x$ can also be plotted without general contouring by drawing both real branches

$$y = \pm\sqrt{x^3 - x}$$

where $x^3 - x \geq 0$. Domain checking and a second sign are enough.



6.10 Parametric quick table

Plot	Adapter mathematics	Suggested steps
Circle	$\cos t, \sin t$	800
Ellipse	$a\cos t, b\sin t$	800
Rotated ellipse	rotation formula above	900
Lissajous	$\sin at, \sin(bt + p)$	1400
Cycloid	$t - \sin t, 1 - \cos t$	1200
Epicycloid	rolling-circle formula	1600
Hypotrochoid	inside rolling-circle formula	2200
Butterfly	$\sin t\, q(t), \cos t\, q(t)$	2400

Higher step counts cost time but reduce holes and jagged diagonals. Once adjacent points differ by no more than one pixel, further increases usually duplicate pixels rather than add visible detail.

Part III

Beyond a single curve

CHAPTER 7

Implicit curves, contours and fields

The contour driver calls AXES before sampling. Its ticks use the same R10, R11 and R17 settings as the Cartesian and parametric plotters. For a dense contour map, marks without numbers (R17 equal to zero) often leave more room for the field itself.

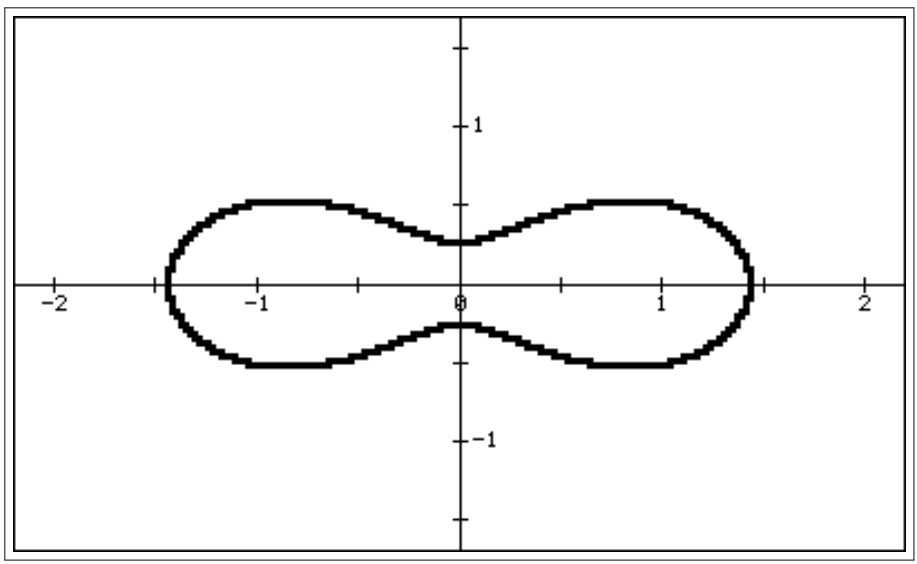
What if an equation does not supply exactly one Y for every X? A circle written $x^2 + y^2 = 1$, a contour $F(x, y) = C$, and a vector field all require the program to inspect a two-dimensional world.

7.1 Cassini ovals: geometry before algorithm

For foci at $(-a, 0)$ and $(a, 0)$, a Cassini oval contains points whose distance product is constant:

$$((x - a)^2 + y^2)((x + a)^2 + y^2) = b^4.$$

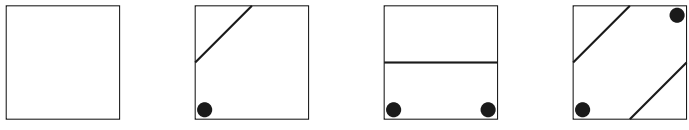
Changing b/a moves the shape from two loops through a lemniscate to one oval. The shown case uses $a = 1$, $b^4 = 1.15$, window $[-2.2, 2.2] \times [-1.7, 1.7]$.



7.2 From a threshold to Marching Squares

The simplest implicit plot evaluates $F(x, y) - C$ at each sampled location and lights a pixel when its absolute value is smaller than a tolerance. It is easy but produces thickness that changes with the local gradient.

Marching Squares instead samples the four corners of a small cell. Each corner is classified below or above the contour level. The four bits form one of sixteen cases; the case tells us which cell edges the contour crosses.



0000 0001 0011 0101

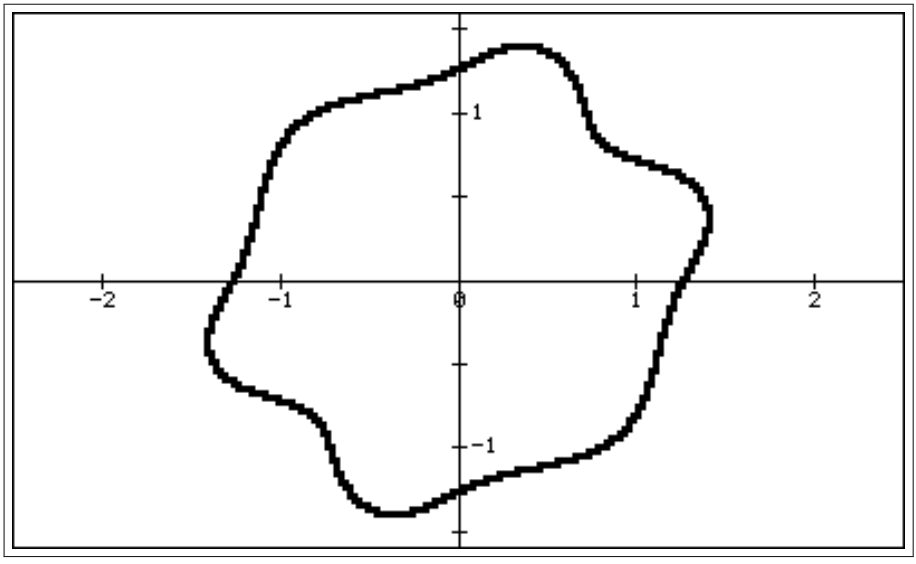
Cases 5 and 10 are ambiguous: two short segments are possible. Testing the cell centre gives a practical consistent choice.

The compact `ICONT` program printed in the appendix uses the threshold method first because every line is transparent. Replace its cell-drawing block with the sixteen-case table when edge-accurate contours matter. Use a coarse 200×120 preview before a full-resolution pass.

7.3 A smooth implicit test

$$F(x, y) = x^2 + y^2 + 0.65 \sin 3x \sin 3y, \quad F = 1.6.$$

The circle term provides a closed baseline; the sine product ripples its boundary. Window: $[-2.5, 2.5] \times [-1.6, 1.6]$.

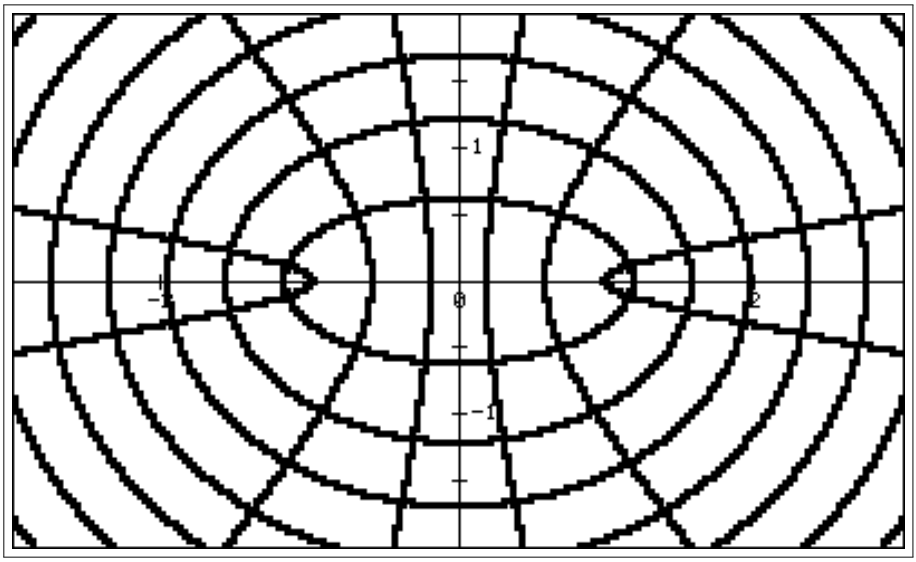


7.4 Two-wave interference

Place two equal point sources at $(-1, 0)$ and $(1, 0)$. With distances r_1, r_2, a simple instantaneous field is

$$F(x, y) = \sin(8r_1) + \sin(8r_2).$$

Plotting the zero contour shows alternating families of cancellation boundaries. This is not an intensity image; it is one selected field level.

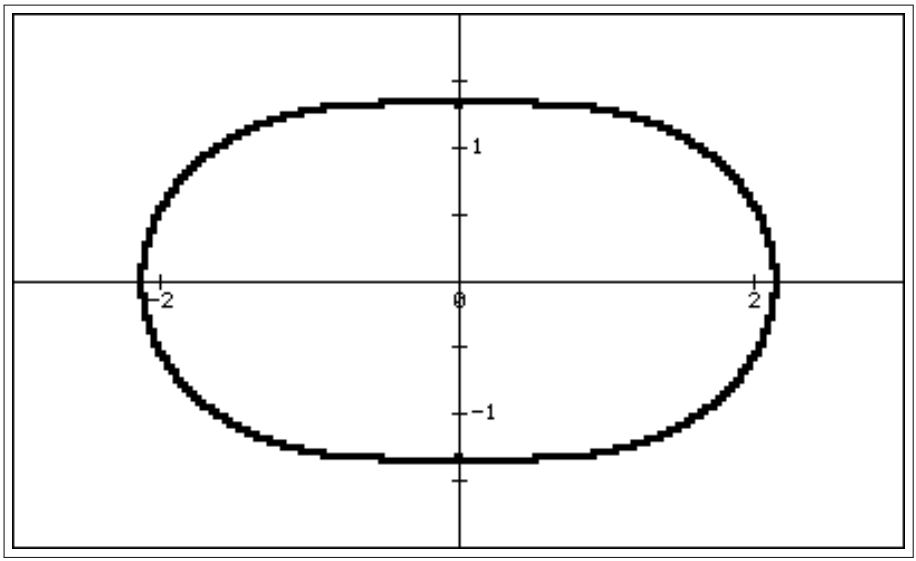


7.5 Equipotential curves

For two equal softened sources, use

$$V(x, y) = \frac{1}{\sqrt{(x-1)^2 + y^2 + \varepsilon^2}} + \frac{1}{\sqrt{(x+1)^2 + y^2 + \varepsilon^2}}.$$

Contours $V = C$ join points of equal scalar potential. The small ε keeps the sampled model finite at the source positions; it represents a modelling choice, not a mathematical identity.



7.6 An arrow routine turns numbers into a field

A vector field assigns two components at each point: $\mathbf{F}(x, y) = (u, v)$. At a coarse grid point:

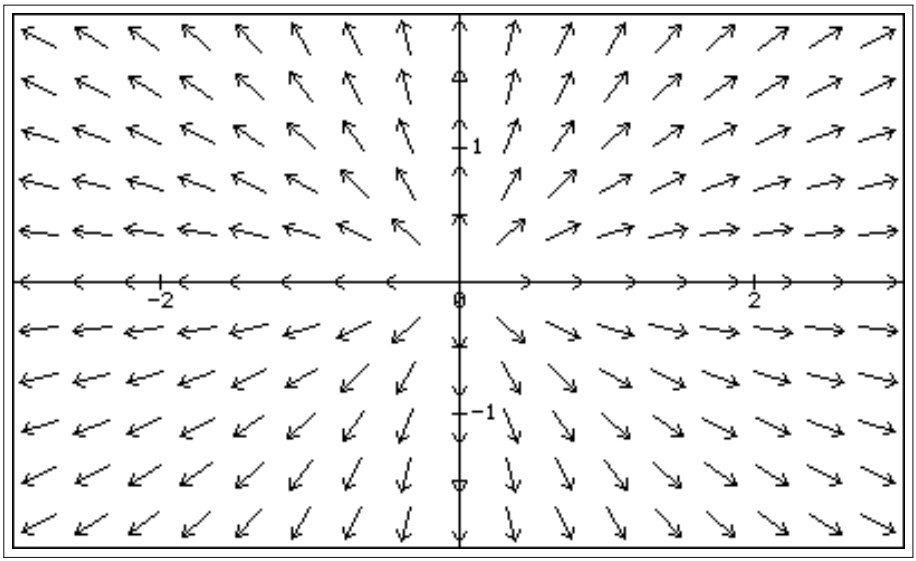
1. evaluate (u, v);
2. calculate $m = \sqrt{u^2 + v^2}$;
3. skip the zero vector;
4. normalise to $(u/m, v/m)$;
5. form a short tail and head;
6. map both endpoints and call `SEG`;
7. add two short head segments.

Normalising keeps arrow lengths readable. If magnitude matters, vary length over a small bounded range instead of allowing one large vector to dominate the screen.

7.6.1 Radial field

$$\mathbf{F}(x, y) = (x, y).$$

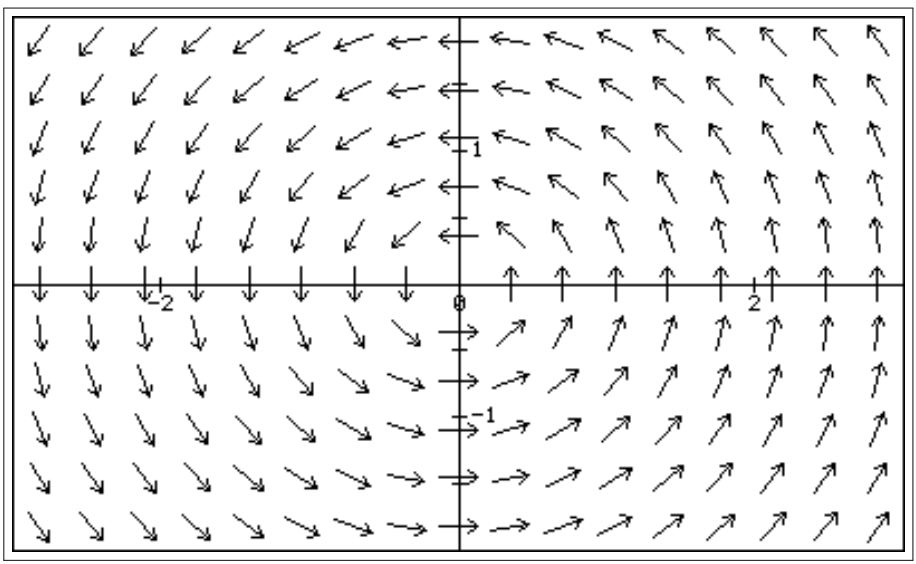
Every arrow points directly away from the origin. At the origin the vector is zero and has no direction.



7.6.2 Rotational field

$$\mathbf{F}(x, y) = (-y, x).$$

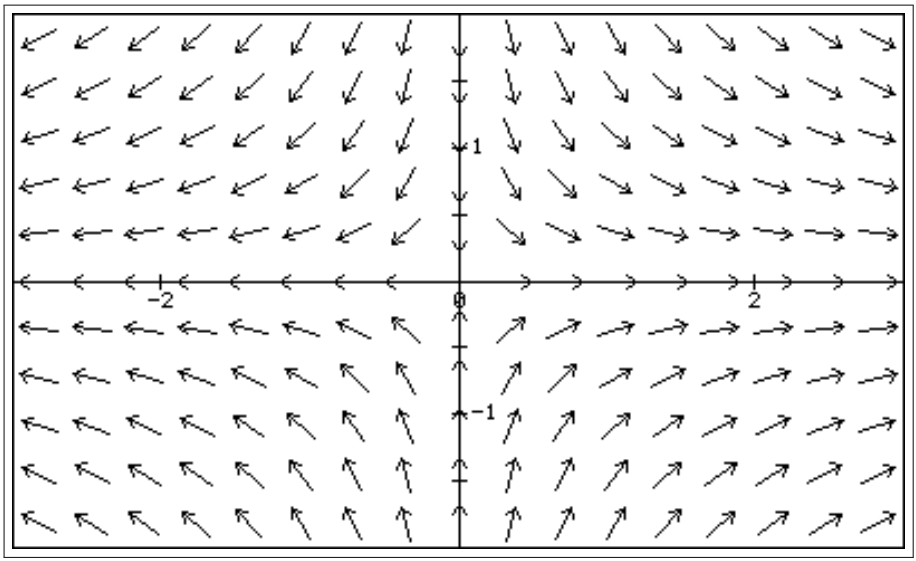
The vector is perpendicular to the radius; arrows circulate counter-clockwise.



7.6.3 Saddle field

$$\mathbf{F}(x, y) = (x, -y).$$

The field expands horizontally and contracts vertically. The axes are invariant directions.

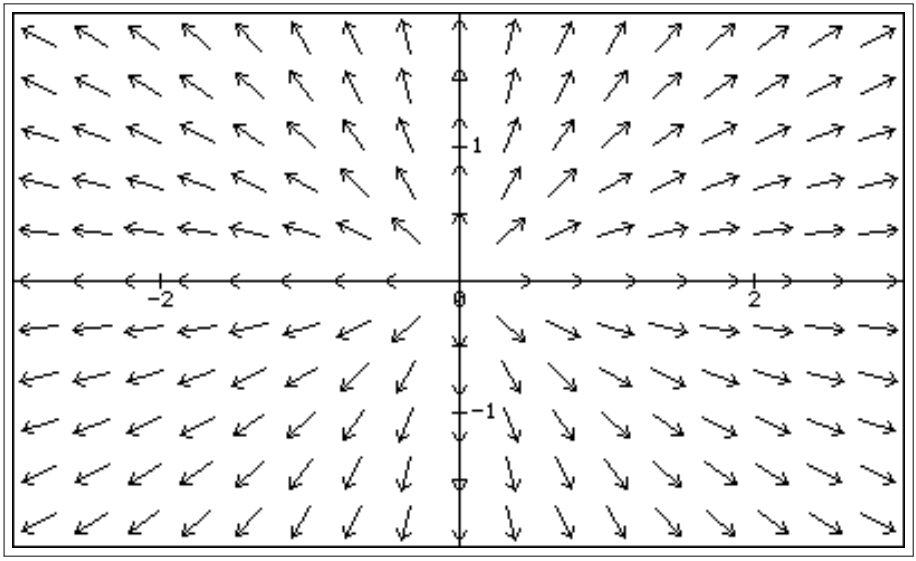


7.6.4 Gradient field

For $f(x, y) = x^2 + y^2$,

$$\nabla f = (2x, 2y).$$

The gradient points in the direction of fastest increase of the scalar field and is perpendicular to its circular level curves.

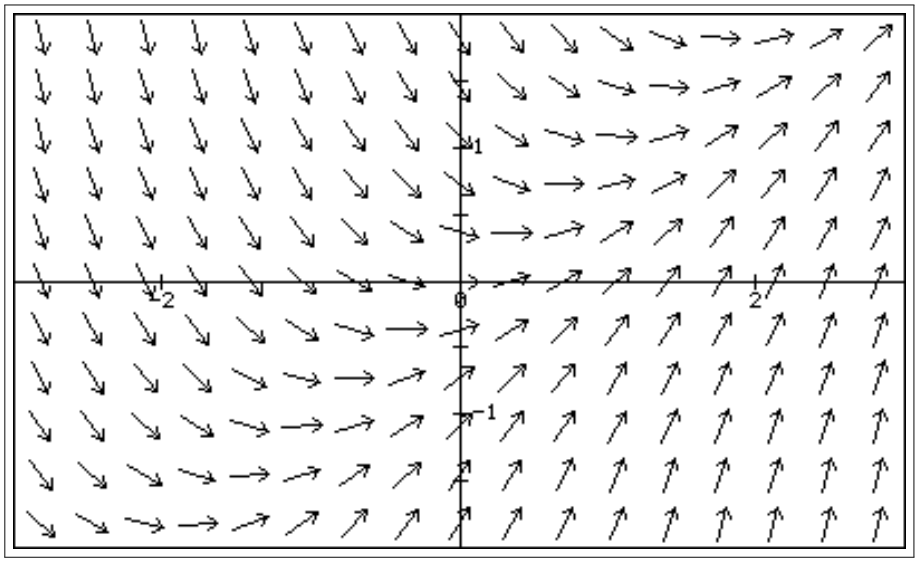


7.7 Slope fields

For the differential equation

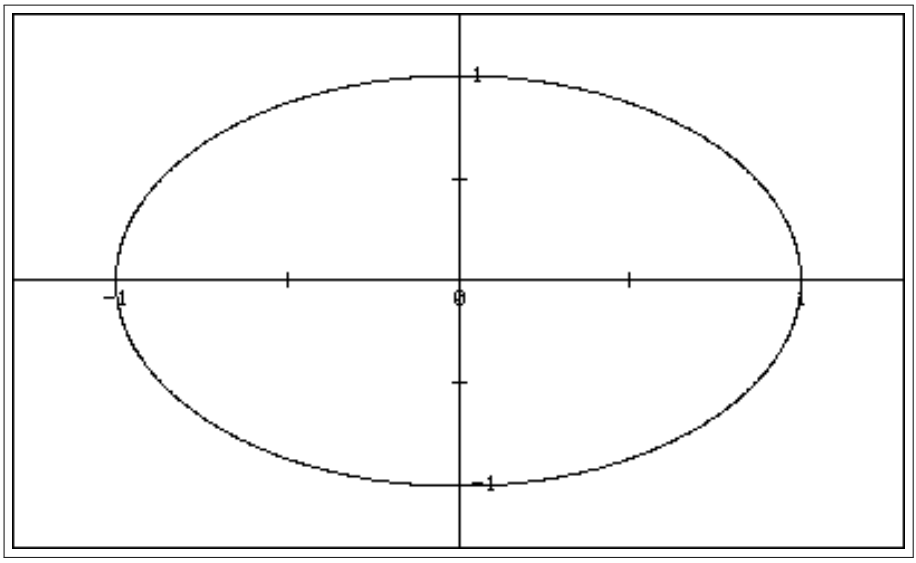
$$\frac{dy}{dx} = x - y,$$

the number $m = x - y$ is the local slope. Draw a short segment in direction $(1, m)$ at every coarse grid point. A solution curve is one that remains tangent to these tiny segments as it moves across the field.



7.8 Phase space

For a unit harmonic oscillator, $x(t) = \cos t$ and velocity $v(t) = -\sin t$. Plotting position against velocity produces a closed orbit.



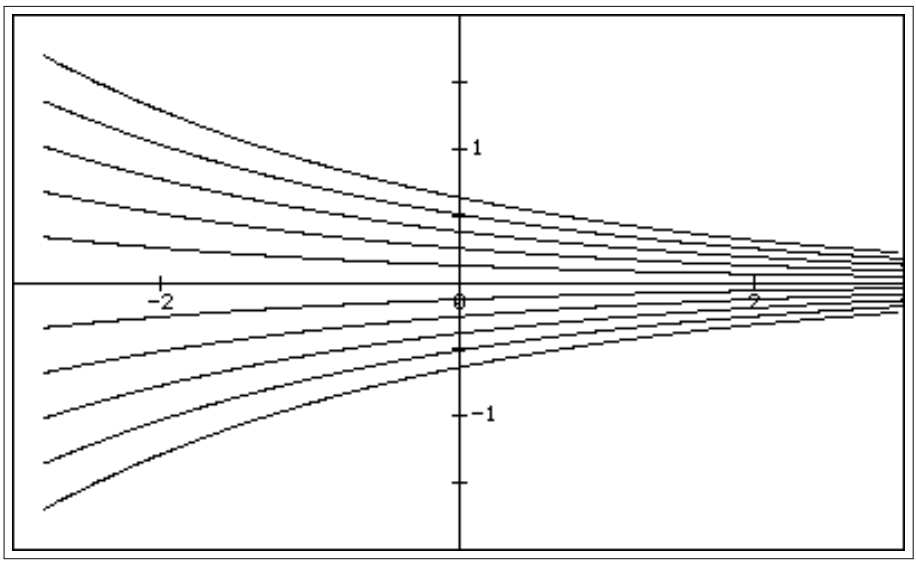
The horizontal and vertical axes are state variables, not two spatial directions. One point represents the entire state of the oscillator at an instant.

7.9 Streamlines

A streamline obeys

$$\frac{d\mathbf{r}}{ds} = \frac{\mathbf{F}(\mathbf{r})}{|\mathbf{F}(\mathbf{r})|}.$$

Starting from several seed points, take small numerical steps in the local field direction and join them with `SEG`. Stop at the screen boundary, near a zero vector or after a fixed step count.



Try this

In the rotational field, seed points at several radii. Euler stepping produces almost-circles that slowly drift. Reduce the step size and observe the drift shrink; this is numerical integration error made visible.

CHAPTER 8

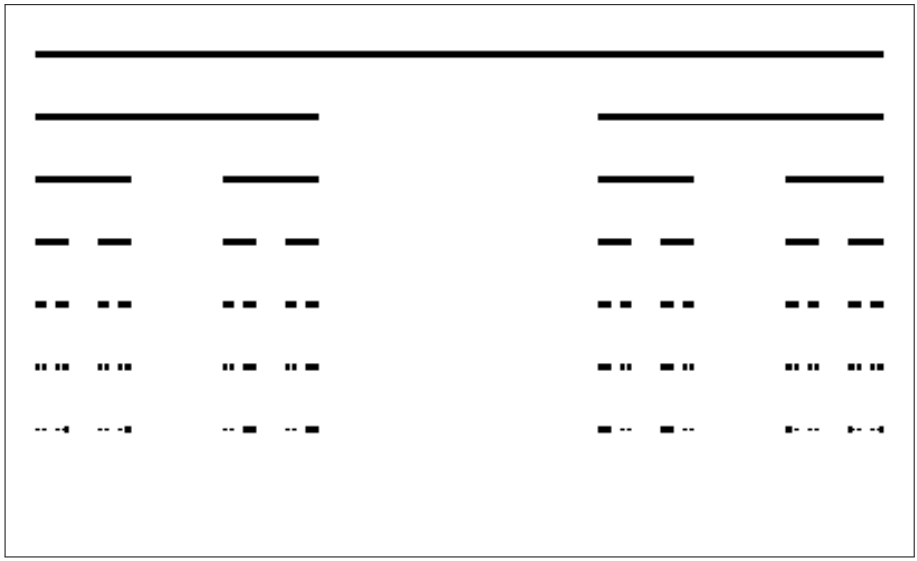
Recursion, iteration and dynamical systems

Some plots do not evaluate a function on a regular X grid. They repeat a geometric rule or update a state. The picture is the history of that repetition.

8.1 Geometric fractals: replace, repeat, stop

8.1.1 Cantor set

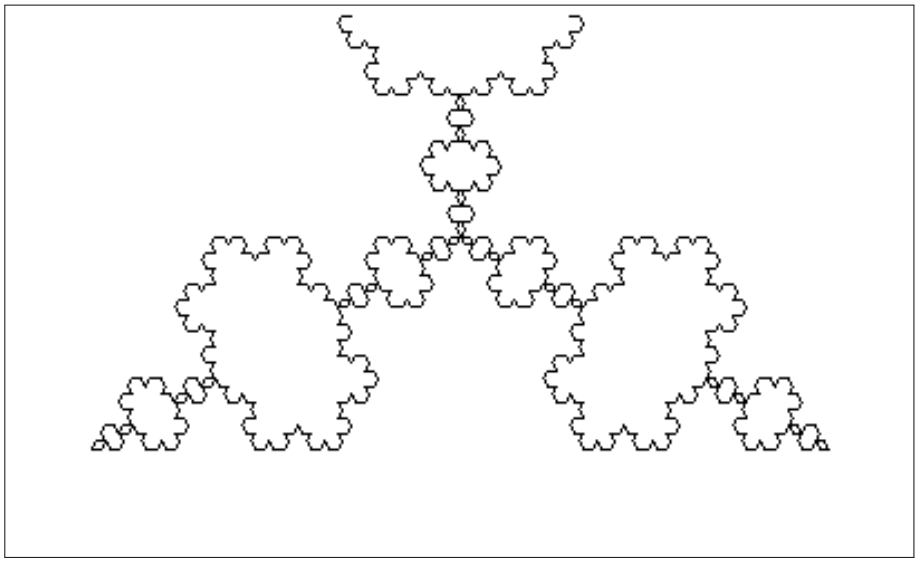
Begin with one interval. Remove its open middle third, then repeat on each remaining interval. The DM42 program draws one row per recursion depth. At depth d, 2^d intervals remain, each 3^{-d} of the original length.



The stopping depth is practical: eventually an interval becomes narrower than a pixel, and further mathematical detail cannot be distinguished on this display.

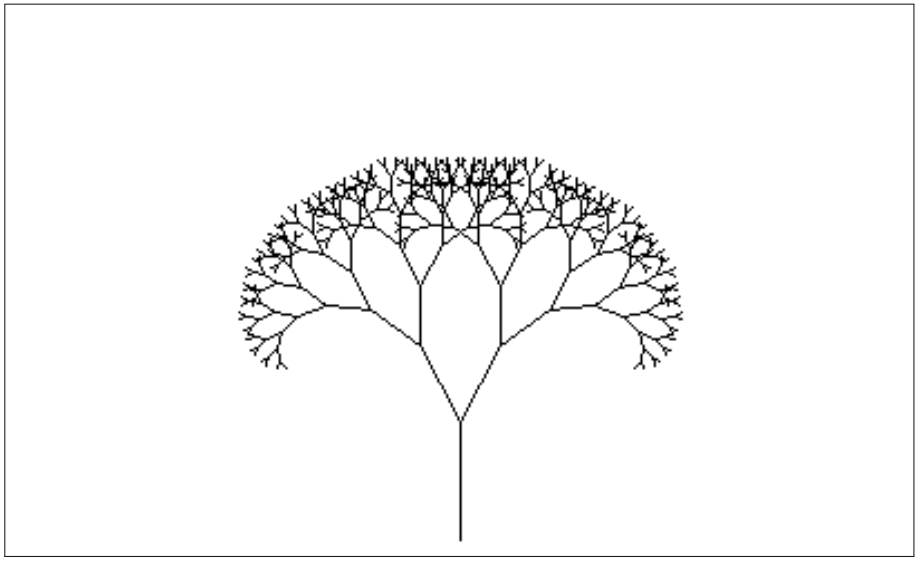
8.1.2 Koch snowflake

Replace each segment by four segments one third as long, with the middle pair forming an equilateral bump. Segment count grows as $3 \cdot 4^d$. Store endpoints and headings, or use a stack of pending segment commands. Depth 4 already draws 768 segments.



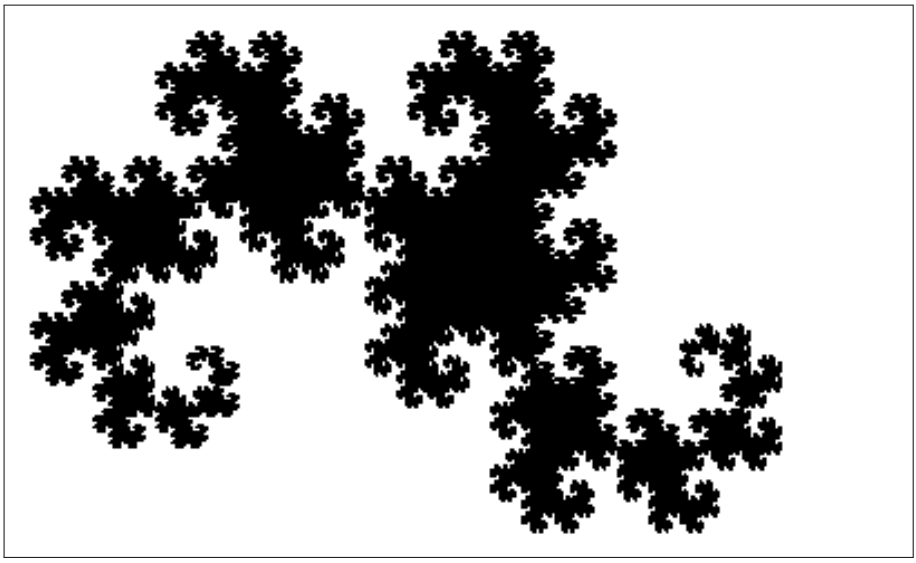
8.1.3 Fractal tree

Draw a trunk, turn by $+\alpha$ for one shorter branch, return, then turn by $-\alpha$ for the other. The return step is why endpoint state must be saved before each recursive branch. The example uses scale 0.72, angle 0.48 radians and depth 9.



8.1.4 Dragon curve

The folding sequence can be built iteratively: take the old turn list, append a left turn, then append the reversed old list with every turn inverted. Convert turns to unit steps and rescale the completed point set to the screen.

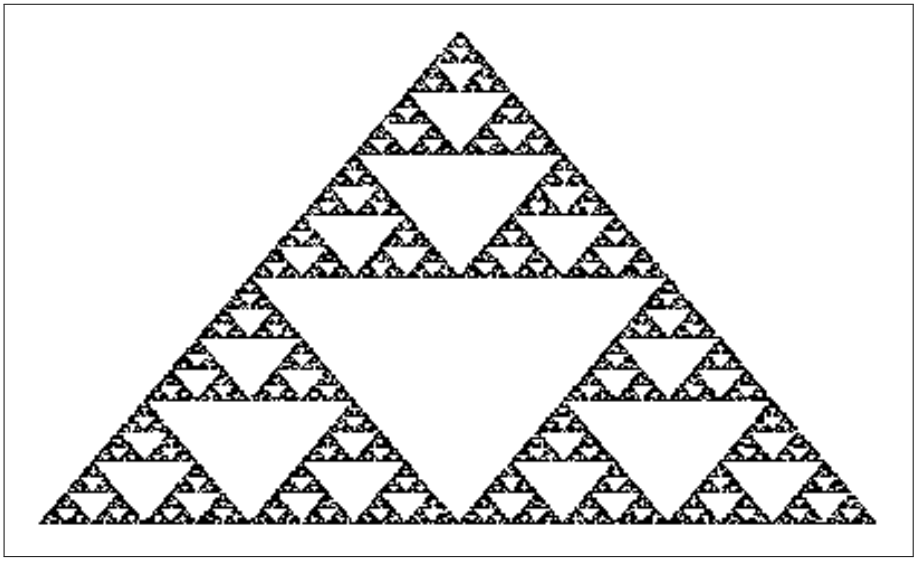


8.2 Iterated point fractals

8.2.1 Sierpiński triangle: the chaos game

Choose one of three vertices at random and move halfway from the current point to that vertex. After a short transient, plot each new point.

1. Start at any point inside the triangle.
2. Generate 0, 1 or 2 from the calculator's random number.
3. Select the corresponding vertex.
4. Replace (x, y) by the midpoint coordinates.
5. Map and plot; repeat thousands of times.



The empty central triangle is not explicitly erased. It is a region the midpoint rule never visits after the process settles.

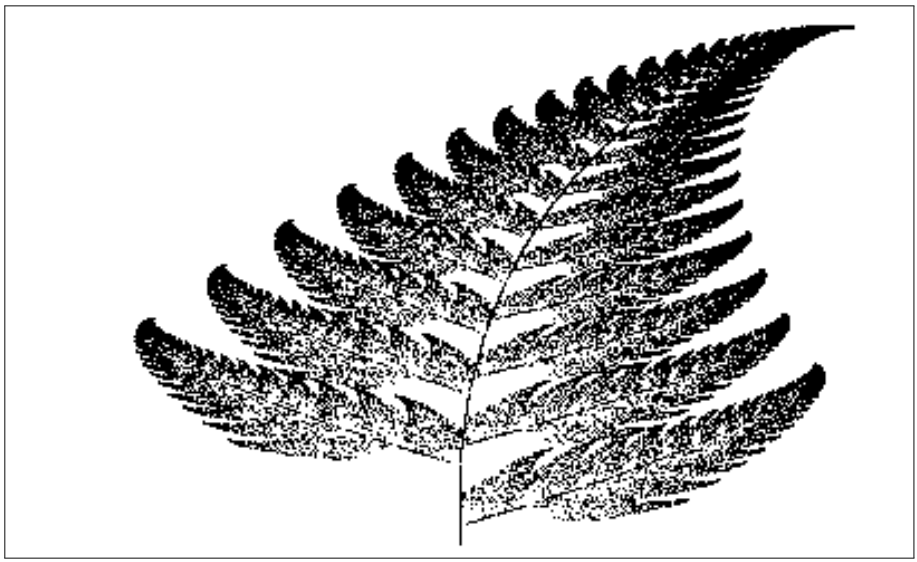
8.2.2 Barnsley fern

Every step chooses one affine transformation:

$$x' = ax + by + e, \quad y' = cx + dy + f.$$

Probability	a	b	c	d	e	f
.01	0	0	0	.16	0	0
.85	.85	.04	-.04	.85	0	1.60
.07	.20	-.26	.23	.22	0	1.60
.07	-.15	.28	.26	.24	0	.44

The first rule draws the stem; the second contracts the whole fern upward; the last two create leaflets. The probabilities determine how densely each structural role is visited.



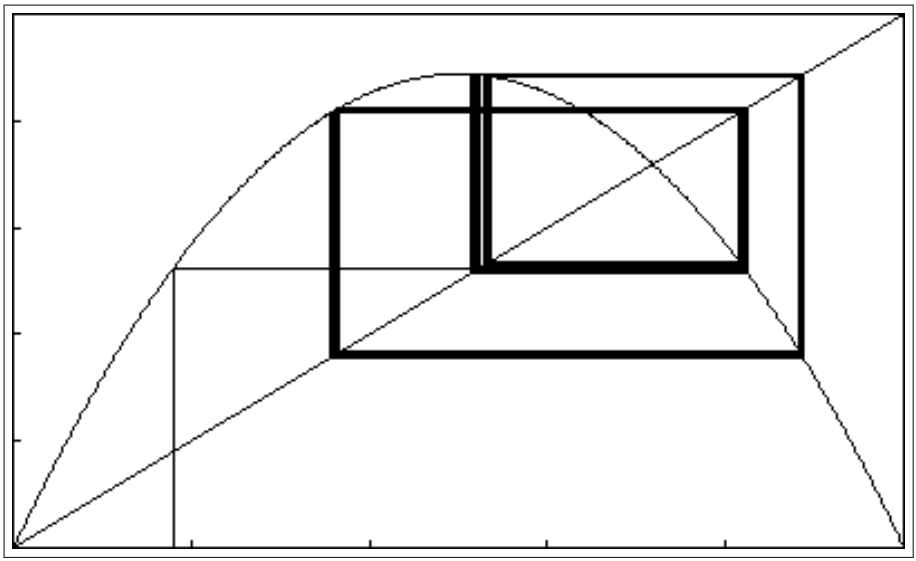
8.3 The logistic map as a laboratory

8.3.1 Cobweb plot

For

$$x_{n+1} = rx_n(1 - x_n),$$

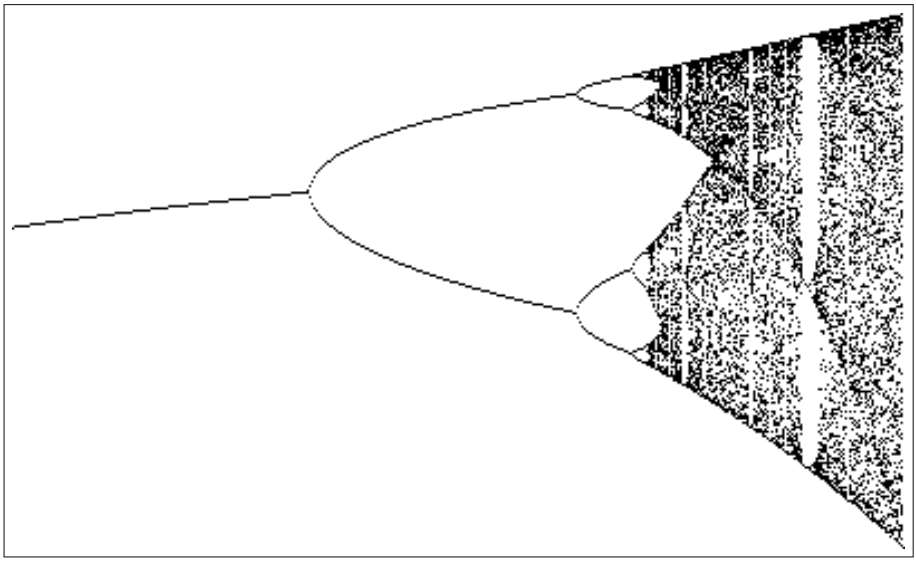
draw the parabola $y = rx(1 - x)$ and the diagonal $y = x$. Starting at x_0 on the X-axis, move vertically to the parabola, horizontally to the diagonal, then repeat. The vertical step computes the next value; the horizontal step makes it the next input.



The displayed case uses $r = 3.55$ and $x_0 = 0.18$. The repeated rectangle-like path settles into a multi-value cycle.

8.3.2 Bifurcation diagram

For each horizontal pixel, map to $r \in [2.5, 4]$. Iterate from $x_0 = 0.5$, discard 500 transient states, then plot the next 100 states vertically. Stable fixed points make one strand; period-doubling makes two, four and eight; chaotic bands become dense.



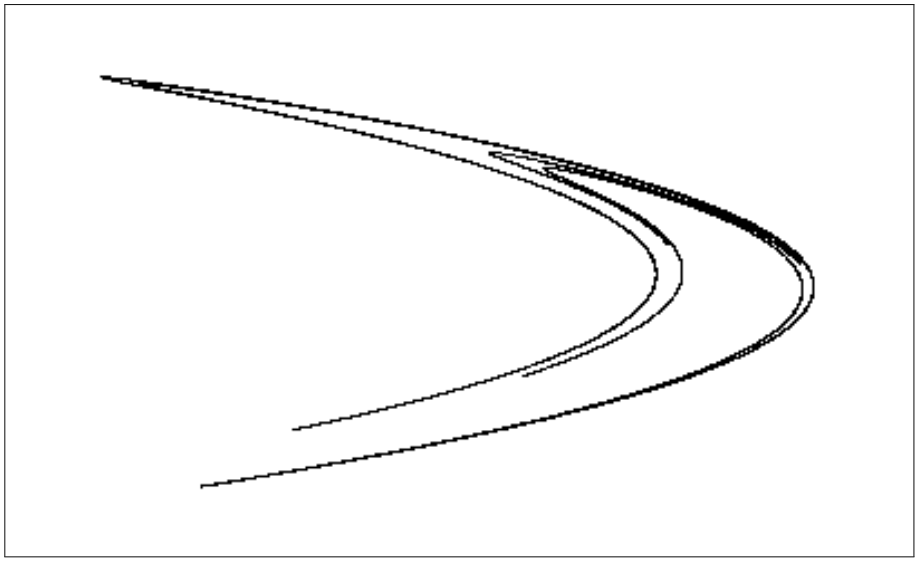
A whole time series is calculated for each column, so this is expensive. Preview at 200×120, then make the 400×240 final pass.

8.4 Two planar attractors

8.4.1 Hénon map

$$x_{n+1} = 1 - 1.4x_n^2 + y_n, \quad y_{n+1} = 0.3x_n.$$

Use temporary registers for both next values before overwriting the old state. Discard the first 50 points, then plot about 50,000.

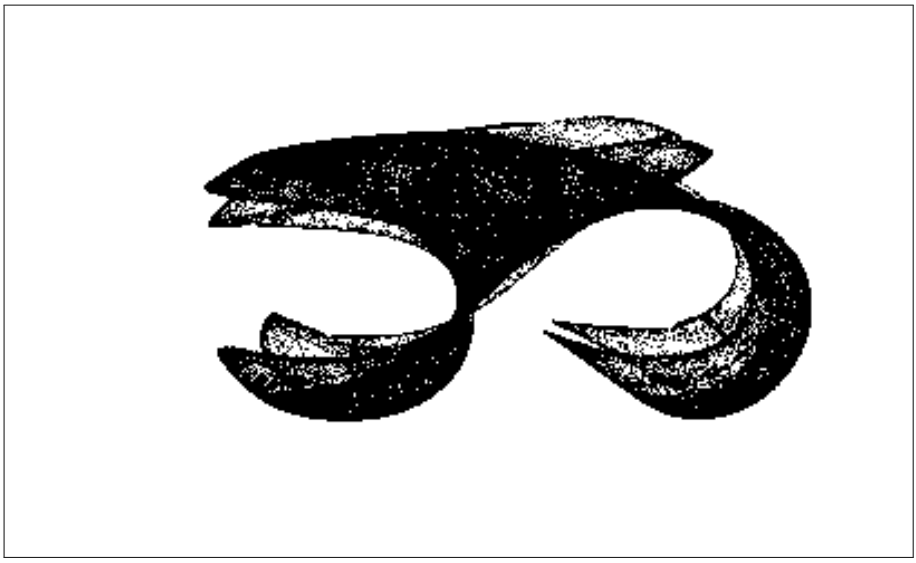


8.4.2 Clifford map

One useful form is

$$x' = \sin(ay) + c\cos(ax), \quad y' = \sin(bx) + d\cos(by).$$

The displayed parameters are $a = -1.4, b = 1.6, c = 1.0, d = 0.7$. Small coefficient changes can replace the attractor completely; there is no expectation of a gentle deformation.



8.5 Continuous systems by small steps

The Lorenz and Rössler equations define derivatives rather than a next point. With Euler stepping,

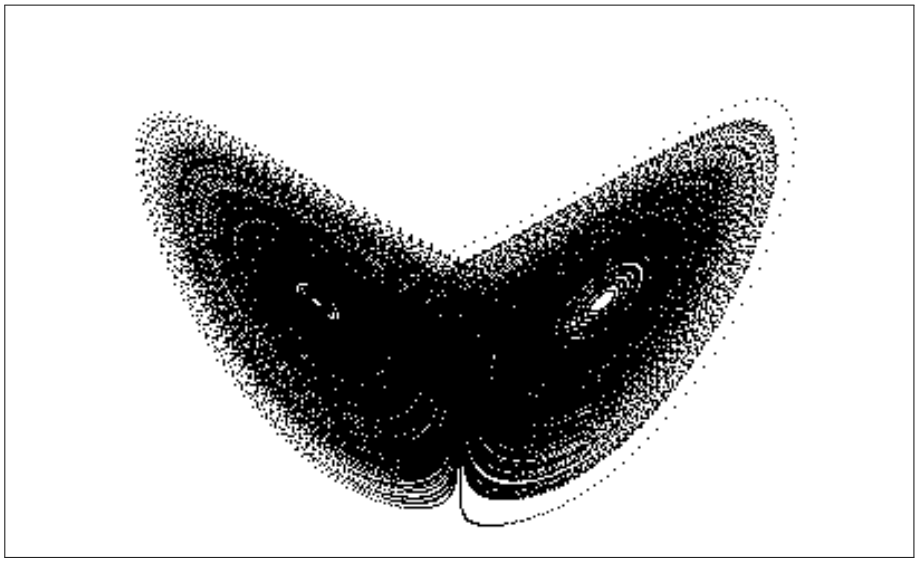
$$x \leftarrow x + h\dot{x}, \quad y \leftarrow y + h\dot{y}, \quad z \leftarrow z + h\dot{z}.$$

All derivatives must be calculated from the old state before any state register is overwritten.

8.5.1 Lorenz attractor

$$\dot{x} = 10(y - x), \quad \dot{y} = x(28 - z) - y, \quad \dot{z} = xy - \frac{8}{3}z.$$

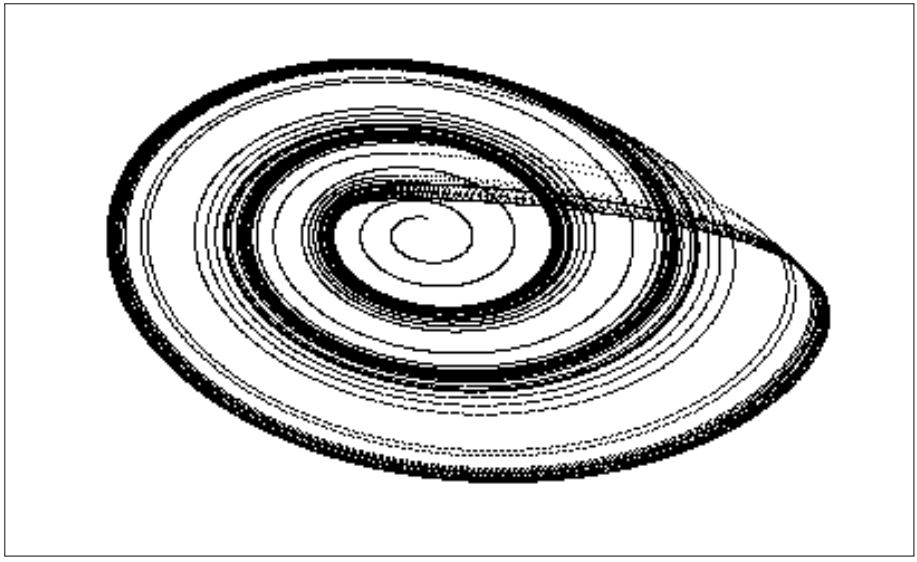
Use $h = 0.005$, discard roughly 2000 steps, then project (x, z) or the rotated 3-D point. Smaller h costs time but improves stability.



8.5.2 Rössler attractor

$$\dot{x} = -y - z, \quad \dot{y} = x + 0.2y, \quad \dot{z} = 0.2 + z(x - 5.7).$$

With $h = 0.01$, the X-Y projection shows a spiral repeatedly folding outward and reinjecting.



Plotting trap

A numerical trajectory is a result of both the differential equations and the step method. If the picture changes drastically when h is halved, the original step was not yet trustworthy.

CHAPTER 9

Three dimensions on a two-dimensional screen

A 3-D plot is still a 2-D framebuffer. The program rotates a three-dimensional point, projects it to two coordinates and then uses the same segment routine as a parametric curve.

9.1 Rotation and orthographic projection

Rotate (x, y, z) by yaw ψ about the vertical axis:

$$x_1 = x\cos\psi - y\sin\psi, \quad y_1 = x\sin\psi + y\cos\psi.$$

Then pitch by θ:

$$y_2 = y_1\cos\theta - z\sin\theta.$$

An orthographic screen projection is

$$x_s = c_x + sx_1, \quad y_s = c_y - sy_2,$$

where s is the screen scale. There is no division by depth, so parallel world lines remain parallel.

Perspective adds depth foreshortening, for example

$$x_s = c_x + \frac{sx_1}{d + z_2}, \quad y_s = c_y - \frac{sy_2}{d + z_2},$$

but it also introduces a near-plane singularity. Orthographic projection is the clearer first toolkit.

9.2 The 3-D routine contract

`PROJ` reads world X,Y,Z from R50–R52 and rotation/scale values from R53–R57. It writes screen X,Y to R27,R28 so the existing `SEG` can join projected points.

R53	yaw ψ	R54	pitch θ
R55	scale s	R56,R57	screen centre (c_x, c_y)

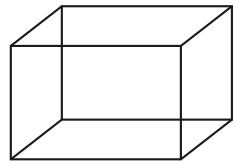
The `GRID3D` program samples a 13×13 grid on $[-2, 2]^2$, first drawing constant-Y rows and then constant-X columns. `ZFUN` supplies $z = f(x, y)$. The full listing and keystrokes are in appendix B.5.

For the default view, store

[·] [7] [5] [+/-] [STO] [5] [3] [·] [5] [2] [STO] [5] [4] [4] [8] [STO] [5] [5] [2] [0] [0] [STO] [5] [6] [1] [2] [0] [STO] [5] [7]

9.3 A wireframe cube before surfaces

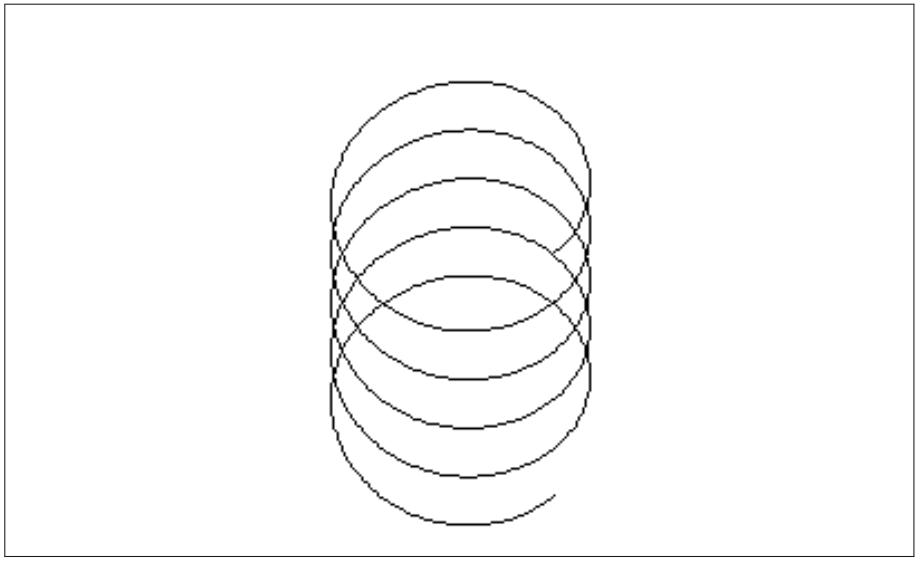
Store eight vertices $(\pm 1, \pm 1, \pm 1)$. Project all eight. Join the twelve vertex pairs that differ in exactly one coordinate. This separates the projection problem from surface evaluation and makes a wrong rotation sign obvious.



9.4 3-D helix

$$x = \cos t, \quad y = \sin t, \quad z = 0.12t - 1.8.$$

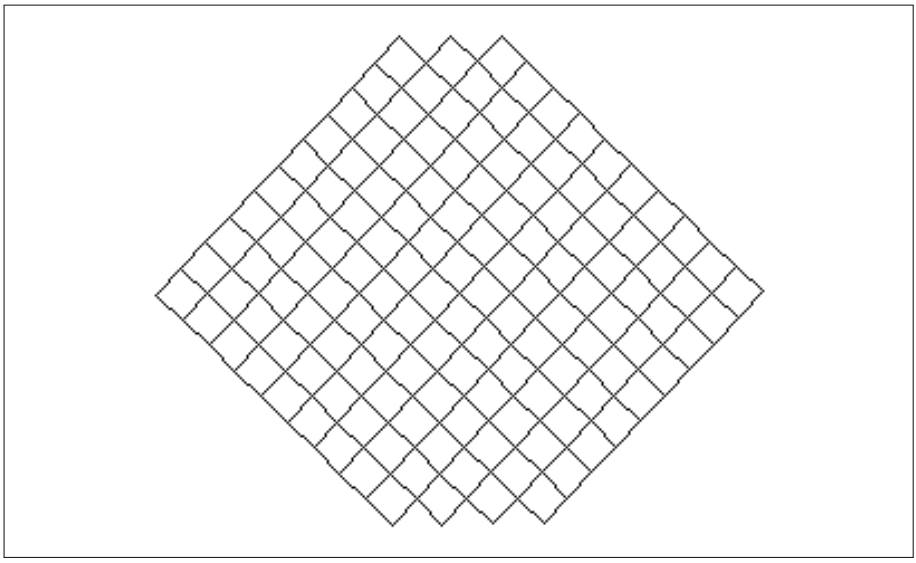
This is a parametric curve in 3-D: calculate one point, project, join, repeat. It is the natural bridge from `PPLOT` to surfaces.



9.5 Plane

$$z = 0.5x - 0.35y.$$

This is the default `ZFUN`. Because all world grid lines are straight and the projection is linear, every mesh line remains straight.

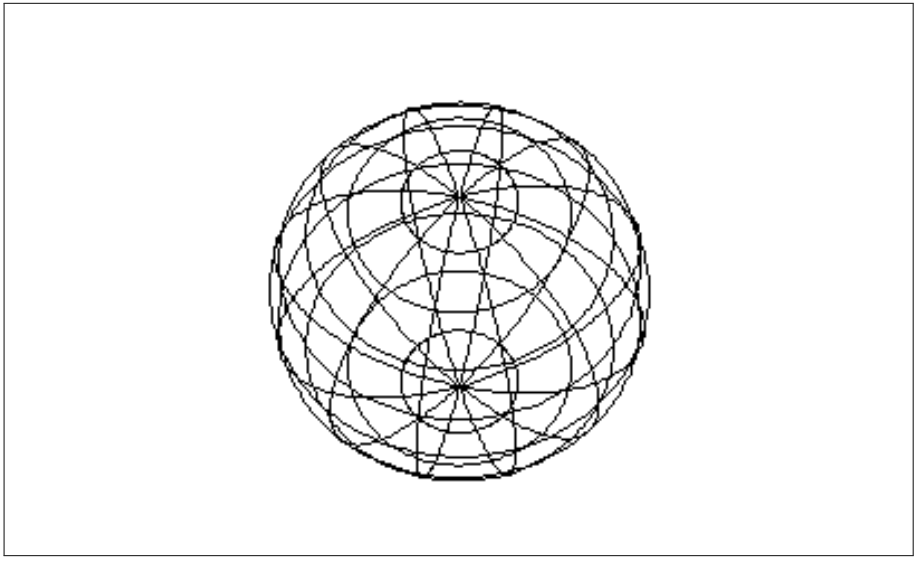


9.6 Sphere

Use latitude ϕ and longitude λ:

$$x = \cos\phi\cos\lambda, \quad y = \cos\phi\sin\lambda, \quad z = \sin\phi.$$

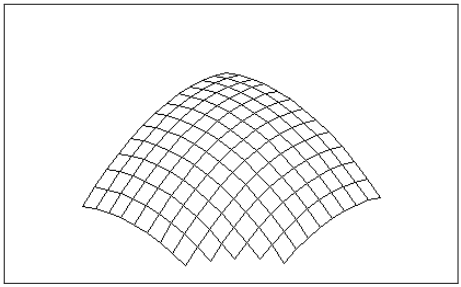
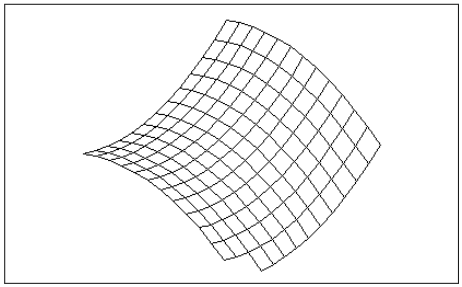
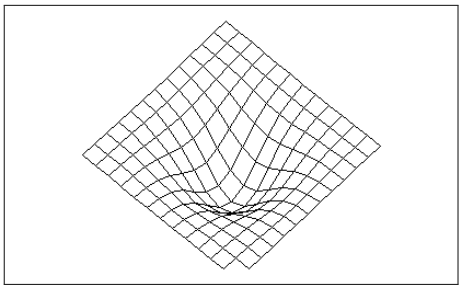
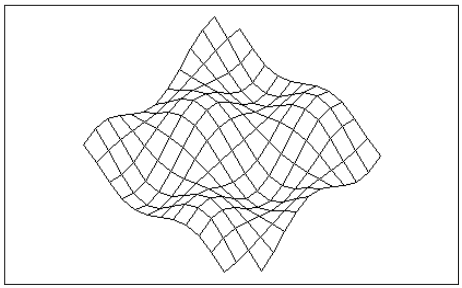
Draw selected constant-latitude and constant-longitude curves rather than a rectangular height mesh.



9.7 Four height surfaces

Replace `ZFUN`; leave projection and mesh traversal untouched.

Surface	$z = f(x, y)$	New issue
Paraboloid	$x^2 + y^2$	rapid positive growth
Saddle	$x^2 - y^2$	positive and negative curvature
Gaussian	$e^{-x^2-y^2}$	localised peak
Sinusoidal	$\sin x \cos y$	two-directional oscillation

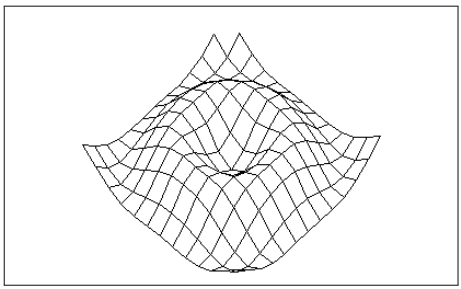
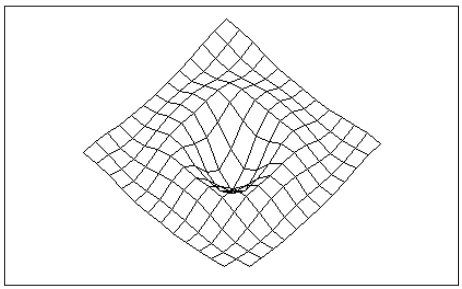





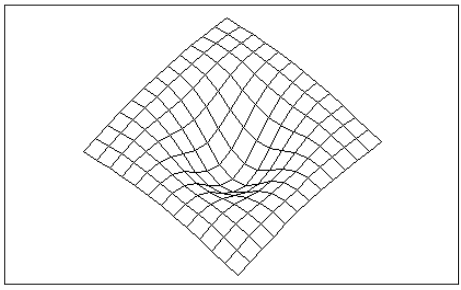
9.8 Radial surfaces

Let $r = \sqrt{x^2 + y^2}$.

$$\text{ripple: } z = \cos r,$$
$$\text{sinc: } z = \frac{\sin r}{r}, \quad z(0) = 1,$$
$$\text{Mexican hat: } z = (1 - r^2)e^{-r^2/2}.$$

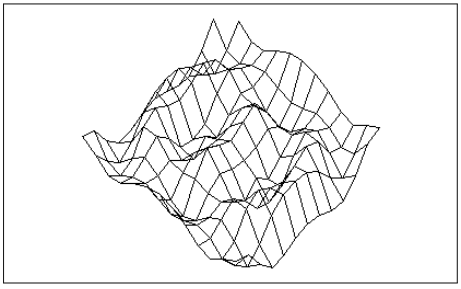
The sinc centre needs an explicit $r = 0$ branch. The limiting value is 1; blindly dividing would stop the program.

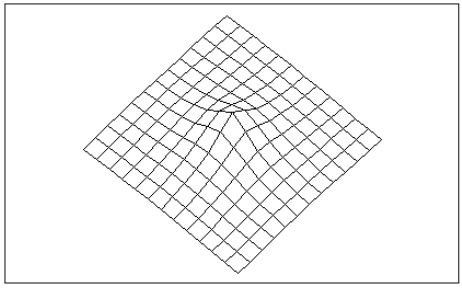





9.9 Two-source waves and a potential well

For wave sources at $x = \pm 0.7$, use $z = \sin(5r_1) + \sin(5r_2)$. For a softened potential well, use $z = -1/\sqrt{x^2 + y^2 + 0.15}$. Both demand careful vertical scale: the first oscillates; the second has a deep narrow feature.



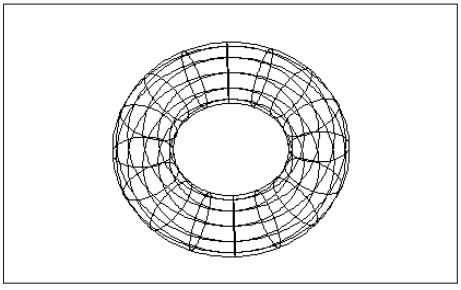


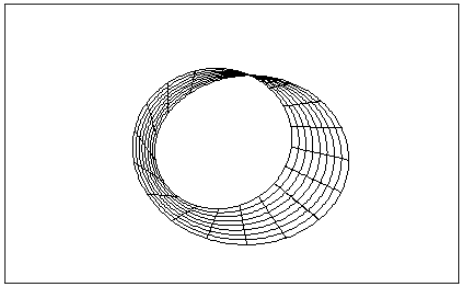
9.10 Torus and Möbius strip

These are two-parameter surfaces rather than height functions.

$$\begin{aligned} \text{torus: } & x = (R + r\cos v)\cos u, \quad & y = (R + r\cos v)\sin u, \quad & z = r\sin v, \\ \text{Möbius: } & x = (1 + v\cos(u/2))\cos u, \quad & y = (1 + v\cos(u/2))\sin u, \quad & z = v\sin(u/2). \end{aligned}$$

For the Möbius strip, $u \in [0, 2\pi]$ and $v \in [-w, w]$. The half-angle gives the strip one twist before it closes.





Try this

Change only yaw R53 and rerun the plane. Then change only pitch R54. Because the surface formula is unchanged, every visual change comes from the view transformation. This is the cleanest way to learn the rotations.

CHAPTER 10

Complex-plane fractals

The final plots classify each screen pixel by an iterative experiment. A pixel first becomes a complex number; the program then asks how its orbit behaves.

10.1 Mandelbrot: understand one pixel first

Map screen column $u = 0, \ldots, 399$ and row $v = 0, \ldots, 239$ to

$$c_x = -2.35 + 3.2\frac{u}{399}, \quad c_y = 1.2 - 2.4\frac{v}{239}.$$

Set $z_0 = 0$, then repeat

$$z_{n+1} = z_n^2 + c.$$

Writing $z = x + iy$ makes the calculator work explicit:

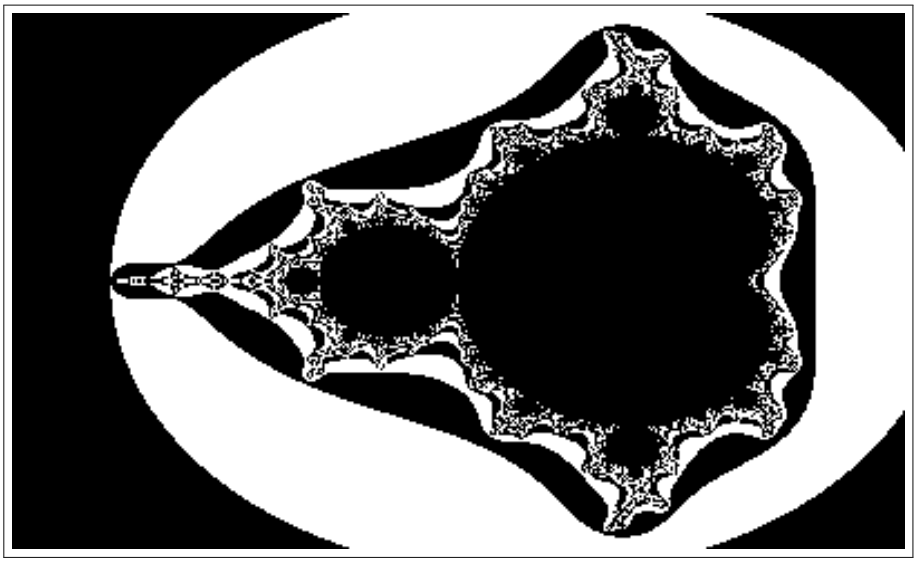
$$x' = x^2 - y^2 + c_x, \quad y' = 2xy + c_y.$$

If $x^2 + y^2 > 4$, the orbit has escaped. If it has not escaped after 48 iterations, light the pixel. Then move to the next row and column.

Central idea for escape-time plots

The screen pixel is not merely where the answer is drawn. It also chooses the input c. Mapping, iteration and classification happen once for every pixel.

The complete `MANDEL` program in appendix B.6 uses real register pairs so the update order is visible. R40,R41 are zero-based screen indices; R42,R43 hold (c_x, c_y); R44,R45 hold (x, y); R46 is the iteration count; R47 protects the new X while the new Y is calculated. `RefLCD` suppresses periodic redraw during the long nested loops.



10.2 Preview, symmetry and progressive work

The set is symmetric about the real axis. A faster program may compute only 120 rows and mirror them, provided the window is symmetric. A 200×120 mode-2 preview is four times fewer pixels than mode 3. Progressive refinement can calculate every fourth pixel, then fill the gaps.

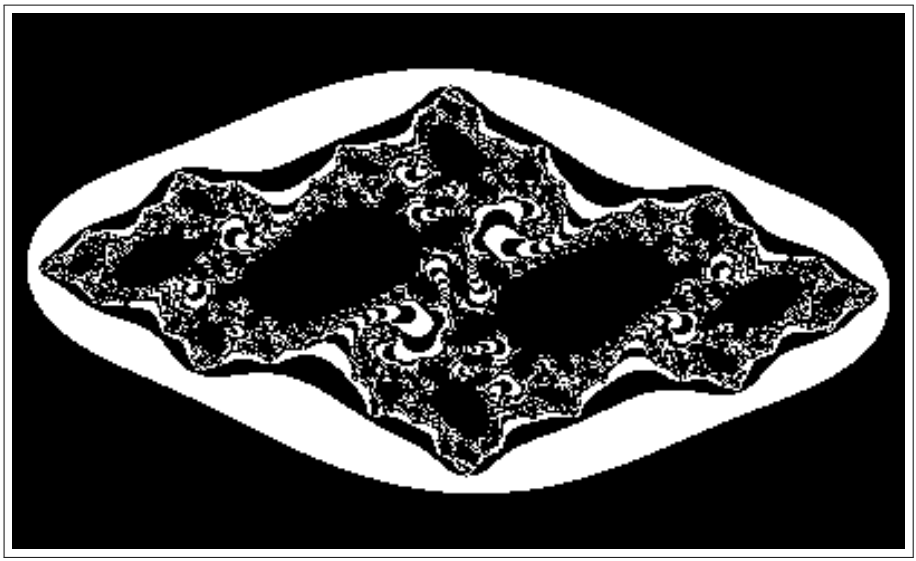
Monochrome escape-time decoration can use iteration parity, bands such as $n \bmod 4$, or ordered dithering. The solid membership test is the most mathematically direct and is what `MANDEL` implements.

10.3 Julia sets: fix C, vary the starting point

The iteration is still $z \leftarrow z^2 + c$, but now c is a fixed constant and the screen pixel supplies z_0. The displayed set uses

$$c = -0.745 + 0.113i.$$

Modify `MANDEL` by mapping R44,R45 directly from the pixel before the iteration, and store the fixed constant in R42,R43 once outside the loops.



Try this

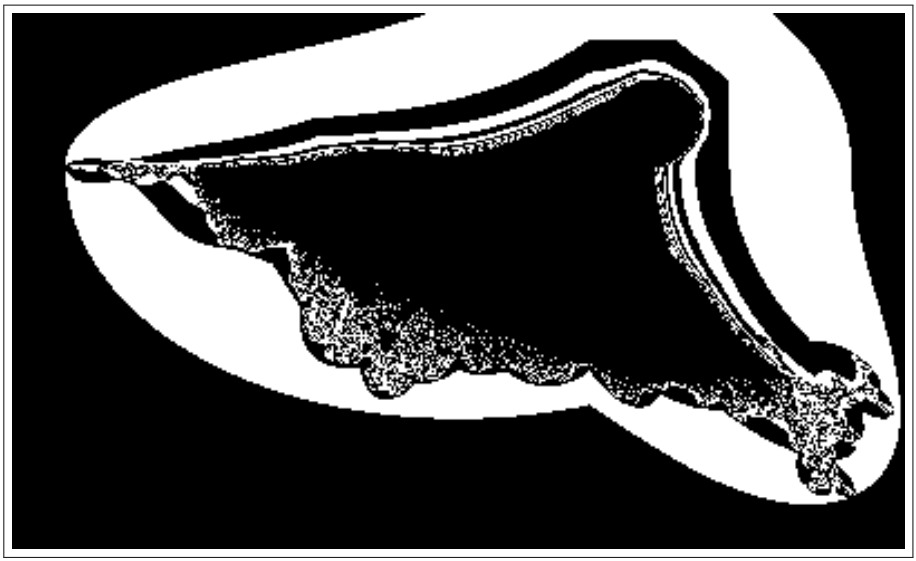
Try $c = -0.8 + 0.156i$, $c = -0.4 + 0.6i$, and $c = 0.285 + 0.01i$. Keep the same window so the change of geometry is attributable to c.

10.4 Burning Ship: a small formula change

Before squaring, replace X and Y by their absolute values:

$$z_{n+1} = (|\Re z_n| + i|\Im z_n|)^2 + c.$$

In the real-pair program, insert `ABS` after recalling each of R44 and R45 in both update expressions. Use window $[-2.2, 1.2] \times [-2.0, 0.8]$.



10.5 Newton fractal: classify convergence, not escape

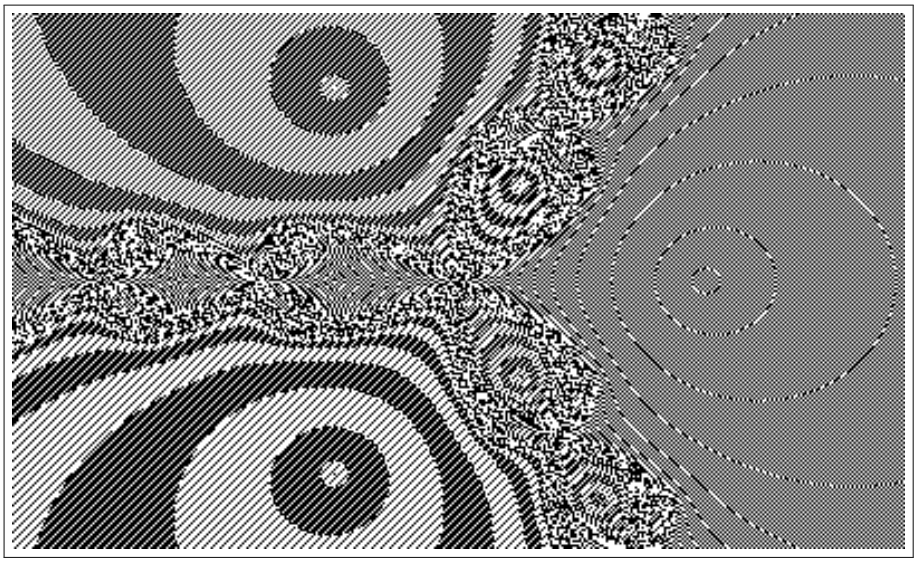
For $p(z) = z^3 - 1$, Newton's method is

$$z_{n+1} = z_n - \frac{z_n^3 - 1}{3z_n^2}.$$

The polynomial has three roots:

$$1, \quad -\frac{1}{2} + \frac{\sqrt{3}}{2}i, \quad -\frac{1}{2} - \frac{\sqrt{3}}{2}i.$$

Each pixel supplies z_0. Iterate until the point is close to a root or a limit is reached. The picture classifies which root wins. On a monochrome screen, use distinct patterns: solid, checkerboard and two-on/two-off bands; iteration parity can add convergence-speed texture.



10.6 Complex-number operations on the DM42

The calculator can store $a + ib$ directly: enter the real part, [ENTER], the imaginary part, then [f] [COMPLEX]. With RECT coordinate mode selected, `COMPLEX` combines Y as the real part and X as the imaginary part. Complex multiplication and powers then follow normal arithmetic rules.

For teaching and speed, `MANDEL` keeps real and imaginary parts in separate registers. A complex version is shorter on paper but hides the two temporary values needed to avoid overwriting an old component too early.

Plotting trap

At 400×240 and 48 iterations, the worst case approaches 4.6 million orbit updates. A blank physical display during controlled refresh does not mean the program has failed. Use a preview or add a coarse progress indicator before assuming a fault.

10.7 The final lesson

The Mandelbrot set looks like a new subject, but its plotting sequence is familiar:

1. map a pixel to mathematics;
2. evaluate a rule repeatedly;
3. classify the result;
4. light a pixel;
5. move to the next sample.

The equation became complicated. The plotting machinery did not.

CHAPTER A

Keystroke atlas

A.1 Program editing

Task	Exact route
New program area	f GTO ·· , then f PRGM
Leave/enter program mode	f PRGM
Global or local `LBL`	f PGM.FCN LBL, then name or two digits
`RTN`	f PGM.FCN RTN
Call a named routine	XEQ, spell label, ENTER
Local `GTO`	f GTO, then two digits
Delete current line	←
Move one line	▲/▼ when no menu is open

A.2 Program-function pages

Page	Softkeys
1	`LBL`, `RTN`, `INPUT`, `VIEW`, `AVIEW`, `XEQ`
2	`X?0`, `X?Y`, `PROMPT`, `PSE`, `ISG`, `DSE`
3	`AIP`, `XTOA`, `AGRAPH`, `PIXEL`, `BEEP`, `TONE`
4	`MVAR`, `VARMENU`, `GETKEY`, `MENU`, `KEYG`, `KEYX`

Open page 1 with f PGM.FCN; ▼ moves forward and ▲ moves backward. The physical unshifted XEQ inserts a real subroutine call in program mode.

A.3 Exact mixed-case virtual names

Virtual-variable names are case-sensitive. When an alpha prompt opens, use f ▲ or f ▼ until the annunciator shows DM uppercase `[A]`.

Name	Exact character sequence from `[A]`
`GrMod`	G f ▼ r f ▲ M f ▼ o d
`RefLCD`	R f ▼ e f f ▲ L C D
`ResX`	R f ▼ e s f ▲ X
`ResY`	R f ▼ e s f ▲ Y

A.4 Frequently used functions

Instruction	Keys
CLLCD	f CLEAR ▼ CLLCD
PIXEL	f PGM.FCN ▼ ▼ PIXEL
AGRAPH	f PGM.FCN ▼ ▼ AGRAPH
XTOA	f PGM.FCN ▼ ▼ XTOA
ABS	f CONVERT ▼ ABS
X↑2	f x^2
E↑X	f E↑X
PI	f π
Radians	f MODES RAD
Set/clear flag	f FLAGS SF or CF, two digits
Compare X with 0	f PGM.FCN ▼ X?0, choose test
Compare X with Y	f PGM.FCN ▼ X?Y, choose test

CHAPTER B

Complete program listings and entry tables

The listings below are consolidated program areas. Create a new area once, then enter the global routines consecutively before the automatic `END`. Each listing is followed by the exact keys for every program line.

B.1 First-pixel and first-line programs

```
00 { 107-Byte Prgm }
01 LBL "DOT"
02 3
03 STO "GrMod"
04 CLLCD
05 120
06 ENTER
07 200
08 PIXEL
09 RTN
10 LBL "LINE1"
11 3
12 STO "GrMod"
13 CLLCD
14 0
15 STO 00
16 LBL 00
17 RCL 00
18 399
19 ÷
20 10
21 ×
22 5
23 -
24 STO 01
25 5
26 RCL- 01
27 10
28 ÷
29 239
30 ×
31 1
32 +
33 STO 02
34 RCL 02
35 RCL 00
36 1
37 +
38 PIXEL
39 RCL 00
40 1
41 +
42 STO 00
43 RCL 00
44 400
45 X>Y?
46 GTO 00
47 RTN
48 END
```

First Plots: complete entry table

Line	Instruction	Exact keys
01	LBL "DOT"	f PGM.FCN LBL D O T ENTER
02	3	3
03	STO "GrMod"	STO ENTER *set [A]* G f ▼ r f ▲ M f ▼ o d ENTER
04	CLLCD	f CLEAR ▼ CLLCD
05	120	1 2 0
06	ENTER	ENTER
07	200	2 0 0
08	PIXEL	f PGM.FCN ▼ ▼ PIXEL
09	RTN	f PGM.FCN RTN
10	LBL "LINE1"	f PGM.FCN LBL L I N E 1 ENTER
11	3	3
12	STO "GrMod"	STO ENTER *set [A]* G f ▼ r f ▲ M f ▼ o d ENTER
13	CLLCD	f CLEAR ▼ CLLCD
14	0	0
15	STO 00	STO 0 0
16	LBL 00	f PGM.FCN LBL 0 0
17	RCL 00	RCL 0 0
18	399	3 9 9
19	÷	÷
20	10	1 0
21	×	×
22	5	5
23	-	-
24	STO 01	STO 0 1
25	5	5
26	RCL- 01	RCL - 0 1
27	10	1 0
28	÷	÷
29	239	2 3 9
30	×	×
31	1	1
32	+	+
33	STO 02	STO 0 2
34	RCL 02	RCL 0 2
35	RCL 00	RCL 0 0
36	1	1
37	+	+
38	PIXEL	f PGM.FCN ▼ ▼ PIXEL
39	RCL 00	RCL 0 0
40	1	1
41	+	+
42	STO 00	STO 0 0
43	RCL 00	RCL 0 0
44	400	4 0 0
45	X>Y?	f PGM.FCN ▼ X?Y X>Y?
46	GTO 00	f GTO 0 0
47	RTN	f PGM.FCN RTN
48	END	Already present at the end of a new program area.

B.2 Cartesian toolkit

Registers R00–R03 are the window; R04 is the zero-based column; R05,R06 are world X,Y; R07,R08 are current/previous rows; R09 is the previous-point flag. R10 and R11 are the X and Y tick intervals, and R17 switches integer tick values on or off. R12–R16 and R18–R19 are temporary workspace for `VJOIN`, ticks and labels. Flag 00 is the function-valid flag.

The listing begins with the plotting core: `GSET`, the coordinate mappings, `AXES`, `VJOIN`, `CPLOT` and `YFUN`. The optional tick-mark and integer-label helpers come afterward, keeping the principal plotting path together and easy to find.

```
00 { 1240-Byte Prgm }
01 LBL "GSET"
02 3
03 STO "GrMod"
04 RAD
05 CLLCD
06 RTN
07 LBL "MAPX"
08 RCL- 00
09 RCL 01
10 RCL- 00
11 ÷
12 399
13 ×
14 1
15 +
16 RTN
17 LBL "MAPY"
18 RCL 03
19 X<>Y
20 -
21 RCL 03
22 RCL- 02
23 ÷
24 239
25 ×
26 1
27 +
28 RTN
29 LBL "AXES"
30 1
31 +/-
32 ENTER
33 0
34 PIXEL
35 240
36 +/-
37 ENTER
38 0
39 PIXEL
40 0
41 ENTER
42 1
43 +/-
44 PIXEL
45 0
46 ENTER
47 400
48 +/-
49 PIXEL
50 0
51 XEQ "MAPY"
52 +/-
53 ENTER
54 0
55 PIXEL
56 0
57 XEQ "MAPX"
58 +/-
59 0
60 X<>Y
61 PIXEL
62 CF 34
63 CF 35
64 XEQ "XTICK"
65 XEQ "YTICK"
66 RTN
67 LBL "VJOIN"
68 RCL 08
69 STO 13
70 RCL 07
71 RCL- 08
72 STO 12
73 X<0?
74 GTO 20
75 1
76 GTO 21
77 LBL 20
78 1
79 +/-
```

```
80 LBL 21
81 STO 14
82 RCL 12
83 ABS
84 STO 15
85 X=0?
86 RTN
87 LBL 22
88 RCL 13
89 RCL+ 14
90 STO 13
91 RCL 13
92 RCL 04
93 1
94 +
95 PIXEL
96 RCL 15
97 1
98 -
99 STO 15
100 X>0?
101 GTO 22
102 RTN
103 LBL "CPLOT"
104 CLLCD
105 XEQ "AXES"
106 0
107 STO 04
108 STO 09
109 LBL 00
110 RCL 04
111 RCL 01
112 RCL- 00
113 ×
114 399
115 ÷
116 RCL+ 00
117 STO 05
118 XEQ "YFUN"
119 STO 06
120 FS? 00
121 GTO 01
122 0
123 STO 09
124 GTO 04
125 LBL 01
126 RCL 06
127 RCL- 02
128 X<0?
129 GTO 02
130 RCL 06
131 RCL- 03
132 X>0?
133 GTO 02
134 RCL 06
135 XEQ "MAPY"
136 STO 07
137 RCL 09
138 X=0?
139 GTO 03
140 RCL 07
141 RCL- 08
142 ABS
143 80
144 X<Y?
145 GTO 02
146 XEQ "VJOIN"
147 GTO 05
148 LBL 03
149 RCL 07
150 RCL 04
151 1
152 +
153 PIXEL
154 LBL 05
155 RCL 07
156 STO 08
157 1
158 STO 09
159 GTO 04
```

```
160 LBL 02
161 0
162 STO 09
163 LBL 04
164 RCL 04
165 1
166 +
167 STO 04
168 RCL 04
169 400
170 X>Y?
171 GTO 00
172 RTN
173 LBL "YFUN"
174 SF 00
175 RCL 05
176 RTN
177 LBL "XTICK"
178 RCL 10
179 ABS
180 STO 10
181 X=0?
182 RTN
183 0
184 RCL- 02
185 X<0?
186 RTN
187 0
188 RCL- 03
189 X>0?
190 RTN
191 0
192 XEQ "MAPY"
193 STO 12
194 0
195 STO 16
196 LBL 30
197 RCL 16
198 RCL- 01
199 X>0?
200 GTO 31
201 XEQ "XMARK"
202 RCL 16
203 RCL+ 10
204 STO 16
205 GTO 30
206 LBL 31
207 RCL 10
208 +/-
209 STO 16
210 LBL 32
211 RCL 16
212 RCL- 00
213 X<0?
214 GTO 33
215 XEQ "XMARK"
216 RCL 16
217 RCL- 10
218 STO 16
219 GTO 32
220 LBL 33
221 RTN
222 LBL "YTICK"
223 RCL 11
224 ABS
225 STO 11
226 X=0?
227 RTN
228 0
229 RCL- 00
230 X<0?
231 RTN
232 0
233 RCL- 01
234 X>0?
235 RTN
236 0
237 XEQ "MAPX"
238 STO 12
239 0
```

```
240 STO 16
241 LBL 34
242 RCL 16
243 RCL- 03
244 X>0?
245 GTO 35
246 XEQ "YMARK"
247 RCL 16
248 RCL+ 11
249 STO 16
250 GTO 34
251 LBL 35
252 RCL 11
253 +/-
254 STO 16
255 LBL 36
256 RCL 16
257 RCL- 02
258 X<0?
259 GTO 37
260 XEQ "YMARK"
261 RCL 16
262 RCL- 11
263 STO 16
264 GTO 36
265 LBL 37
266 RTN
267 LBL "XMARK"
268 RCL 16
269 XEQ "MAPX"
270 STO 13
271 CLA
272 127
273 XTOA
274 RCL 12
275 3
276 -
277 RCL 13
278 AGRAPH
279 RCL 17
280 X=0?
281 RTN
282 RCL 16
283 ABS
284 10
285 X>Y?
286 GTO 38
287 RTN
288 LBL 38
289 RCL 12
290 5
291 +
292 STO 18
293 RCL 13
294 STO 19
295 RCL 16
296 XEQ "ILAB"
297 RTN
298 LBL "YMARK"
299 RCL 16
300 XEQ "MAPY"
301 STO 13
302 CLA
303 1
304 XTOA
305 1
306 XTOA
307 1
308 XTOA
309 1
310 XTOA
311 1
312 XTOA
313 1
314 XTOA
315 1
316 XTOA
317 RCL 13
318 RCL 12
319 3
```

```
320 -
321 AGRAPH
322 RCL 17
323 X=0?
324 RTN
325 RCL 16
326 X=0?
327 RTN
328 RCL 16
329 ABS
330 10
331 X>Y?
332 GTO 39
333 RTN
334 LBL 39
335 RCL 13
336 4
337 -
338 STO 18
339 RCL 12
340 15
341 +
342 STO 19
343 RCL 16
344 XEQ "ILAB"
345 RTN
346 LBL "ILAB"
347 RCL 16
348 ENTER
349 IP
350 -
351 X=0?
352 GTO 48
353 RTN
354 LBL 48
355 CLA
356 RCL 16
357 X<0?
358 GTO 50
359 GTO 51
360 LBL 50
361 8
362 XTOA
363 8
364 XTOA
365 8
366 XTOA
367 8
368 XTOA
369 8
370 XTOA
371 0
372 XTOA
373 LBL 51
374 RCL 16
375 ABS
376 STO 14
377 10
378 X>Y?
379 GTO 52
380 RTN
381 LBL 52
382 RCL 14
383 XEQ "DIGIT"
384 5
385 STO 15
386 RCL 16
387 X<0?
388 GTO 54
389 GTO 55
390 LBL 54
391 RCL 15
392 6
393 +
394 STO 15
395 LBL 55
396 RCL 15
397 2
398 ÷
399 RCL 19
```

```
400 X<>Y
401 -
402 STO 19
403 RCL 18
404 RCL 19
405 AGRAPH
406 RTN
407 LBL "DIGIT"
408 STO 15
409 RCL 15
410 X=0?
411 GTO 60
412 RCL 15
413 1
414 X=Y?
415 GTO 61
416 RCL 15
417 2
418 X=Y?
419 GTO 62
420 RCL 15
421 3
422 X=Y?
423 GTO 63
424 RCL 15
425 4
426 X=Y?
427 GTO 64
428 RCL 15
429 5
430 X=Y?
431 GTO 65
432 RCL 15
433 6
434 X=Y?
435 GTO 66
436 RCL 15
437 7
438 X=Y?
439 GTO 67
440 RCL 15
441 8
442 X=Y?
443 GTO 68
444 RCL 15
445 9
446 X=Y?
447 GTO 69
448 RTN
449 LBL 60
450 62
451 XTOA
452 81
453 XTOA
454 73
455 XTOA
456 69
457 XTOA
458 62
459 XTOA
460 GTO 70
461 LBL 61
462 0
463 XTOA
464 66
465 XTOA
466 127
467 XTOA
468 64
469 XTOA
470 0
471 XTOA
472 GTO 70
473 LBL 62
474 66
475 XTOA
476 97
477 XTOA
478 81
479 XTOA
```

```
480 73
481 XTOA
482 70
483 XTOA
484 GTO 70
485 LBL 63
486 33
487 XTOA
488 65
489 XTOA
490 69
491 XTOA
492 75
493 XTOA
494 49
495 XTOA
496 GTO 70
497 LBL 64
498 24
499 XTOA
500 20
501 XTOA
502 18
503 XTOA
504 127
505 XTOA
506 16
507 XTOA
508 GTO 70
509 LBL 65
510 39
511 XTOA
512 69
513 XTOA
514 69
515 XTOA
516 69
517 XTOA
518 57
519 XTOA
520 GTO 70
521 LBL 66
522 60
523 XTOA
524 74
525 XTOA
526 73
527 XTOA
528 73
529 XTOA
530 48
531 XTOA
532 GTO 70
533 LBL 67
534 1
535 XTOA
536 113
537 XTOA
538 9
539 XTOA
540 5
541 XTOA
542 3
543 XTOA
544 GTO 70
545 LBL 68
546 54
547 XTOA
548 73
549 XTOA
550 73
551 XTOA
552 73
553 XTOA
554 54
555 XTOA
556 GTO 70
557 LBL 69
558 6
559 XTOA
```

```
560 73
561 XTOA
562 73
563 XTOA
564 41
565 XTOA
566 30
567 XTOA
568 LBL 70
569 RTN
570 END
```

Cartesian: complete entry table

Line	Instruction	Exact keys
01	LBL "GSET"	f PGM.FCN LBL G S E T ENTER
02	3	3
03	STO "GrMod"	STO ENTER *set [A]* G f ▼ r f ▲ M f ▼ o d ENTER
04	RAD	f MODES RAD
05	CLLCD	f CLEAR ▼ CLLCD
06	RTN	f PGM.FCN RTN
07	LBL "MAPX"	f PGM.FCN LBL M A P X ENTER
08	RCL- 00	RCL - 0 0
09	RCL 01	RCL 0 1
10	RCL- 00	RCL - 0 0
11	÷	÷
12	399	3 9 9
13	×	×
14	1	1
15	+	+
16	RTN	f PGM.FCN RTN
17	LBL "MAPY"	f PGM.FCN LBL M A P Y ENTER
18	RCL 03	RCL 0 3
19	X<>Y	$x \leftrightarrow y$
20	-	-
21	RCL 03	RCL 0 3
22	RCL- 02	RCL - 0 2
23	÷	÷
24	239	2 3 9
25	×	×
26	1	1
27	+	+
28	RTN	f PGM.FCN RTN
29	LBL "AXES"	f PGM.FCN LBL A X E S ENTER
30	1	1
31	+/-	+/-
32	ENTER	ENTER
33	0	0
34	PIXEL	f PGM.FCN ▼ ▼ PIXEL
35	240	2 4 0
36	+/-	+/-
37	ENTER	ENTER
38	0	0
39	PIXEL	f PGM.FCN ▼ ▼ PIXEL
40	0	0
41	ENTER	ENTER
42	1	1
43	+/-	+/-
44	PIXEL	f PGM.FCN ▼ ▼ PIXEL
45	0	0
46	ENTER	ENTER
47	400	4 0 0
48	+/-	+/-
49	PIXEL	f PGM.FCN ▼ ▼ PIXEL
50	0	0
51	XEQ "MAPY"	XEQ M A P Y ENTER
52	+/-	+/-
53	ENTER	ENTER

Line	Instruction	Exact keys
54	0	0
55	PIXEL	f PGM.FCN ▼ ▼ PIXEL
56	0	0
57	XEQ "MAPX"	XEQ M A P X ENTER
58	+/-	+/-
59	0	0
60	X<>Y	$x \leftrightarrow y$
61	PIXEL	f PGM.FCN ▼ ▼ PIXEL
62	CF 34	f FLAGS CF 3 4
63	CF 35	f FLAGS CF 3 5
64	XEQ "XTICK"	XEQ X T I C K ENTER
65	XEQ "YTICK"	XEQ Y T I C K ENTER
66	RTN	f PGM.FCN RTN
67	LBL "VJOIN"	f PGM.FCN LBL V J O I N ENTER
68	RCL 08	RCL 0 8
69	STO 13	STO 1 3
70	RCL 07	RCL 0 7
71	RCL- 08	RCL - 0 8
72	STO 12	STO 1 2
73	X<0?	f PGM.FCN ▼ X?0 X<0?
74	GTO 20	f GTO 2 0
75	1	1
76	GTO 21	f GTO 2 1
77	LBL 20	f PGM.FCN LBL 2 0
78	1	1
79	+/-	+/-
80	LBL 21	f PGM.FCN LBL 2 1
81	STO 14	STO 1 4
82	RCL 12	RCL 1 2
83	ABS	f CONVERT ▼ ABS
84	STO 15	STO 1 5
85	X=0?	f PGM.FCN ▼ X?0 X=0?
86	RTN	f PGM.FCN RTN
87	LBL 22	f PGM.FCN LBL 2 2
88	RCL 13	RCL 1 3
89	RCL+ 14	RCL + 1 4
90	STO 13	STO 1 3
91	RCL 13	RCL 1 3
92	RCL 04	RCL 0 4
93	1	1
94	+	+
95	PIXEL	f PGM.FCN ▼ ▼ PIXEL
96	RCL 15	RCL 1 5
97	1	1
98	-	-
99	STO 15	STO 1 5
100	X>0?	f PGM.FCN ▼ X?0 X>0?
101	GTO 22	f GTO 2 2
102	RTN	f PGM.FCN RTN
103	LBL "CPLOT"	f PGM.FCN LBL C P L O T ENTER
104	CLLCD	f CLEAR ▼ CLLCD
105	XEQ "AXES"	XEQ A X E S ENTER
106	0	0
107	STO 04	STO 0 4
108	STO 09	STO 0 9
109	LBL 00	f PGM.FCN LBL 0 0
110	RCL 04	RCL 0 4
111	RCL 01	RCL 0 1
112	RCL- 00	RCL - 0 0
113	×	×
114	399	3 9 9
115	÷	÷
116	RCL+ 00	RCL + 0 0
117	STO 05	STO 0 5
118	XEQ "YFUN"	XEQ Y F U N ENTER
119	STO 06	STO 0 6
120	FS? 00	f FLAGS FS? 0 0
121	GTO 01	f GTO 0 1
122	0	0

Line	Instruction	Exact keys
123	STO 09	STO 0 9
124	GTO 04	f GTO 0 4
125	LBL 01	f PGM.FCN LBL 0 1
126	RCL 06	RCL 0 6
127	RCL- 02	RCL - 0 2
128	X<0?	f PGM.FCN ▼ X?0 X<0?
129	GTO 02	f GTO 0 2
130	RCL 06	RCL 0 6
131	RCL- 03	RCL - 0 3
132	X>0?	f PGM.FCN ▼ X?0 X>0?
133	GTO 02	f GTO 0 2
134	RCL 06	RCL 0 6
135	XEQ "MAPY"	XEQ M A P Y ENTER
136	STO 07	STO 0 7
137	RCL 09	RCL 0 9
138	X=0?	f PGM.FCN ▼ X?0 X=0?
139	GTO 03	f GTO 0 3
140	RCL 07	RCL 0 7
141	RCL- 08	RCL - 0 8
142	ABS	f CONVERT ▼ ABS
143	80	8 0
144	X<Y?	f PGM.FCN ▼ X?Y X<Y?
145	GTO 02	f GTO 0 2
146	XEQ "VJOIN"	XEQ V J O I N ENTER
147	GTO 05	f GTO 0 5
148	LBL 03	f PGM.FCN LBL 0 3
149	RCL 07	RCL 0 7
150	RCL 04	RCL 0 4
151	1	1
152	+	+
153	PIXEL	f PGM.FCN ▼ ▼ PIXEL
154	LBL 05	f PGM.FCN LBL 0 5
155	RCL 07	RCL 0 7
156	STO 08	STO 0 8
157	1	1
158	STO 09	STO 0 9
159	GTO 04	f GTO 0 4
160	LBL 02	f PGM.FCN LBL 0 2
161	0	0
162	STO 09	STO 0 9
163	LBL 04	f PGM.FCN LBL 0 4
164	RCL 04	RCL 0 4
165	1	1
166	+	+
167	STO 04	STO 0 4
168	RCL 04	RCL 0 4
169	400	4 0 0
170	X>Y?	f PGM.FCN ▼ X?Y X>Y?
171	GTO 00	f GTO 0 0
172	RTN	f PGM.FCN RTN
173	LBL "YFUN"	f PGM.FCN LBL Y F U N ENTER
174	SF 00	f FLAGS SF 0 0
175	RCL 05	RCL 0 5
176	RTN	f PGM.FCN RTN
177	LBL "XTICK"	f PGM.FCN LBL X T I C K ENTER
178	RCL 10	RCL 1 0
179	ABS	f CONVERT ▼ ABS
180	STO 10	STO 1 0
181	X=0?	f PGM.FCN ▼ X?0 X=0?
182	RTN	f PGM.FCN RTN
183	0	0
184	RCL- 02	RCL - 0 2
185	X<0?	f PGM.FCN ▼ X?0 X<0?
186	RTN	f PGM.FCN RTN
187	0	0
188	RCL- 03	RCL - 0 3
189	X>0?	f PGM.FCN ▼ X?0 X>0?
190	RTN	f PGM.FCN RTN
191	0	0

Line	Instruction	Exact keys
192	XEQ "MAPY"	XEQ M A P Y ENTER
193	STO 12	STO 1 2
194	0	0
195	STO 16	STO 1 6
196	LBL 30	f PGM.FCN LBL 3 0
197	RCL 16	RCL 1 6
198	RCL- 01	RCL - 0 1
199	X>0?	f PGM.FCN ▼ X?0 X>0?
200	GTO 31	f GTO 3 1
201	XEQ "XMARK"	XEQ X M A R K ENTER
202	RCL 16	RCL 1 6
203	RCL+ 10	RCL + 1 0
204	STO 16	STO 1 6
205	GTO 30	f GTO 3 0
206	LBL 31	f PGM.FCN LBL 3 1
207	RCL 10	RCL 1 0
208	+/-	+/-
209	STO 16	STO 1 6
210	LBL 32	f PGM.FCN LBL 3 2
211	RCL 16	RCL 1 6
212	RCL- 00	RCL - 0 0
213	X<0?	f PGM.FCN ▼ X?0 X<0?
214	GTO 33	f GTO 3 3
215	XEQ "XMARK"	XEQ X M A R K ENTER
216	RCL 16	RCL 1 6
217	RCL- 10	RCL - 1 0
218	STO 16	STO 1 6
219	GTO 32	f GTO 3 2
220	LBL 33	f PGM.FCN LBL 3 3
221	RTN	f PGM.FCN RTN
222	LBL "YTICK"	f PGM.FCN LBL Y T I C K ENTER
223	RCL 11	RCL 1 1
224	ABS	f CONVERT ▼ ABS
225	STO 11	STO 1 1
226	X=0?	f PGM.FCN ▼ X?0 X=0?
227	RTN	f PGM.FCN RTN
228	0	0
229	RCL- 00	RCL - 0 0
230	X<0?	f PGM.FCN ▼ X?0 X<0?
231	RTN	f PGM.FCN RTN
232	0	0
233	RCL- 01	RCL - 0 1
234	X>0?	f PGM.FCN ▼ X?0 X>0?
235	RTN	f PGM.FCN RTN
236	0	0
237	XEQ "MAPX"	XEQ M A P X ENTER
238	STO 12	STO 1 2
239	0	0
240	STO 16	STO 1 6
241	LBL 34	f PGM.FCN LBL 3 4
242	RCL 16	RCL 1 6
243	RCL- 03	RCL - 0 3
244	X>0?	f PGM.FCN ▼ X?0 X>0?
245	GTO 35	f GTO 3 5
246	XEQ "YMARK"	XEQ Y M A R K ENTER
247	RCL 16	RCL 1 6
248	RCL+ 11	RCL + 1 1
249	STO 16	STO 1 6
250	GTO 34	f GTO 3 4
251	LBL 35	f PGM.FCN LBL 3 5
252	RCL 11	RCL 1 1
253	+/-	+/-
254	STO 16	STO 1 6
255	LBL 36	f PGM.FCN LBL 3 6
256	RCL 16	RCL 1 6
257	RCL- 02	RCL - 0 2
258	X<0?	f PGM.FCN ▼ X?0 X<0?
259	GTO 37	f GTO 3 7

Line	Instruction	Exact keys
260	XEQ "YMARK"	XEQ Y M A R K ENTER
261	RCL 16	RCL 1 6
262	RCL- 11	RCL - 1 1
263	STO 16	STO 1 6
264	GTO 36	f GTO 3 6
265	LBL 37	f PGM.FCN LBL 3 7
266	RTN	f PGM.FCN RTN
267	LBL "XMARK"	f PGM.FCN LBL X M A R K ENTER
268	RCL 16	RCL 1 6
269	XEQ "MAPX"	XEQ M A P X ENTER
270	STO 13	STO 1 3
271	CLA	f CLEAR CLA
272	127	1 2 7
273	XTOA	f PGM.FCN ▼ ▼ XTOA
274	RCL 12	RCL 1 2
275	3	3
276	-	-
277	RCL 13	RCL 1 3
278	AGRAPH	f PGM.FCN ▼ ▼ AGRAPH
279	RCL 17	RCL 1 7
280	X=0?	f PGM.FCN ▼ X?0 X=0?
281	RTN	f PGM.FCN RTN
282	RCL 16	RCL 1 6
283	ABS	f CONVERT ▼ ABS
284	10	1 0
285	X>Y?	f PGM.FCN ▼ X?Y X>Y?
286	GTO 38	f GTO 3 8
287	RTN	f PGM.FCN RTN
288	LBL 38	f PGM.FCN LBL 3 8
289	RCL 12	RCL 1 2
290	5	5
291	+	+
292	STO 18	STO 1 8
293	RCL 13	RCL 1 3
294	STO 19	STO 1 9
295	RCL 16	RCL 1 6
296	XEQ "ILAB"	XEQ I L A B ENTER
297	RTN	f PGM.FCN RTN
298	LBL "YMARK"	f PGM.FCN LBL Y M A R K ENTER
299	RCL 16	RCL 1 6
300	XEQ "MAPY"	XEQ M A P Y ENTER
301	STO 13	STO 1 3
302	CLA	f CLEAR CLA
303	1	1
304	XTOA	f PGM.FCN ▼ ▼ XTOA
305	1	1
306	XTOA	f PGM.FCN ▼ ▼ XTOA
307	1	1
308	XTOA	f PGM.FCN ▼ ▼ XTOA
309	1	1
310	XTOA	f PGM.FCN ▼ ▼ XTOA
311	1	1
312	XTOA	f PGM.FCN ▼ ▼ XTOA
313	1	1
314	XTOA	f PGM.FCN ▼ ▼ XTOA
315	1	1
316	XTOA	f PGM.FCN ▼ ▼ XTOA
317	RCL 13	RCL 1 3
318	RCL 12	RCL 1 2
319	3	3
320	-	-
321	AGRAPH	f PGM.FCN ▼ ▼ AGRAPH
322	RCL 17	RCL 1 7
323	X=0?	f PGM.FCN ▼ X?0 X=0?
324	RTN	f PGM.FCN RTN
325	RCL 16	RCL 1 6
326	X=0?	f PGM.FCN ▼ X?0 X=0?
327	RTN	f PGM.FCN RTN
328	RCL 16	RCL 1 6

Line	Instruction	Exact keys
329	ABS	f CONVERT ▼ ABS
330	10	1 0
331	X>Y?	f PGM.FCN ▼ X?Y X>Y?
332	GTO 39	f GTO 3 9
333	RTN	f PGM.FCN RTN
334	LBL 39	f PGM.FCN LBL 3 9
335	RCL 13	RCL 1 3
336	4	4
337	-	-
338	STO 18	STO 1 8
339	RCL 12	RCL 1 2
340	15	1 5
341	+	+
342	STO 19	STO 1 9
343	RCL 16	RCL 1 6
344	XEQ "ILAB"	XEQ I L A B ENTER
345	RTN	f PGM.FCN RTN
346	LBL "ILAB"	f PGM.FCN LBL I L A B ENTER
347	RCL 16	RCL 1 6
348	ENTER	ENTER
349	IP	f CONVERT ▼ IP
350	-	-
351	X=0?	f PGM.FCN ▼ X?0 X=0?
352	GTO 48	f GTO 4 8
353	RTN	f PGM.FCN RTN
354	LBL 48	f PGM.FCN LBL 4 8
355	CLA	f CLEAR CLA
356	RCL 16	RCL 1 6
357	X<0?	f PGM.FCN ▼ X?0 X<0?
358	GTO 50	f GTO 5 0
359	GTO 51	f GTO 5 1
360	LBL 50	f PGM.FCN LBL 5 0
361	8	8
362	XTOA	f PGM.FCN ▼ ▼ XTOA
363	8	8
364	XTOA	f PGM.FCN ▼ ▼ XTOA
365	8	8
366	XTOA	f PGM.FCN ▼ ▼ XTOA
367	8	8
368	XTOA	f PGM.FCN ▼ ▼ XTOA
369	8	8
370	XTOA	f PGM.FCN ▼ ▼ XTOA
371	0	0
372	XTOA	f PGM.FCN ▼ ▼ XTOA
373	LBL 51	f PGM.FCN LBL 5 1
374	RCL 16	RCL 1 6
375	ABS	f CONVERT ▼ ABS
376	STO 14	STO 1 4
377	10	1 0
378	X>Y?	f PGM.FCN ▼ X?Y X>Y?
379	GTO 52	f GTO 5 2
380	RTN	f PGM.FCN RTN
381	LBL 52	f PGM.FCN LBL 5 2
382	RCL 14	RCL 1 4
383	XEQ "DIGIT"	XEQ D I G I T ENTER
384	5	5
385	STO 15	STO 1 5
386	RCL 16	RCL 1 6
387	X<0?	f PGM.FCN ▼ X?0 X<0?
388	GTO 54	f GTO 5 4
389	GTO 55	f GTO 5 5
390	LBL 54	f PGM.FCN LBL 5 4
391	RCL 15	RCL 1 5
392	6	6
393	+	+
394	STO 15	STO 1 5
395	LBL 55	f PGM.FCN LBL 5 5
396	RCL 15	RCL 1 5
397	2	2

Line	Instruction	Exact keys
398	÷	÷
399	RCL 19	RCL 1 9
400	X<>Y	$x \leftrightarrow y$
401	-	-
402	STO 19	STO 1 9
403	RCL 18	RCL 1 8
404	RCL 19	RCL 1 9
405	AGRAPH	f PGM.FCN ▼ ▼ AGRAPH
406	RTN	f PGM.FCN RTN
407	LBL "DIGIT"	f PGM.FCN LBL D I G I T ENTER
408	STO 15	STO 1 5
409	RCL 15	RCL 1 5
410	X=0?	f PGM.FCN ▼ X?0 X=0?
411	GTO 60	f GTO 6 0
412	RCL 15	RCL 1 5
413	1	1
414	X=Y?	f PGM.FCN ▼ X?Y X=Y?
415	GTO 61	f GTO 6 1
416	RCL 15	RCL 1 5
417	2	2
418	X=Y?	f PGM.FCN ▼ X?Y X=Y?
419	GTO 62	f GTO 6 2
420	RCL 15	RCL 1 5
421	3	3
422	X=Y?	f PGM.FCN ▼ X?Y X=Y?
423	GTO 63	f GTO 6 3
424	RCL 15	RCL 1 5
425	4	4
426	X=Y?	f PGM.FCN ▼ X?Y X=Y?
427	GTO 64	f GTO 6 4
428	RCL 15	RCL 1 5
429	5	5
430	X=Y?	f PGM.FCN ▼ X?Y X=Y?
431	GTO 65	f GTO 6 5
432	RCL 15	RCL 1 5
433	6	6
434	X=Y?	f PGM.FCN ▼ X?Y X=Y?
435	GTO 66	f GTO 6 6
436	RCL 15	RCL 1 5
437	7	7
438	X=Y?	f PGM.FCN ▼ X?Y X=Y?
439	GTO 67	f GTO 6 7
440	RCL 15	RCL 1 5
441	8	8
442	X=Y?	f PGM.FCN ▼ X?Y X=Y?
443	GTO 68	f GTO 6 8
444	RCL 15	RCL 1 5
445	9	9
446	X=Y?	f PGM.FCN ▼ X?Y X=Y?
447	GTO 69	f GTO 6 9
448	RTN	f PGM.FCN RTN
449	LBL 60	f PGM.FCN LBL 6 0
450	62	6 2
451	XTOA	f PGM.FCN ▼ ▼ XTOA
452	81	8 1
453	XTOA	f PGM.FCN ▼ ▼ XTOA
454	73	7 3
455	XTOA	f PGM.FCN ▼ ▼ XTOA
456	69	6 9
457	XTOA	f PGM.FCN ▼ ▼ XTOA
458	62	6 2
459	XTOA	f PGM.FCN ▼ ▼ XTOA
460	GTO 70	f GTO 7 0
461	LBL 61	f PGM.FCN LBL 6 1
462	0	0
463	XTOA	f PGM.FCN ▼ ▼ XTOA
464	66	6 6
465	XTOA	f PGM.FCN ▼ ▼ XTOA
466	127	1 2 7

Line	Instruction	Exact keys
467	XTOA	f PGM.FCN ▼ ▼ XTOA
468	64	6 4
469	XTOA	f PGM.FCN ▼ ▼ XTOA
470	0	0
471	XTOA	f PGM.FCN ▼ ▼ XTOA
472	GTO 70	f GTO 7 0
473	LBL 62	f PGM.FCN LBL 6 2
474	66	6 6
475	XTOA	f PGM.FCN ▼ ▼ XTOA
476	97	9 7
477	XTOA	f PGM.FCN ▼ ▼ XTOA
478	81	8 1
479	XTOA	f PGM.FCN ▼ ▼ XTOA
480	73	7 3
481	XTOA	f PGM.FCN ▼ ▼ XTOA
482	70	7 0
483	XTOA	f PGM.FCN ▼ ▼ XTOA
484	GTO 70	f GTO 7 0
485	LBL 63	f PGM.FCN LBL 6 3
486	33	3 3
487	XTOA	f PGM.FCN ▼ ▼ XTOA
488	65	6 5
489	XTOA	f PGM.FCN ▼ ▼ XTOA
490	69	6 9
491	XTOA	f PGM.FCN ▼ ▼ XTOA
492	75	7 5
493	XTOA	f PGM.FCN ▼ ▼ XTOA
494	49	4 9
495	XTOA	f PGM.FCN ▼ ▼ XTOA
496	GTO 70	f GTO 7 0
497	LBL 64	f PGM.FCN LBL 6 4
498	24	2 4
499	XTOA	f PGM.FCN ▼ ▼ XTOA
500	20	2 0
501	XTOA	f PGM.FCN ▼ ▼ XTOA
502	18	1 8
503	XTOA	f PGM.FCN ▼ ▼ XTOA
504	127	1 2 7
505	XTOA	f PGM.FCN ▼ ▼ XTOA
506	16	1 6
507	XTOA	f PGM.FCN ▼ ▼ XTOA
508	GTO 70	f GTO 7 0
509	LBL 65	f PGM.FCN LBL 6 5
510	39	3 9
511	XTOA	f PGM.FCN ▼ ▼ XTOA
512	69	6 9
513	XTOA	f PGM.FCN ▼ ▼ XTOA
514	69	6 9
515	XTOA	f PGM.FCN ▼ ▼ XTOA
516	69	6 9
517	XTOA	f PGM.FCN ▼ ▼ XTOA
518	57	5 7
519	XTOA	f PGM.FCN ▼ ▼ XTOA
520	GTO 70	f GTO 7 0
521	LBL 66	f PGM.FCN LBL 6 6
522	60	6 0
523	XTOA	f PGM.FCN ▼ ▼ XTOA
524	74	7 4
525	XTOA	f PGM.FCN ▼ ▼ XTOA
526	73	7 3
527	XTOA	f PGM.FCN ▼ ▼ XTOA
528	73	7 3
529	XTOA	f PGM.FCN ▼ ▼ XTOA
530	48	4 8
531	XTOA	f PGM.FCN ▼ ▼ XTOA
532	GTO 70	f GTO 7 0
533	LBL 67	f PGM.FCN LBL 6 7
534	1	1
535	XTOA	f PGM.FCN ▼ ▼ XTOA

Line	Instruction	Exact keys
536	113	1 1 3
537	XTOA	f PGM.FCN ▼ ▼ XTOA
538	9	9
539	XTOA	f PGM.FCN ▼ ▼ XTOA
540	5	5
541	XTOA	f PGM.FCN ▼ ▼ XTOA
542	3	3
543	XTOA	f PGM.FCN ▼ ▼ XTOA
544	GTO 70	f GTO 7 0
545	LBL 68	f PGM.FCN LBL 6 8
546	54	5 4
547	XTOA	f PGM.FCN ▼ ▼ XTOA
548	73	7 3
549	XTOA	f PGM.FCN ▼ ▼ XTOA
550	73	7 3
551	XTOA	f PGM.FCN ▼ ▼ XTOA
552	73	7 3
553	XTOA	f PGM.FCN ▼ ▼ XTOA
554	54	5 4
555	XTOA	f PGM.FCN ▼ ▼ XTOA
556	GTO 70	f GTO 7 0
557	LBL 69	f PGM.FCN LBL 6 9
558	6	6
559	XTOA	f PGM.FCN ▼ ▼ XTOA
560	73	7 3
561	XTOA	f PGM.FCN ▼ ▼ XTOA
562	73	7 3
563	XTOA	f PGM.FCN ▼ ▼ XTOA
564	41	4 1
565	XTOA	f PGM.FCN ▼ ▼ XTOA
566	30	3 0
567	XTOA	f PGM.FCN ▼ ▼ XTOA
568	LBL 70	f PGM.FCN LBL 7 0
569	RTN	f PGM.FCN RTN
570	END	Already present at the end of a new program area.

B.3 Parametric toolkit

This program depends on MAPX, MAPY and AXES from the Cartesian toolkit. R20–R22 define the parameter interval and step count; R23–R30 hold parameter, world and screen state; R31 is the valid previous-point flag; R32–R39 are SEG work registers.

```
00 { 322-Byte Prgm }
01 LBL "SEG"
02 RCL 27
03 RCL- 29
04 STO 32
05 RCL 28
06 RCL- 30
07 STO 33
08 RCL 32
09 ABS
10 STO 34
11 RCL 33
12 ABS
13 RCL 34
14 X>Y?
15 GTO 31
16 X<>Y
17 STO 34
18 LBL 31
19 RCL 34
20 X=0?
21 GTO 35
22 RCL 32
23 RCL÷ 34
24 STO 35
25 RCL 33
26 RCL÷ 34
```

```
27 STO 36
28 RCL 29
29 STO 37
30 RCL 30
31 STO 38
32 0
33 STO 39
34 LBL 32
35 RCL 37
36 RCL+ 35
37 STO 37
38 RCL 38
39 RCL+ 36
40 STO 38
41 RCL 38
42 RCL 37
43 PIXEL
44 RCL 39
45 1
46 +
47 STO 39
48 RCL 39
49 RCL 34
50 X>Y?
51 GTO 32
52 RTN
53 LBL 35
54 RCL 28
55 RCL 27
56 PIXEL
57 RTN
58 LBL "PPLOT"
59 CLLCD
60 XEQ "AXES"
61 0
62 STO 23
63 STO 31
64 LBL 40
65 RCL 23
66 RCL 22
67 ÷
68 RCL 21
69 RCL- 20
70 ×
71 RCL+ 20
72 STO 24
73 XEQ "XFUN"
74 STO 25
75 XEQ "YFUNP"
76 STO 26
77 RCL 25
78 RCL- 00
79 X<0?
80 GTO 44
81 RCL 25
82 RCL- 01
83 X>0?
84 GTO 44
85 RCL 26
86 RCL- 02
87 X<0?
88 GTO 44
89 RCL 26
90 RCL- 03
91 X>0?
92 GTO 44
93 RCL 25
94 XEQ "MAPX"
95 STO 27
96 RCL 26
97 XEQ "MAPY"
98 STO 28
99 RCL 31
100 X=0?
101 GTO 42
102 XEQ "SEG"
103 GTO 43
104 LBL 42
105 RCL 28
106 RCL 27
```

```
107 PIXEL
108 LBL 43
109 RCL 27
110 STO 29
111 RCL 28
112 STO 30
113 1
114 STO 31
115 GTO 45
116 LBL 44
117 0
118 STO 31
119 LBL 45
120 RCL 23
121 1
122 +
123 STO 23
124 RCL 23
125 RCL 22
126 ≥XY?
127 GTO 40
128 RTN
129 LBL "XFUN"
130 RCL 24
131 COS
132 RTN
133 LBL "YFUNP"
134 RCL 24
135 SIN
136 RTN
137 END
```

Parametric: complete entry table

Line	Instruction	Exact keys
01	LBL "SEG"	f PGM.FCN LBL S E G ENTER
02	RCL 27	RCL 2 7
03	RCL- 29	RCL - 2 9
04	STO 32	STO 3 2
05	RCL 28	RCL 2 8
06	RCL- 30	RCL - 3 0
07	STO 33	STO 3 3
08	RCL 32	RCL 3 2
09	ABS	f CONVERT ▼ ABS
10	STO 34	STO 3 4
11	RCL 33	RCL 3 3
12	ABS	f CONVERT ▼ ABS
13	RCL 34	RCL 3 4
14	X>Y?	f PGM.FCN ▼ X?Y X>Y?
15	GTO 31	f GTO 3 1
16	X<>Y	$x \leftrightarrow y$
17	STO 34	STO 3 4
18	LBL 31	f PGM.FCN LBL 3 1
19	RCL 34	RCL 3 4
20	X=0?	f PGM.FCN ▼ X?0 X=0?
21	GTO 35	f GTO 3 5
22	RCL 32	RCL 3 2
23	RCL÷ 34	RCL ÷ 3 4
24	STO 35	STO 3 5
25	RCL 33	RCL 3 3
26	RCL÷ 34	RCL ÷ 3 4
27	STO 36	STO 3 6
28	RCL 29	RCL 2 9
29	STO 37	STO 3 7
30	RCL 30	RCL 3 0
31	STO 38	STO 3 8
32	0	0
33	STO 39	STO 3 9
34	LBL 32	f PGM.FCN LBL 3 2

Line	Instruction	Exact keys
35	RCL 37	RCL 3 7
36	RCL+ 35	RCL + 3 5
37	STO 37	STO 3 7
38	RCL 38	RCL 3 8
39	RCL+ 36	RCL + 3 6
40	STO 38	STO 3 8
41	RCL 38	RCL 3 8
42	RCL 37	RCL 3 7
43	PIXEL	f PGM.FCN ▼ ▼ PIXEL
44	RCL 39	RCL 3 9
45	1	1
46	+	+
47	STO 39	STO 3 9
48	RCL 39	RCL 3 9
49	RCL 34	RCL 3 4
50	X>Y?	f PGM.FCN ▼ X?Y X>Y?
51	GTO 32	f GTO 3 2
52	RTN	f PGM.FCN RTN
53	LBL 35	f PGM.FCN LBL 3 5
54	RCL 28	RCL 2 8
55	RCL 27	RCL 2 7
56	PIXEL	f PGM.FCN ▼ ▼ PIXEL
57	RTN	f PGM.FCN RTN
58	LBL "PPLOT"	f PGM.FCN LBL P P L O T ENTER
59	CLLCD	f CLEAR ▼ CLLCD
60	XEQ "AXES"	XEQ A X E S ENTER
61	0	0
62	STO 23	STO 2 3
63	STO 31	STO 3 1
64	LBL 40	f PGM.FCN LBL 4 0
65	RCL 23	RCL 2 3
66	RCL 22	RCL 2 2
67	÷	÷
68	RCL 21	RCL 2 1
69	RCL- 20	RCL - 2 0
70	×	×
71	RCL+ 20	RCL + 2 0
72	STO 24	STO 2 4
73	XEQ "XFUN"	XEQ X F U N ENTER
74	STO 25	STO 2 5
75	XEQ "YFUNP"	XEQ Y F U N P ENTER
76	STO 26	STO 2 6
77	RCL 25	RCL 2 5
78	RCL- 00	RCL - 0 0
79	X<0?	f PGM.FCN ▼ X?0 X<0?
80	GTO 44	f GTO 4 4
81	RCL 25	RCL 2 5
82	RCL- 01	RCL - 0 1
83	X>0?	f PGM.FCN ▼ X?0 X>0?
84	GTO 44	f GTO 4 4
85	RCL 26	RCL 2 6
86	RCL- 02	RCL - 0 2
87	X<0?	f PGM.FCN ▼ X?0 X<0?
88	GTO 44	f GTO 4 4
89	RCL 26	RCL 2 6
90	RCL- 03	RCL - 0 3
91	X>0?	f PGM.FCN ▼ X?0 X>0?
92	GTO 44	f GTO 4 4
93	RCL 25	RCL 2 5
94	XEQ "MAPX"	XEQ M A P X ENTER
95	STO 27	STO 2 7
96	RCL 26	RCL 2 6
97	XEQ "MAPY"	XEQ M A P Y ENTER
98	STO 28	STO 2 8
99	RCL 31	RCL 3 1
100	X=0?	f PGM.FCN ▼ X?0 X=0?
101	GTO 42	f GTO 4 2
102	XEQ "SEG"	XEQ S E G ENTER
103	GTO 43	f GTO 4 3

Line	Instruction	Exact keys
104	LBL 42	f PGM.FCN LBL 4 2
105	RCL 28	RCL 2 8
106	RCL 27	RCL 2 7
107	PIXEL	f PGM.FCN ▼ ▼ PIXEL
108	LBL 43	f PGM.FCN LBL 4 3
109	RCL 27	RCL 2 7
110	STO 29	STO 2 9
111	RCL 28	RCL 2 8
112	STO 30	STO 3 0
113	1	1
114	STO 31	STO 3 1
115	GTO 45	f GTO 4 5
116	LBL 44	f PGM.FCN LBL 4 4
117	0	0
118	STO 31	STO 3 1
119	LBL 45	f PGM.FCN LBL 4 5
120	RCL 23	RCL 2 3
121	1	1
122	+	+
123	STO 23	STO 2 3
124	RCL 23	RCL 2 3
125	RCL 22	RCL 2 2
126	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
127	GTO 40	f GTO 4 0
128	RTN	f PGM.FCN RTN
129	LBL "XFUN"	f PGM.FCN LBL X F U N ENTER
130	RCL 24	RCL 2 4
131	COS	COS
132	RTN	f PGM.FCN RTN
133	LBL "YFUNP"	f PGM.FCN LBL Y F U N P ENTER
134	RCL 24	RCL 2 4
135	SIN	SIN
136	RTN	f PGM.FCN RTN
137	END	Already present at the end of a new program area.

B.4 Threshold contour toolkit

R48 is the contour level C; R49 is a positive tolerance. The program uses `AXES` from the Cartesian toolkit, then samples every second row and column for a practical preview. Replace `FFUN`; leave `ICONT` unchanged.

```
00 { 181-Byte Prgm }
01 LBL "ICONT"
02 0
03 STO "RefLCD"
04 CLLCD
05 XEQ "AXES"
06 0
07 STO 40
08 LBL 60
09 0
10 STO 41
11 LBL 61
12 RCL 40
13 399
14 ÷
15 RCL 01
16 RCL- 00
17 ×
18 RCL+ 00
19 STO 42
20 RCL 41
21 239
22 ÷
23 RCL 03
24 RCL- 02
25 ×
26 RCL 03
27 X<>Y
```

```
28 -
29 STO 43
30 XEQ "FFUN"
31 RCL- 48
32 ABS
33 RCL 49
34 ≥XY?
35 GTO 62
36 GTO 63
37 LBL 62
38 RCL 41
39 1
40 +
41 RCL 40
42 1
43 +
44 PIXEL
45 LBL 63
46 RCL 41
47 2
48 +
49 STO 41
50 RCL 41
51 240
52 X>Y?
53 GTO 61
54 RCL 40
55 2
56 +
57 STO 40
58 RCL 40
59 400
60 X>Y?
61 GTO 60
62 -1
63 STO "RefLCD"
64 1
65 STO "RefLCD"
66 RTN
67 LBL "FFUN"
68 RCL 42
69 ↑X2
70 RCL 43
71 ↑X2
72 +
73 RTN
74 END
```

Contour: complete entry table

Line	Instruction	Exact keys
01	LBL "ICONT"	f PGM.FCN LBL I C O N T ENTER
02	0	0
03	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
04	CLLCD	f CLEAR ▼ CLLCD
05	XEQ "AXES"	XEQ A X E S ENTER
06	0	0
07	STO 40	STO 4 0
08	LBL 60	f PGM.FCN LBL 6 0
09	0	0
10	STO 41	STO 4 1
11	LBL 61	f PGM.FCN LBL 6 1
12	RCL 40	RCL 4 0
13	399	3 9 9
14	÷	÷
15	RCL 01	RCL 0 1
16	RCL- 00	RCL - 0 0
17	×	×
18	RCL+ 00	RCL + 0 0
19	STO 42	STO 4 2
20	RCL 41	RCL 4 1
21	239	2 3 9

Line	Instruction	Exact keys
22	÷	÷
23	RCL 03	RCL 0 3
24	RCL- 02	RCL - 0 2
25	×	×
26	RCL 03	RCL 0 3
27	X<>Y	$x \leftrightarrow y$
28	-	-
29	STO 43	STO 4 3
30	XEQ "FFUN"	XEQ F F U N ENTER
31	RCL- 48	RCL - 4 8
32	ABS	f CONVERT ▼ ABS
33	RCL 49	RCL 4 9
34	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
35	GTO 62	f GTO 6 2
36	GTO 63	f GTO 6 3
37	LBL 62	f PGM.FCN LBL 6 2
38	RCL 41	RCL 4 1
39	1	1
40	+	+
41	RCL 40	RCL 4 0
42	1	1
43	+	+
44	PIXEL	f PGM.FCN ▼ ▼ PIXEL
45	LBL 63	f PGM.FCN LBL 6 3
46	RCL 41	RCL 4 1
47	2	2
48	+	+
49	STO 41	STO 4 1
50	RCL 41	RCL 4 1
51	240	2 4 0
52	X>Y?	f PGM.FCN ▼ X?Y X>Y?
53	GTO 61	f GTO 6 1
54	RCL 40	RCL 4 0
55	2	2
56	+	+
57	STO 40	STO 4 0
58	RCL 40	RCL 4 0
59	400	4 0 0
60	X>Y?	f PGM.FCN ▼ X?Y X>Y?
61	GTO 60	f GTO 6 0
62	-1	1 +/-
63	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
64	1	1
65	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
66	RTN	f PGM.FCN RTN
67	LBL "FFUN"	f PGM.FCN LBL F F U N ENTER
68	RCL 42	RCL 4 2
69	X↑2	f x^2
70	RCL 43	RCL 4 3
71	X↑2	f x^2
72	+	+
73	RTN	f PGM.FCN RTN
74	END	Already present at the end of a new program area.

B.5 3-D projection and wireframe toolkit

This program depends on SEG. Store yaw, pitch, scale and screen centre in R53–R57, then replace ZFUN as needed and run GRID3D.

```
00 { 397-Byte Prgm }
01 LBL "PROJ"
02 RCL 50
03 RCL 53
04 COS
05 ×
06 RCL 51
07 RCL 53
```

```
08 SIN
09 ×
10 -
11 STO 58
12 RCL 50
13 RCL 53
14 SIN
15 ×
16 RCL 51
17 RCL 53
18 COS
19 ×
20 +
21 STO 59
22 RCL 59
23 RCL 54
24 COS
25 ×
26 RCL 52
27 RCL 54
28 SIN
29 ×
30 -
31 STO 60
32 RCL 58
33 RCL× 55
34 RCL+ 56
35 STO 27
36 RCL 60
37 RCL× 55
38 +/-
39 RCL+ 57
40 STO 28
41 RTN
42 LBL "ROW3D"
43 0
44 STO 61
45 STO 31
46 LBL 70
47 RCL 61
48 12
49 ÷
50 4
51 ×
52 2
53 -
54 STO 50
55 XEQ "ZFUN"
56 STO 52
57 XEQ "PROJ"
58 RCL 31
59 X=0?
60 GTO 71
61 XEQ "SEG"
62 GTO 72
63 LBL 71
64 RCL 28
65 RCL 27
66 PIXEL
67 LBL 72
68 RCL 27
69 STO 29
70 RCL 28
71 STO 30
72 1
73 STO 31
74 RCL 61
75 1
76 +
77 STO 61
78 RCL 61
79 12
80 ≥XY?
81 GTO 70
82 RTN
83 LBL "COL3D"
84 0
85 STO 62
86 STO 31
87 LBL 73
```

```
88 RCL 62
89 12
90 ÷
91 4
92 ×
93 2
94 -
95 STO 51
96 XEQ "ZFUN"
97 STO 52
98 XEQ "PROJ"
99 RCL 31
100 X=0?
101 GTO 74
102 XEQ "SEG"
103 GTO 75
104 LBL 74
105 RCL 28
106 RCL 27
107 PIXEL
108 LBL 75
109 RCL 27
110 STO 29
111 RCL 28
112 STO 30
113 1
114 STO 31
115 RCL 62
116 1
117 +
118 STO 62
119 RCL 62
120 12
121 ≥XY?
122 GTO 73
123 RTN
124 LBL "GRID3D"
125 CLLCD
126 0
127 STO 63
128 LBL 76
129 RCL 63
130 12
131 ÷
132 4
133 ×
134 2
135 -
136 STO 51
137 XEQ "ROW3D"
138 RCL 63
139 1
140 +
141 STO 63
142 RCL 63
143 12
144 ≥XY?
145 GTO 76
146 0
147 STO 63
148 LBL 77
149 RCL 63
150 12
151 ÷
152 4
153 ×
154 2
155 -
156 STO 50
157 XEQ "COL3D"
158 RCL 63
159 1
160 +
161 STO 63
162 RCL 63
163 12
164 ≥XY?
165 GTO 77
166 RTN
167 LBL "ZFUN"
```

```
168 RCL 50
169 .5
170 ×
171 RCL 51
172 .35
173 ×
174 -
175 RTN
176 END
```

Threed: complete entry table

Line	Instruction	Exact keys
01	LBL "PROJ"	f PGM.FCN LBL P R O J ENTER
02	RCL 50	RCL 5 0
03	RCL 53	RCL 5 3
04	COS	COS
05	×	×
06	RCL 51	RCL 5 1
07	RCL 53	RCL 5 3
08	SIN	SIN
09	×	×
10	-	-
11	STO 58	STO 5 8
12	RCL 50	RCL 5 0
13	RCL 53	RCL 5 3
14	SIN	SIN
15	×	×
16	RCL 51	RCL 5 1
17	RCL 53	RCL 5 3
18	COS	COS
19	×	×
20	+	+
21	STO 59	STO 5 9
22	RCL 59	RCL 5 9
23	RCL 54	RCL 5 4
24	COS	COS
25	×	×
26	RCL 52	RCL 5 2
27	RCL 54	RCL 5 4
28	SIN	SIN
29	×	×
30	-	-
31	STO 60	STO 6 0
32	RCL 58	RCL 5 8
33	RCL× 55	RCL × 5 5
34	RCL+ 56	RCL + 5 6
35	STO 27	STO 2 7
36	RCL 60	RCL 6 0
37	RCL× 55	RCL × 5 5
38	+/-	+/-
39	RCL+ 57	RCL + 5 7
40	STO 28	STO 2 8
41	RTN	f PGM.FCN RTN
42	LBL "ROW3D"	f PGM.FCN LBL R O W 3 D ENTER
43	0	0
44	STO 61	STO 6 1
45	STO 31	STO 3 1
46	LBL 70	f PGM.FCN LBL 7 0
47	RCL 61	RCL 6 1
48	12	1 2
49	÷	÷
50	4	4
51	×	×
52	2	2
53	-	-
54	STO 50	STO 5 0
55	XEQ "ZFUN"	XEQ Z F U N ENTER

Line	Instruction	Exact keys
56	STO 52	STO 5 2
57	XEQ "PROJ"	XEQ P R O J ENTER
58	RCL 31	RCL 3 1
59	X=0?	f PGM.FCN ▼ X?0 X=0?
60	GTO 71	f GTO 7 1
61	XEQ "SEG"	XEQ S E G ENTER
62	GTO 72	f GTO 7 2
63	LBL 71	f PGM.FCN LBL 7 1
64	RCL 28	RCL 2 8
65	RCL 27	RCL 2 7
66	PIXEL	f PGM.FCN ▼ ▼ PIXEL
67	LBL 72	f PGM.FCN LBL 7 2
68	RCL 27	RCL 2 7
69	STO 29	STO 2 9
70	RCL 28	RCL 2 8
71	STO 30	STO 3 0
72	1	1
73	STO 31	STO 3 1
74	RCL 61	RCL 6 1
75	1	1
76	+	+
77	STO 61	STO 6 1
78	RCL 61	RCL 6 1
79	12	1 2
80	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
81	GTO 70	f GTO 7 0
82	RTN	f PGM.FCN RTN
83	LBL "COL3D"	f PGM.FCN LBL C O L 3 D ENTER
84	0	0
85	STO 62	STO 6 2
86	STO 31	STO 3 1
87	LBL 73	f PGM.FCN LBL 7 3
88	RCL 62	RCL 6 2
89	12	1 2
90	÷	÷
91	4	4
92	×	×
93	2	2
94	-	-
95	STO 51	STO 5 1
96	XEQ "ZFUN"	XEQ Z F U N ENTER
97	STO 52	STO 5 2
98	XEQ "PROJ"	XEQ P R O J ENTER
99	RCL 31	RCL 3 1
100	X=0?	f PGM.FCN ▼ X?0 X=0?
101	GTO 74	f GTO 7 4
102	XEQ "SEG"	XEQ S E G ENTER
103	GTO 75	f GTO 7 5
104	LBL 74	f PGM.FCN LBL 7 4
105	RCL 28	RCL 2 8
106	RCL 27	RCL 2 7
107	PIXEL	f PGM.FCN ▼ ▼ PIXEL
108	LBL 75	f PGM.FCN LBL 7 5
109	RCL 27	RCL 2 7
110	STO 29	STO 2 9
111	RCL 28	RCL 2 8
112	STO 30	STO 3 0
113	1	1
114	STO 31	STO 3 1
115	RCL 62	RCL 6 2
116	1	1
117	+	+
118	STO 62	STO 6 2
119	RCL 62	RCL 6 2
120	12	1 2
121	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
122	GTO 73	f GTO 7 3
123	RTN	f PGM.FCN RTN
124	LBL "GRID3D"	f PGM.FCN LBL G R I D 3 D ENTER

Line	Instruction	Exact keys
125	CLLCD	f CLEAR ▼ CLLCD
126	0	0
127	STO 63	STO 6 3
128	LBL 76	f PGM.FCN LBL 7 6
129	RCL 63	RCL 6 3
130	12	1 2
131	÷	÷
132	4	4
133	×	×
134	2	2
135	-	-
136	STO 51	STO 5 1
137	XEQ "ROW3D"	XEQ R O W 3 D ENTER
138	RCL 63	RCL 6 3
139	1	1
140	+	+
141	STO 63	STO 6 3
142	RCL 63	RCL 6 3
143	12	1 2
144	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
145	GTO 76	f GTO 7 6
146	0	0
147	STO 63	STO 6 3
148	LBL 77	f PGM.FCN LBL 7 7
149	RCL 63	RCL 6 3
150	12	1 2
151	÷	÷
152	4	4
153	×	×
154	2	2
155	-	-
156	STO 50	STO 5 0
157	XEQ "COL3D"	XEQ C O L 3 D ENTER
158	RCL 63	RCL 6 3
159	1	1
160	+	+
161	STO 63	STO 6 3
162	RCL 63	RCL 6 3
163	12	1 2
164	X≥Y?	f PGM.FCN ▼ X?Y X≥Y?
165	GTO 77	f GTO 7 7
166	RTN	f PGM.FCN RTN
167	LBL "ZFUN"	f PGM.FCN LBL Z F U N ENTER
168	RCL 50	RCL 5 0
169	.5	· 5
170	×	×
171	RCL 51	RCL 5 1
172	.35	· 3 5
173	×	×
174	-	-
175	RTN	f PGM.FCN RTN
176	END	Already present at the end of a new program area.

B.6 Mandelbrot renderer

Set `GrMod` to 3 before running this self-contained program. It draws $[-2.35, 0.85] \times [-1.2, 1.2]$ with a 48-iteration limit.

```
00 { 211-Byte Prgm }
01 LBL "MANDEL"
02 0
03 STO "RefLCD"
04 CLLCD
05 0
06 STO 40
07 LBL 60
08 0
09 STO 41
```

```
10 LBL 61
11 RCL 40
12 399
13 ÷
14 3.2
15 ×
16 2.35
17 -
18 STO 42
19 1.2
20 RCL 41
21 239
22 ÷
23 2.4
24 ×
25 -
26 STO 43
27 0
28 STO 44
29 STO 45
30 STO 46
31 LBL 62
32 RCL 44
33 ↑X2
34 RCL 45
35 ↑X2
36 -
37 RCL+ 42
38 STO 47
39 RCL 44
40 RCL× 45
41 2
42 ×
43 RCL+ 43
44 STO 45
45 RCL 47
46 STO 44
47 RCL 46
48 1
49 +
50 STO 46
51 RCL 44
52 ↑X2
53 RCL 45
54 ↑X2
55 +
56 4
57 X<Y?
58 GTO 63
59 RCL 46
60 48
61 X>Y?
62 GTO 62
63 RCL 41
64 1
65 +
66 RCL 40
67 1
68 +
69 PIXEL
70 LBL 63
71 RCL 41
72 1
73 +
74 STO 41
75 RCL 41
76 240
77 X>Y?
78 GTO 61
79 RCL 40
80 1
81 +
82 STO 40
83 RCL 40
84 400
85 X>Y?
86 GTO 60
87 -1
88 STO "RefLCD"
89 1
```

```
90 STO "RefLCD"
91 RTN
92 END
```

Fractal: complete entry table

Line	Instruction	Exact keys
01	LBL "MANDEL"	f PGM.FCN LBL M A N D E L ENTER
02	0	0
03	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
04	CLLCD	f CLEAR ▼ CLLCD
05	0	0
06	STO 40	STO 4 0
07	LBL 60	f PGM.FCN LBL 6 0
08	0	0
09	STO 41	STO 4 1
10	LBL 61	f PGM.FCN LBL 6 1
11	RCL 40	RCL 4 0
12	399	3 9 9
13	÷	÷
14	3.2	3 · 2
15	×	×
16	2.35	2 · 3 5
17	-	-
18	STO 42	STO 4 2
19	1.2	1 · 2
20	RCL 41	RCL 4 1
21	239	2 3 9
22	÷	÷
23	2.4	2 · 4
24	×	×
25	-	-
26	STO 43	STO 4 3
27	0	0
28	STO 44	STO 4 4
29	STO 45	STO 4 5
30	STO 46	STO 4 6
31	LBL 62	f PGM.FCN LBL 6 2
32	RCL 44	RCL 4 4
33	X↑2	f x^2
34	RCL 45	RCL 4 5
35	X↑2	f x^2
36	-	-
37	RCL+ 42	RCL + 4 2
38	STO 47	STO 4 7
39	RCL 44	RCL 4 4
40	RCL× 45	RCL × 4 5
41	2	2
42	×	×
43	RCL+ 43	RCL + 4 3
44	STO 45	STO 4 5
45	RCL 47	RCL 4 7
46	STO 44	STO 4 4
47	RCL 46	RCL 4 6
48	1	1
49	+	+
50	STO 46	STO 4 6
51	RCL 44	RCL 4 4
52	X↑2	f x^2
53	RCL 45	RCL 4 5
54	X↑2	f x^2
55	+	+
56	4	4
57	X<Y?	f PGM.FCN ▼ X?Y X<Y?
58	GTO 63	f GTO 6 3
59	RCL 46	RCL 4 6

Line	Instruction	Exact keys
60	48	4 8
61	X>Y?	f PGM.FCN ▼ X?Y X>Y?
62	GTO 62	f GTO 6 2
63	RCL 41	RCL 4 1
64	1	1
65	+	+
66	RCL 40	RCL 4 0
67	1	1
68	+	+
69	PIXEL	f PGM.FCN ▼ ▼ PIXEL
70	LBL 63	f PGM.FCN LBL 6 3
71	RCL 41	RCL 4 1
72	1	1
73	+	+
74	STO 41	STO 4 1
75	RCL 41	RCL 4 1
76	240	2 4 0
77	X>Y?	f PGM.FCN ▼ X?Y X>Y?
78	GTO 61	f GTO 6 1
79	RCL 40	RCL 4 0
80	1	1
81	+	+
82	STO 40	STO 4 0
83	RCL 40	RCL 4 0
84	400	4 0 0
85	X>Y?	f PGM.FCN ▼ X?Y X>Y?
86	GTO 60	f GTO 6 0
87	-1	1 +/-
88	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
89	1	1
90	STO "RefLCD"	STO ENTER *set [A]* R f ▼ e f f ▲ L C D ENTER
91	RTN	f PGM.FCN RTN
92	END	Already present at the end of a new program area.

CHAPTER C

Bitmap labels with AGRAPH

AGRAPH treats each byte in Alpha as one vertical column of eight bits. Bit 0 is the top pixel of the column. A compact five-column font is therefore just a table of five integers.

Glyph	Column bytes	Glyph	Column bytes
0	62,81,73,69,62	1	0,66,127,64,0
2	66,97,81,73,70	3	33,65,69,75,49
4	24,20,18,127,16	5	39,69,69,69,57
6	60,74,73,73,48	7	1,113,9,5,3
8	54,73,73,73,54	9	6,73,73,41,30
minus	8,8,8,8,8	point	0,96,96,0,0
X	99,20,8,20,99	Y	3,4,120,4,3

To draw X, clear Alpha, enter 99 and execute XTOA; repeat for 20, 8, 20 and 99. Put the target row in Y and column in X, then execute AGRAPH. Clear flags 34 and 35 for additive drawing. The Cartesian toolkit's DIGIT and ILAB routines use the same table for optional tick values. They intentionally label only integer values from −9 through 9; all other ticks remain unlabelled marks.

```
CLA
99
XTOA
20
XTOA
8
XTOA
20
XTOA
99
XTOA
120
ENTER
380
AGRAPH
```

Enter CLA from the first page of the CLEAR menu and XTOA/AGRAPH from page 3 of PGM.FCN. Fixed labels are intentionally simple. A general proportional font engine would distract from plotting.

CHAPTER D

Quick reference

D.1 Modes and dimensions

GrMod	Width	Height	Use
0	131	16	HP-42S compatibility
2	200	120	fast preview
3	400	240	full-resolution default

D.2 Coordinate formulae

For one-based screen coordinates:

$$x_s = 1 + \frac{x - x_{\min}}{x_{\max} - x_{\min}}(W - 1), \quad y_s = 1 + \frac{y_{\max} - y}{y_{\max} - y_{\min}}(H - 1).$$

For screen to world:

$$x = x_{\min} + \frac{x_s - 1}{W - 1}(x_{\max} - x_{\min}), \quad y = y_{\max} - \frac{y_s - 1}{H - 1}(y_{\max} - y_{\min}).$$

D.3 Common windows

Purpose	X range	Y range
General algebra	$[-5, 5]$	$[-5, 5]$
Equal physical scale	$[-5, 5]$	approximately $[-3, 3]$
Trig in radians	$[-2\pi, 2\pi]$	$[-1.5, 1.5]$
Unit parametric curves	$[-1.7, 1.7]$	$[-1.02, 1.02]$
Mandelbrot	$[-2.35, .85]$	$[-1.2, 1.2]$

D.4 Tick controls

Register	Effect
R10	Positive X-axis tick interval; zero disables X ticks.
R11	Positive Y-axis tick interval; zero disables Y ticks.
R17	Zero for marks only; non-zero for supported integer values.

Tick intervals are world-coordinate distances. The routine draws a family only when the corresponding zero axis is inside the window.

D.5 Domain checklist

- Square root: radicand ≥ 0.
- Logarithm: argument > 0.
- Division: denominator not zero.
- Inverse trig: real-domain restrictions where applicable.
- Implicit branch: branch radicand ≥ 0.
- Sinc: replace the removable $0/0$ by its limiting value.

D.6 Discontinuity checklist

- Domain validity does not guarantee a valid connecting segment.
- Break after every invalid or clipped sample.
- Break across a suspiciously large pixel jump.
- Inspect known asymptotes at increased sample density.
- A jump heuristic may also break a legitimate steep segment.

D.7 Troubleshooting

Symptom	Likely cause and test
Upside-down graph	Y mapping lacks the reversal $y_{\max} - y$.
Blank screen	Wrong mode, window, domain, validity flag or refresh left disabled.
Circle looks oval	World units do not have equal pixel scale.
Bridge across asymptote	Clipping without a segment-break test.
Wavy result differs from expectation	Wrong angle mode or aliasing.
Missing edge column/row	Zero-based value sent directly to one-based `PIXEL`.
Jagged steep line	Samples were not joined or are too sparse.
Slow fractal	Preview resolution, iteration cap, repeated work and refresh policy.

CHAPTER E

Glossary

Aliasing A false pattern caused when sampling is too sparse to represent the variation present.

Attractor A set toward which a dynamical system's state tends after transients.

Clipping Rejecting or trimming geometry outside the visible viewport.

Contour A curve joining points where a scalar function has one constant value.

Escape time The number of iterations required for an orbit to pass a defined bound.

Fractal A structure whose detail is generated by repeated rules, often across many scales.

Framebuffer The stored on/off pixel image that will be shown on the display.

Implicit curve A curve specified by a relation such as $F(x, y) = C$, rather than one Y for each X.

Iteration Repeatedly applying a rule to the result of the previous step.

Parametric plot A curve whose X and Y coordinates are separate functions of a parameter.

Phase portrait A plot of system state variables, such as position against velocity.

Pixel One addressable monochrome picture element.

Polar plot A curve expressed by radius and angle, then converted to Cartesian coordinates.

Projection A rule converting 3-D coordinates into 2-D screen coordinates.

Sampling Evaluating a continuous-looking object at a finite set of points.

Screen coordinate A pixel address, one-based in the DM42 graphics commands.

Slope field Short segments showing the derivative specified at sampled points.

Streamline A curve tangent everywhere to a vector field.

Vector field A vector assigned to each point in a region.

Viewport The selected mathematical rectangle represented by the screen.

Wireframe A surface represented by connected mesh lines instead of filled polygons.

World coordinate A mathematical coordinate before it is mapped to a screen pixel.

CHAPTER F

Complete plot catalogue

Each displayed plot identifies a complete program or the reusable routine and replacement formula needed to reproduce it.

No.	Plot	Equation or rule	Type	Required routine
01	Linear function	$y = mx + c$	Cartesian	`CPLOT/YFUN`
02	Quadratic	$y = ax^2 + bx + c$	Cartesian	`CPLOT/YFUN`
03	Cubic	$y = x^3 - 3x$	Cartesian	`CPLOT/YFUN`
04	Fifth-order polynomial	$y = x^5 - 5x^3 + 4x$	Cartesian	`CPLOT/YFUN`
05	Absolute value	$y = \lvert x \rvert$	Cartesian	`CPLOT/YFUN`
06	Reciprocal	$y = 1/x$	Cartesian	`CPLOT/domain+jump`
07	Square root	$y = \sqrt{x}$	Cartesian	`CPLOT/domain`
08	Exponential	$y = e^x$	Cartesian	`CPLOT/YFUN`
09	Logarithm	$y = \ln x$	Cartesian	`CPLOT/domain`
10	Sine	$y = \sin x$	Cartesian	`CPLOT/YFUN`
11	Sine and cosine	$\sin x,\ \cos x$	multiple curves	`CPLOT twice`
12	Tangent	$y = \tan x$	Cartesian	`CPLOT/jump`
13	Gaussian	$y = e^{-x^2}$	Cartesian	`CPLOT/YFUN`
14	Logistic sigmoid	$y = (1 + e^{-x})^{-1}$	Cartesian	`CPLOT/YFUN`
15	Damped sine	$e^{-0.2x} \sin 5x$	Cartesian	`CPLOT/YFUN`
16	Fourier approximation	$\frac{4}{\pi} \sum \sin(kx)/k$	Cartesian sum	`CPLOT/YFUN`
17	Circle	$(\cos t, \sin t)$	parametric	`PPLOT`
18	Ellipse	$(a \cos t, b \sin t)$	parametric	`PPLOT`
19	Rotated ellipse	$R_\phi(a \cos t, b \sin t)$	parametric	`PPLOT`
20	Archimedean spiral	$r = a + b\theta$	polar	`PPLOT/RFUN`
21	Polar rose	$r = a \cos n\theta$	polar	`PPLOT/RFUN`
22	Cardioid	$r = a(1 + \cos\theta)$	polar	`PPLOT/RFUN`
23	Logarithmic spiral	$r = ae^{b\theta}$	polar	`PPLOT/RFUN`
24	Lissajous figure	$(\sin at, \sin(bt + p))$	parametric	`PPLOT`
25	Cycloid	$(t - \sin t, 1 - \cos t)$	parametric	`PPLOT`
26	Epicycloid	rolling-circle form	parametric	`PPLOT`
27	Hypotrochoid	inside rolling-circle form	parametric	`PPLOT`
28	Superellipse	$\lvert x/a \rvert^n + \lvert y/b \rvert^n = 1$	parametric branch	`PPLOT`
29	Lemniscate	$r^2 = a^2 \cos 2\theta$	polar/domain	`PPLOT/RFUN`
30	Cassini ovals	$d_1 d_2 = b^2$	implicit	`ICONT`
31	Elliptic algebraic curve	$y^2 = x^3 - x$	two branches	`CPLOT twice`
32	Butterfly curve	$(\sin t\, q, \cos t\, q)$	parametric	`PPLOT`
33	Two-wave interference	$\sin kr_1 + \sin kr_2$	implicit	`ICONT`
34	Radial vector field	(x, y)	field	`SEG arrows`
35	Rotational vector field	$(-y, x)$	field	`SEG arrows`
36	Saddle vector field	$(x, -y)$	field	`SEG arrows`
37	Gradient field	$\nabla(x^2 + y^2)$	field	`SEG arrows`
38	Slope field	$dy/dx = x - y$	slope field	`SEG strokes`
39	Phase portrait	$(\cos t, -\sin t)$	state space	`PPLOT`
40	Implicit contour	$F(x, y) = 1.6$	implicit	`ICONT`
41	Equipotential map	$V(x, y) = C$	contours	`ICONT`
42	Streamlines	$d\mathbf{r}/ds = \hat{\mathbf{F}}$	field integration	`SEG`
43	Cantor set	middle-third removal	geometric fractal	`recursive lines`
44	Sierpiński triangle	random midpoint rule	point fractal	`chaos game`
45	Barnsley fern	four affine maps	point fractal	`weighted iteration`
46	Koch snowflake	four-segment replacement	geometric fractal	`recursive lines`
47	Fractal tree	two scaled branches	geometric fractal	`recursive lines`
48	Dragon curve	folding turn sequence	geometric fractal	`turn iteration`
49	Logistic cobweb	$x_{n+1} = rx_n(1 - x_n)$	dynamical	`CPLOT+SEG`
50	Bifurcation diagram	long-term logistic states	dynamical	`nested iteration`
51	Hénon attractor	$(1 - 1.4x^2 + y, 0.3x)$	map	`point iteration`
52	Clifford attractor	trigonometric map	map	`point iteration`
53	3-D plane	$z = ax + by + c$	wireframe	`GRID3D/ZFUN`

No.	Plot	Equation or rule	Type	Required routine
54	3-D helix	$(\cos t, \sin t, kt)$	3-D parametric	`PROJ+SEG`
55	Wireframe sphere	latitude/longitude	two-parameter	`PROJ+SEG`
56	Paraboloid	$z = x^2 + y^2$	wireframe	`GRID3D/ZFUN`
57	3-D saddle	$z = x^2 - y^2$	wireframe	`GRID3D/ZFUN`
58	Gaussian surface	$z = e^{-x^2-y^2}$	wireframe	`GRID3D/ZFUN`
59	Sinusoidal surface	$z = \sin x \cos y$	wireframe	`GRID3D/ZFUN`
60	Radial ripple	$z = \cos r$	wireframe	`GRID3D/ZFUN`
61	Sinc surface	$z = \sin r/r$	wireframe/domain	`GRID3D/ZFUN`
62	Mexican hat	$(1 - r^2)e^{-r^2/2}$	wireframe	`GRID3D/ZFUN`
63	Two-source surface	$\sin kr_1 + \sin kr_2$	wireframe	`GRID3D/ZFUN`
64	Potential well	$-1/\sqrt{r^2 + \epsilon^2}$	wireframe	`GRID3D/ZFUN`
65	Torus	two-angle parametrisation	two-parameter	`PROJ+SEG`
66	Möbius strip	half-angle strip	two-parameter	`PROJ+SEG`
67	Lorenz attractor	Lorenz ODEs	3-D dynamics	`Euler+PROJ`
68	Rössler attractor	Rössler ODEs	3-D dynamics	`Euler+PROJ`
69	Mandelbrot set	$z \leftarrow z^2 + c$	escape time	`MANDEL`
70	Julia set	fixed c, varying z_0	escape time	`MANDEL variant`
71	Burning Ship	$(\lvert\Re z\rvert + i\lvert\Im z\rvert)^2 + c$	escape time	`MANDEL variant`
72	Newton fractal	$z \leftarrow z - p(z)/p'(z)$	convergence basin	`Newton iterator`

Colophon

This handbook was authored and typeset in LaTeX. The calculator illustrations are one-bit 400×240 framebuffers: no anti-aliasing, smoothing or colour has been applied. Every calculator routine used in the text is printed in full in the appendices.